

# KUBERNETES

ARQUITETURA • WORKLOADS • OPERAÇÃO

## Do Pod à produção

Da arquitetura do cluster ao Helm: Pods, Deployments, volumes, Services, Ingress e Gateway API, autoescalonamento, agendamento, políticas, Network Policies e RBAC, com tabelas de decisão, pegadinhas e flashcards. Conferido com o Kubernetes 1.37.

Erik Nathan [eriknathan.me](https://eriknathan.me)

### CINCO BLOCOS DO GUIA

#### 01

Fundamentos  
Capítulos 1 a 3

#### 02

Workloads e cluster  
Capítulos 4 a 7

#### 03

Dados, rede e  
configuração  
Capítulos 8 a 12

#### 04

Operação e segurança  
Capítulos 13 a 17

#### 05

Helm e revisão  
Capítulos 18 a 21

# Sumário

## COMO USAR ESTE GUIA

|                                             |   |              |   |
|---------------------------------------------|---|--------------|---|
| Versões e atualizações                      | 6 | Mapa do guia | 7 |
| Método para investigar problemas no cluster | 6 |              |   |

## CAPÍTULO 1 – CONTÊINERES, RUNTIMES E O PADRÃO OCI

|       |                                                     |   |       |                                        |    |
|-------|-----------------------------------------------------|---|-------|----------------------------------------|----|
| 1.1   | Engine, runtime e as camadas por baixo do contêiner | 8 | 1.2.1 | OCI: o formato comum                   | 9  |
| 1.1.1 | Container engine                                    | 8 | 1.2.2 | CRI: como o kubelet fala com o runtime | 9  |
| 1.1.2 | Níveis de runtime                                   | 8 | 1.2.3 | Do kubelet ao processo                 | 9  |
| 1.2   | Os padrões OCI e CRI                                | 9 | 1.3   | Decisão rápida — Runtimes              | 10 |

## CAPÍTULO 2 – KUBERNETES: ORIGEM E ARQUITETURA

|       |                                               |    |       |                              |    |
|-------|-----------------------------------------------|----|-------|------------------------------|----|
| 2.1   | De onde veio o Kubernetes                     | 11 | 2.2.6 | Alta disponibilidade         | 13 |
| 2.1.1 | Borg, Omega e Kubernetes                      | 11 | 2.3   | Portas usadas pelo cluster   | 13 |
| 2.1.2 | O que o Kubernetes faz (e o que não faz)      | 11 | 2.3.1 | Control plane                | 13 |
| 2.2   | Arquitetura do cluster                        | 12 | 2.3.2 | Nós de trabalho              | 13 |
| 2.2.1 | Control plane e nós                           | 12 | 2.4   | Objetos e conceitos-chave    | 14 |
| 2.2.2 | Componentes do control plane                  | 12 | 2.4.1 | Principais objetos           | 14 |
| 2.2.3 | Componentes de cada nó                        | 12 | 2.4.2 | Anatomia de um manifesto     | 14 |
| 2.2.4 | Add-ons essenciais                            | 12 | 2.5   | Decisão rápida — Arquitetura | 14 |
| 2.2.5 | O que acontece quando você cria um Deployment | 12 |       |                              |    |

## CAPÍTULO 3 – KUBECTL E CLUSTERS LOCAIS

|       |                                     |    |       |                                            |    |
|-------|-------------------------------------|----|-------|--------------------------------------------|----|
| 3.1   | Instalar o kubectl                  | 16 | 3.4   | Primeiros comandos                         | 19 |
| 3.1.1 | Linux, macOS e Windows              | 16 | 3.4.1 | Explorar o cluster                         | 19 |
| 3.2   | Configurar o kubectl                | 16 | 3.4.2 | Primeiro Pod e primeiro Service            | 20 |
| 3.2.1 | Kubeconfig e contextos              | 16 | 3.4.3 | Imperativo e declarativo                   | 20 |
| 3.2.2 | Autocompletar e alias               | 17 | 3.4.4 | Formatos de saída e seletores              | 20 |
| 3.3   | Clusters locais e distribuições     | 17 | 3.5   | Labels, seletores e annotations            | 21 |
| 3.3.1 | Opções para estudar e para produção | 17 | 3.5.1 | Labels                                     | 21 |
| 3.3.2 | kind                                | 18 | 3.5.2 | Annotations                                | 21 |
| 3.3.3 | Minikube                            | 19 | 3.6   | Decisão rápida — kubectl e clusters locais | 21 |

## CAPÍTULO 4 – PODS

|       |                             |    |       |                                           |    |
|-------|-----------------------------|----|-------|-------------------------------------------|----|
| 4.1   | O que é um Pod              | 23 | 4.4   | Interagir com um Pod em execução          | 25 |
| 4.1.1 | Características             | 23 | 4.4.1 | exec , attach , port-forward , cp e debug | 25 |
| 4.2   | Criar e inspecionar Pods    | 23 | 4.5   | Ciclo de vida do Pod                      | 26 |
| 4.2.1 | Manifesto comentado         | 23 | 4.5.1 | Fases e estados                           | 26 |
| 4.2.2 | apply , create e replace    | 24 | 4.6   | Recursos: requests e limits               | 27 |
| 4.2.3 | Logs                        | 24 | 4.6.1 | Como funcionam                            | 27 |
| 4.3   | Pods com vários contêineres | 24 | 4.6.2 | Classes de QoS                            | 27 |
| 4.3.1 | Padrões de uso              | 24 | 4.6.3 | Testar os limites e ajustar sem recriar   | 28 |

|       |                 |    |     |                       |    |
|-------|-----------------|----|-----|-----------------------|----|
| 4.7   | Volume emptyDir | 28 | 4.8 | Decisão rápida — Pods | 29 |
| 4.7.1 | Como funciona   | 28 |     |                       |    |

## CAPÍTULO 5 – DEPLOYMENTS E REPLICASETS

|       |                              |    |       |                              |    |
|-------|------------------------------|----|-------|------------------------------|----|
| 5.1   | Deployment, ReplicaSet e Pod | 30 | 5.3.1 | RollingUpdate e Recreate     | 32 |
| 5.1.1 | A hierarquia                 | 30 | 5.4   | Rollout e rollback           | 33 |
| 5.1.2 | Manifesto comentado          | 31 | 5.4.1 | Acompanhar, pausar, voltar   | 33 |
| 5.2   | Criar, inspecionar e escalar | 32 | 5.5   | ReplicaSet diretamente       | 33 |
| 5.2.1 | Comandos do dia a dia        | 32 | 5.5.1 | Quando e por que não         | 33 |
| 5.3   | Estratégias de atualização   | 32 | 5.6   | Decisão rápida — Deployments | 34 |

## CAPÍTULO 6 – DAEMONSETS E PROBES

|       |                        |    |       |                                      |    |
|-------|------------------------|----|-------|--------------------------------------|----|
| 6.1   | DaemonSet              | 35 | 6.2.2 | Mecanismos de teste                  | 37 |
| 6.1.1 | Para que serve         | 35 | 6.2.3 | Parâmetros                           | 38 |
| 6.1.2 | Comandos e atualização | 37 | 6.2.4 | Exemplo completo                     | 38 |
| 6.2   | Probes                 | 37 | 6.2.5 | Boas práticas e armadilhas           | 38 |
| 6.2.1 | Os três tipos          | 37 | 6.3   | Decisão rápida — DaemonSets e probes | 39 |

## CAPÍTULO 7 – INSTALANDO UM CLUSTER COM KUBEADM

|       |                                     |    |       |                             |    |
|-------|-------------------------------------|----|-------|-----------------------------|----|
| 7.1   | Formas de ter um cluster            | 40 | 7.3.1 | kubeadm init e kubeadm join | 42 |
| 7.1.1 | Comparação                          | 40 | 7.3.2 | Kubeconfig do administrador | 43 |
| 7.2   | Preparar os nós                     | 40 | 7.3.3 | Escolher o CNI              | 43 |
| 7.2.1 | Requisitos                          | 40 | 7.4   | Operar o cluster            | 43 |
| 7.2.2 | Módulos do kernel, sysctl e runtime | 41 | 7.4.1 | Manutenção do dia a dia     | 43 |
| 7.2.3 | Pacotes do Kubernetes               | 41 | 7.5   | Decisão rápida — Instalação | 44 |
| 7.3   | Criar o cluster                     | 42 |       |                             |    |

## CAPÍTULO 8 – VOLUMES E ARMAZENAMENTO PERSISTENTE

|       |                                          |    |       |                                    |    |
|-------|------------------------------------------|----|-------|------------------------------------|----|
| 8.1   | Tipos de volume                          | 45 | 8.3.1 | Vínculo estático (PV criado à mão) | 46 |
| 8.1.1 | Efêmeros e persistentes                  | 45 | 8.3.2 | Vínculo dinâmico (o caso comum)    | 47 |
| 8.2   | StorageClass                             | 45 | 8.3.3 | Modos de acesso                    | 47 |
| 8.2.1 | O que define                             | 45 | 8.3.4 | Estados e ciclo de vida            | 47 |
| 8.3   | PersistentVolume e PersistentVolumeClaim | 46 | 8.4   | Decisão rápida — Armazenamento     | 48 |

## CAPÍTULO 9 – SERVICES, ENDPOINTS e DNS

|       |                                        |    |       |                           |    |
|-------|----------------------------------------|----|-------|---------------------------|----|
| 9.1   | Como um Service funciona               | 49 | 9.2.2 | Opções úteis              | 50 |
| 9.1.1 | Seletores, EndpointSlices e kube-proxy | 49 | 9.3   | DNS dentro do cluster     | 51 |
| 9.1.2 | Portas                                 | 49 | 9.3.1 | Nomes                     | 51 |
| 9.2   | Tipos de Service                       | 50 | 9.4   | Decisão rápida — Services | 51 |
| 9.2.1 | Visão geral                            | 50 |       |                           |    |

## CAPÍTULO 10 – STATEFULSETS

|        |                                |    |        |                               |    |
|--------|--------------------------------|----|--------|-------------------------------|----|
| 10.1   | Garantias do StatefulSet       | 52 | 10.3   | Escala, atualização e remoção | 54 |
| 10.1.1 | Deployment × StatefulSet       | 52 | 10.3.1 | Comportamentos                | 54 |
| 10.2   | Criar um StatefulSet           | 53 | 10.4   | Decisão rápida — StatefulSets | 54 |
| 10.2.1 | Service headless e StatefulSet | 53 |        |                               |    |

## CAPÍTULO 11 – CONFIGMAPS E SECRETS

|        |                           |    |        |                                          |    |
|--------|---------------------------|----|--------|------------------------------------------|----|
| 11.1   | ConfigMaps                | 55 | 11.2.2 | Tipos                                    | 56 |
| 11.1.1 | Criar                     | 55 | 11.2.3 | Exemplo: nginx com HTTPS                 | 57 |
| 11.1.2 | Consumir no Pod           | 56 | 11.3   | Segredos em cofres externos              | 58 |
| 11.2   | Secrets                   | 56 | 11.3.1 | External Secrets Operator                | 58 |
| 11.2.1 | Base64 não é criptografia | 56 | 11.4   | Decisão rápida – Configuração e segredos | 59 |

## CAPÍTULO 12 – INGRESS, GATEWAY API E CERTIFICADOS

|        |                          |    |        |                                     |    |
|--------|--------------------------|----|--------|-------------------------------------|----|
| 12.1   | Ingress                  | 61 | 12.2.1 | Modelo                              | 62 |
| 12.1.1 | Componentes              | 61 | 12.3   | Certificados com cert-manager       | 63 |
| 12.1.2 | ingress-nginx aposentado | 62 | 12.3.1 | Como funciona                       | 63 |
| 12.2   | Gateway API              | 62 | 12.4   | Decisão rápida – Entrada de tráfego | 64 |

## CAPÍTULO 13 – METRICS SERVER E AUTOESCALONAMENTO

|        |                         |    |        |                                  |    |
|--------|-------------------------|----|--------|----------------------------------|----|
| 13.1   | Metrics Server          | 66 | 13.2.2 | Algoritmo                        | 67 |
| 13.1.1 | Instalação e uso        | 66 | 13.2.3 | Teste de carga                   | 68 |
| 13.2   | HorizontalPodAutoscaler | 67 | 13.2.4 | Outros escaladores               | 68 |
| 13.2.1 | Manifesto               | 67 | 13.3   | Decisão rápida – Autoescalamento | 68 |

## CAPÍTULO 14 – AGENDAMENTO: TAINTS, TOLERATIONS E AFINIDADE

|        |                      |    |        |                                             |    |
|--------|----------------------|----|--------|---------------------------------------------|----|
| 14.1   | Labels nos nós       | 70 | 14.3   | Afinidade e antiafinidade                   | 71 |
| 14.1.1 | Organizar o cluster  | 70 | 14.3.1 | nodeAffinity                                | 71 |
| 14.2   | Taints e tolerations | 70 | 14.3.2 | podAffinity, podAntiAffinity e distribuição | 72 |
| 14.2.1 | Efeitos              | 70 | 14.3.3 | Prioridade e interrupções                   | 72 |
|        |                      |    | 14.4   | Decisão rápida – Agendamento                | 72 |

## CAPÍTULO 15 – POLÍTICAS E CONTROLE DE ADMISSÃO

|        |                           |    |        |                            |    |
|--------|---------------------------|----|--------|----------------------------|----|
| 15.1   | O caminho de um pedido    | 74 | 15.3.1 | Regras em CEL, sem webhook | 75 |
| 15.1.1 | Etapas                    | 74 | 15.4   | Kyverno                    | 75 |
| 15.2   | Pod Security Admission    | 74 | 15.4.1 | O que faz                  | 75 |
| 15.2.1 | Padrões por namespace     | 74 | 15.4.2 | Tipos de política          | 76 |
| 15.3   | ValidatingAdmissionPolicy | 75 | 15.5   | Decisão rápida – Políticas | 77 |

## CAPÍTULO 16 – NETWORK POLICIES

|        |                     |    |        |                                   |    |
|--------|---------------------|----|--------|-----------------------------------|----|
| 16.1   | Como funcionam      | 79 | 16.2.1 | Negar tudo e liberar o necessário | 80 |
| 16.1.1 | Regras fundamentais | 79 | 16.2.2 | Testar                            | 81 |
| 16.1.2 | Seletores: E e OU   | 79 | 16.3   | Decisão rápida – Network Policies | 81 |
| 16.2   | Receitas            | 80 |        |                                   |    |

## CAPÍTULO 17 – RBAC E CONTROLE DE ACESSO

|        |                                    |    |        |                       |    |
|--------|------------------------------------|----|--------|-----------------------|----|
| 17.1   | Identities                         | 83 | 17.2   | Roles e bindings      | 84 |
| 17.1.1 | Usuários, grupos e ServiceAccounts | 83 | 17.2.1 | Anatomia de uma regra | 84 |
| 17.1.2 | Criar um usuário com certificado   | 84 | 17.2.2 | Cuidados de segurança | 86 |
| 17.1.3 | ServiceAccounts e tokens           | 84 | 17.3   | Decisão rápida – RBAC | 86 |

## CAPÍTULO 18 – HELM

|        |                        |    |        |                       |    |
|--------|------------------------|----|--------|-----------------------|----|
| 18.1   | Instalar e usar charts | 87 | 18.2.1 | Estrutura             | 88 |
| 18.1.1 | Instalação             | 87 | 18.2.2 | Templates             | 89 |
| 18.1.2 | Comandos do dia a dia  | 87 | 18.2.3 | Validar e publicar    | 90 |
| 18.2   | Criar um chart         | 88 | 18.3   | Decisão rápida — Helm | 90 |

## CAPÍTULO 19 – PEGADINHAS E INFORMAÇÕES DESATUALIZADAS

|      |                                                 |    |      |                             |    |
|------|-------------------------------------------------|----|------|-----------------------------|----|
| 19.1 | Afirmções falsas clássicas                      | 92 | 19.3 | Pares que mais se confundem | 94 |
| 19.2 | Informações desatualizadas em materiais antigos | 93 |      |                             |    |

## CAPÍTULO 20 – REFERÊNCIA RÁPIDA DE COMANDOS

|      |                                    |    |      |                                   |    |
|------|------------------------------------|----|------|-----------------------------------|----|
| 20.1 | Contexto, descoberta e diagnóstico | 96 | 20.3 | Rollouts, nós e autoescalonamento | 97 |
| 20.2 | Criar e alterar                    | 96 | 20.4 | Ferramentas do ecossistema        | 97 |

## CAPÍTULO 21 – AUTOTESTE – FLASHCARDS DE REVISÃO

|      |                                       |     |      |                                                 |     |
|------|---------------------------------------|-----|------|-------------------------------------------------|-----|
| 21.1 | Runtimes e arquitetura                | 98  | 21.6 | Cluster, armazenamento e Services               | 104 |
| 21.2 | kubectl e primeiros passos            | 99  | 21.7 | Configuração, entrada de tráfego e certificados | 105 |
| 21.3 | Pods                                  | 100 | 21.8 | Autoescalonamento, agendamento e políticas      | 107 |
| 21.4 | Deployments, ReplicaSets e DaemonSets | 102 | 21.9 | Network Policies, RBAC e Helm                   | 108 |
| 21.5 | Probes                                | 103 |      |                                                 |     |

# Como usar este guia

- Os capítulos seguem o caminho de quem está aprendendo Kubernetes: primeiro os contêineres e os runtimes por baixo do cluster, depois a arquitetura, o `kubectl` e os clusters locais, e então os objetos, começando pelo Pod.
- Cada capítulo de conteúdo termina com uma tabela de **Decisão rápida**: situações típicas, a abordagem recomendada e o erro mais comum.
- Os manifestos YAML e os comandos podem ser executados em um cluster local (kind ou Minikube, capítulo 3). Onde o resultado depende da versão do cluster, o guia indica.
- O capítulo **Pegadinhas e informações desatualizadas** reúne os erros de conceito mais frequentes e o que mudou em relação a cursos e tutoriais mais antigos.
- O **Autoteste**, no fim, tem flashcards que escondem a resposta até você clicar.

## Versões e atualizações

Os comportamentos deste guia foram conferidos na documentação oficial (kubernetes.io) em **setembro de 2026**. Nessa data, a versão mais recente era o **Kubernetes 1.37**, e as versões com suporte eram **1.35, 1.36 e 1.37**.

| Item                                              | Situação em set/2026                                                                                                                                                                             |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ritmo de lançamentos                              | Cerca de <b>3 versões menores por ano</b> (uma a cada ~4 meses)                                                                                                                                  |
| Suporte de cada versão menor                      | Cerca de <b>1 ano</b> de correções; o projeto mantém as <b>3 versões mais recentes</b>                                                                                                           |
| Diferença de versão do <code>kubectl</code>       | O <code>kubectl</code> pode estar <b>uma versão menor acima ou abaixo</b> do cluster (um <code>kubectl</code> 1.37 fala com clusters 1.36, 1.37 e 1.38)                                          |
| Docker Engine como runtime                        | O <b>dockershim</b> foi removido na <b>1.24</b> (2022). Os nós usam containerd ou CRI-O; o Docker só com o adaptador <code>cri-dockerd</code> . Imagens criadas com Docker continuam funcionando |
| Sidecars nativos                                  | <b>Estáveis desde a 1.33</b> : <code>initContainers</code> com <code>restartPolicy: Always</code>                                                                                                |
| Redimensionar CPU e memória de um Pod em execução | <b>Estável desde a 1.35</b> ( <i>in-place resize</i> )                                                                                                                                           |
| Recursos declarados no nível do Pod               | <b>Beta</b> na 1.37 ( <code>PodLevelResources</code> )                                                                                                                                           |
| Modo padrão do kube-proxy                         | <code>iptables</code> ; o <code>nftables</code> está disponível e deve virar o padrão em uma versão futura                                                                                       |

*Materiais publicados antes de 2024 costumam mostrar saídas com `docker://` nos IDs de contêiner, clusters 1.24 ou anteriores e versões antigas do kind e do Minikube. Os conceitos continuam valendo; os detalhes, nem sempre.*

## Método para investigar problemas no cluster

1. **Olhe o estado real**: `kubectl get` mostra o resumo; `kubectl describe` mostra a configuração e, no fim, os **eventos**, que quase sempre explicam o problema (imagem que não baixa, falta de recursos, falha de probe).
2. **Leia os logs do contêiner**, e os da execução anterior se ele reiniciou: `kubectl logs <pod> --previous`.
3. **Suba na hierarquia**: um Pod é criado por um ReplicaSet, que é criado por um Deployment. Se o Pod está certo mas não é recriado, o problema está no controlador acima dele.
4. **Compare o desejado com o observado**: o Kubernetes é declarativo. A pergunta é sempre "o que eu pedi (`spec`) e o que o cluster conseguiu (`status`)?".

**5. Não conserte o Pod à mão:** Pods são descartáveis. A correção vai no manifesto, que é reaplicado.

## Mapa do guia

| Bloco                      | Capítu-<br>los | O que você deve dominar ao final                                                                                                   |
|----------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Fundamentos                | 1 a 3          | Engine, runtime, OCI e CRI; arquitetura do cluster; <code>kubectl</code> , <code>kubeconfig</code> e clusters locais               |
| Workloads e cluster        | 4 a 7          | Pods, recursos e probes; Deployments, ReplicaSets e DaemonSets; instalar um cluster com <code>kubeadm</code>                       |
| Dados, rede e configuração | 8 a 12         | Volumes e StorageClasses; Services e DNS; StatefulSets; ConfigMaps, Secrets e cofres externos; Ingress, Gateway API e cert-manager |
| Operação e segurança       | 13 a 17        | Metrics Server e HPA; taints, tolerations e afinidade; políticas de admissão e Kyverno; Network Policies; RBAC                     |
| Pacotes e revisão          | 18 a 21        | Helm; pegadinhas e informações desatualizadas; referência de comandos; flashcards                                                  |

# Contêineres, Runtimes e o Padrão OCI

**Ideia central:** o Kubernetes **não executa contêineres sozinho**. Em cada nó, o **kubelet** pede a um **container runtime** (containerd ou CRI-O) que crie os contêineres, por meio de uma interface padronizada, a **CRI**. O runtime, por sua vez, usa um executor de baixo nível (runc ou crun) que segue o padrão **OCI**. Entender essas camadas explica boa parte das mensagens de erro e das mudanças recentes do ecossistema.

## 1.1 Engine, runtime e as camadas por baixo do contêiner

### 1.1.1 Container engine

Um **container engine** é a ferramenta completa que o usuário opera: baixa e constrói imagens, cria redes e volumes, oferece uma CLI e uma API e, no fim, pede a um runtime que execute o contêiner.

| Ferramenta           | O que é                                                                | Observação                                                                                                               |
|----------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Docker Engine</b> | Engine completo (CLI, API, build, redes, volumes)                      | Usa o <b>containerd</b> como runtime e o <b>runc</b> como executor                                                       |
| <b>Podman</b>        | Engine sem daemon, rootless por padrão, CLI compatível com a do Docker | Usa o <b>conmon</b> para supervisionar e o <b>crun</b> ou o <b>runc</b> para executar                                    |
| <b>nerdctl</b>       | CLI no estilo Docker para usar o containerd diretamente                | Útil para depurar nós que só têm containerd                                                                              |
| <b>CRI-O</b>         | Runtime feito sob medida para o Kubernetes                             | Não é um engine de uso geral: não tem comando para o usuário construir imagens nem gerenciar contêineres fora do cluster |

- Em um nó Kubernetes **não é preciso ter o Docker**. O kubelet fala direto com o runtime, e as ferramentas de inspeção do nó são o `crictl` (para qualquer runtime CRI) ou o `nerdctl`.

### 1.1.2 Níveis de runtime

| Nível                            | O que faz                                                                                                                                            | Exemplos                                                                                               |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Baixo nível</b> (OCI runtime) | Recebe um diretório com o sistema de arquivos e um <code>config.json</code> , cria namespaces e cgroups e inicia o processo. Não sabe baixar imagens | <b>runc</b> (referência da OCI, em Go), <b>crun</b> (em C, mais leve e rápido), <b>youki</b> (em Rust) |
| <b>Alto nível</b> (CRI runtime)  | Baixa e armazena imagens, prepara o sistema de arquivos, gerencia o ciclo de vida dos contêineres e chama o runtime de baixo nível                   | <b>containerd</b> , <b>CRI-O</b>                                                                       |
| <b>Sandbox</b>                   | Coloca uma camada entre o contêiner e o kernel do host, interceptando as chamadas de sistema                                                         | <b>gVisor</b> ( <code>runsc</code> ), que implementa um "kernel" em espaço de usuário                  |
| <b>Virtualizado</b>              | Roda cada Pod dentro de uma <b>microVM</b> com kernel próprio                                                                                        | <b>Kata Containers</b> , <b>Firecracker</b> (via Kata)                                                 |

- Runtimes sandbox e virtualizados trocam um pouco de desempenho por **isolamento mais forte**, útil para código não confiável e ambientes multi-tenant. No Kubernetes, eles são escolhidos por Pod com uma **RuntimeClass** (`spec.runtimeClassName`).

## 1.2 Os padrões OCI e CRI

### 1.2.1 OCI: o formato comum

A **Open Container Initiative** foi criada em **2015**, sob a Linux Foundation, com a participação de Docker, CoreOS, Google, IBM, Microsoft, Red Hat, VMware e outras empresas. Ela mantém três especificações:

| Especificação            | Define                                           | Por que importa                                                                      |
|--------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Image spec</b>        | Formato da imagem (manifest, config, camadas)    | Uma imagem criada com Docker, Podman ou BuildKit roda em qualquer runtime compatível |
| <b>Runtime spec</b>      | Como executar um contêiner a partir de um bundle | Permite trocar runc por crun, gVisor ou Kata sem mudar a imagem                      |
| <b>Distribution spec</b> | API de registries                                | Docker Hub, GHCR, ECR, Harbor falam a mesma língua                                   |

- O **runc** nasceu da biblioteca libcontainer, que o Docker doou à OCI, e é o runtime de referência da especificação.

### 1.2.2 CRI: como o kubelet fala com o runtime

- A **Container Runtime Interface** é uma API gRPC definida pelo Kubernetes. O kubelet chama operações como "crie o sandbox do Pod", "baixe a imagem", "inicie o contêiner", e o runtime responde. Qualquer runtime que implemente a CRI v1 serve.
- **Dockershim**: o Docker Engine não implementa a CRI. Por anos o kubelet carregou um adaptador interno para ele, o dockershim, que foi **removido na versão 1.24**. Quem ainda quer Docker Engine nos nós usa o adaptador externo **cri-dockerd** (mantido pela Mirantis). A grande maioria dos clusters usa containerd ou CRI-O.

| Runtime                     | Socket CRI padrão                      |
|-----------------------------|----------------------------------------|
| containerd                  | unix:///run/containerd/containerd.sock |
| CRI-O                       | unix:///run/crio/crio.sock             |
| Docker Engine + cri-dockerd | unix:///run/cri-dockerd.sock           |

### 1.2.3 Do kubelet ao processo

1. O **kubelet** recebe um Pod designado ao seu nó.
  2. Chama o runtime pela **CRI** para criar o **sandbox do Pod** (o namespace de rede compartilhado, mantido pelo contêiner "pause").
  3. O **plugin CNI** configura a interface de rede e o IP do Pod.
  4. O runtime **baixa as imagens** (se necessário) e prepara o sistema de arquivos de cada contêiner.
  5. O runtime chama o **runc** (ou outro OCI runtime), que cria namespaces e cgroups e inicia o processo.
  6. O kubelet acompanha o estado dos contêineres e o reporta ao API server.
- **Driver de cgroup**: o kubelet e o runtime precisam usar o **mesmo** driver. Em distribuições com systemd e cgroup v2, o recomendado é **systemd**. Driver diferente entre os dois é uma causa clássica de nós instáveis em clusters montados à mão.

### 1.3 Decisão rápida — Runtimes

| Situação                                                             | Abordagem recomendada                                                                | Erro comum                                                |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Escolher o runtime de um cluster novo                                | containerd ou CRI-O                                                                  | Instalar Docker Engine nos nós "porque sempre foi assim"  |
| Inspecionar contêineres em um nó sem Docker                          | <code>crictl ps</code> , <code>crictl logs</code> , <code>crictl images</code>       | Procurar o comando <code>docker</code> no nó              |
| Imagens criadas com Docker vão rodar no cluster?                     | Sim: seguem o padrão OCI                                                             | Reescrever Dockerfiles por causa da remoção do dockershim |
| Rodar código não confiável com isolamento forte                      | RuntimeClass com gVisor ou Kata Containers                                           | Confiar só no isolamento padrão do runc                   |
| Nó com Pods reiniciando sem motivo aparente em cluster montado à mão | Conferir se kubelet e runtime usam o mesmo driver de cgroup ( <code>systemd</code> ) | Reinstalar o cluster inteiro                              |

# Kubernetes: Origem e Arquitetura

**Ideia central:** o Kubernetes é um sistema **declarativo**. Você descreve o estado desejado (três réplicas desta imagem, expostas nesta porta), o **API server** guarda essa descrição no **etcd**, e vários **controladores** trabalham em laços contínuos para aproximar o estado real do desejado. Todo o resto (scheduler, kubelet, kube-proxy) são peças desse ciclo.

## 2.1 De onde veio o Kubernetes

### 2.1.1 Borg, Omega e Kubernetes

- O Google executa praticamente tudo em contêineres há mais de duas décadas. Para isso, criou três sistemas de gerenciamento de clusters:

| Sistema           | Contexto                                                                                                            | Legado                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Borg</b>       | Primeiro sistema, interno. Unificou serviços de longa duração e jobs em lote, antes tratados por sistemas separados | Continua sendo o principal gerenciador interno do Google |
| <b>Omega</b>      | Sucessor interno, com arquitetura mais consistente e estado central compartilhado                                   | Muitas ideias voltaram para o Borg                       |
| <b>Kubernetes</b> | Criado em <b>2014</b> , já como <b>open source</b> , com foco na experiência de quem desenvolve e opera aplicações  | Doado à <b>CNCF</b> em 2015, junto com a versão 1.0      |

- O nome vem do grego e significa "**timoneiro**" (piloto de navio), daí o leme no logotipo. A abreviação **k8s** segue a convenção de numerônimos: **k** + 8 letras + **s**, como **i18n** para *internationalization*.
- Ideias que vieram do Borg: agrupar contêineres em Pods, rótulos (labels) para selecionar objetos, um IP por Pod e controladores que reconciliam estado.

### 2.1.2 O que o Kubernetes faz (e o que não faz)

| Faz                                                          | Não faz (por conta própria)                                                          |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Agenda contêineres nos nós conforme recursos e restrições    | Construir imagens ou fazer CI/CD                                                     |
| Recria contêineres e Pods que falham ( <i>self-healing</i> ) | Oferecer banco de dados, fila ou cache prontos                                       |
| Escala réplicas, manual ou automaticamente                   | Monitorar e registrar logs de forma completa (precisa de ferramentas do ecossistema) |
| Descoberta de serviços (DNS) e balanceamento entre Pods      | Garantir que a aplicação seja resiliente: ela precisa tratar falhas e sinais         |
| Atualizações graduais e rollback                             | Substituir boas práticas de segurança da aplicação e das imagens                     |
| Entrega de configuração e segredos                           | Gerenciar a infraestrutura física do cluster                                         |
| Orquestração de volumes persistentes                         |                                                                                      |

## 2.2 Arquitetura do cluster

### 2.2.1 Control plane e nós

Um cluster tem dois papéis:

- **Control plane** (antes chamado de *master*): toma as decisões do cluster e guarda o estado.
- **Nós de trabalho** (*workers*): executam os Pods das aplicações.

Em clusters gerenciados (EKS, GKE, AKS), o provedor opera o control plane e você só vê os nós de trabalho.

### 2.2.2 Componentes do control plane

| Componente                      | Papel                                                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kube-apiserver</b>           | A porta de entrada do cluster. Expõe a <b>API REST</b> (JSON ou Protobuf sobre HTTPS), autentica, autoriza, valida e grava os objetos. <b>Todos</b> os componentes, e o <code>kubectl</code> , conversam com o cluster por ele. É o <b>único</b> que fala com o <code>etcd</code> |
| <b>etcd</b>                     | Banco <b>chave-valor distribuído</b> e consistente (consenso Raft) onde fica todo o estado do cluster: especificações, status, configurações e segredos. Pode rodar nos nós de control plane ( <i>stacked</i> ) ou em um cluster próprio ( <i>external</i> )                      |
| <b>kube-scheduler</b>           | Observa Pods sem nó definido e escolhe o nó de cada um. Primeiro <b>filtra</b> os nós viáveis (recursos solicitados, taints, afinidade, portas, volumes) e depois <b>pontua</b> os restantes. Decide com base nos <b>requests</b> declarados, não no uso real de CPU e memória    |
| <b>kube-controller-manager</b>  | Executa os <b>controladores</b> (Deployment, ReplicaSet, Node, Job, EndpointSlice, ServiceAccount e outros). Cada um observa um tipo de objeto e age para que o estado real chegue ao desejado                                                                                    |
| <b>cloud-controller-manager</b> | Opcional. Integra o cluster ao provedor de nuvem: cria load balancers, rotas e verifica se as máquinas ainda existem                                                                                                                                                              |

### 2.2.3 Componentes de cada nó

| Componente               | Papel                                                                                                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kubelet</b>           | Agente que roda em <b>todos os nós</b> (inclusive nos de control plane). Recebe os Pods designados ao nó, pede ao runtime que os execute, roda as probes, monta volumes e reporta o estado ao API server  |
| <b>kube-proxy</b>        | Implementa os <b>Services</b> no nó: programa regras (iptables, IPVS ou nftables) que levam o tráfego do IP virtual do Service para os Pods. Alguns plugins de rede, como o Cilium, o substituem por eBPF |
| <b>Container runtime</b> | containerd ou CRI-O (capítulo 1)                                                                                                                                                                          |

### 2.2.4 Add-ons essenciais

- **CNI (plugin de rede)**: dá IP a cada Pod e conecta Pods entre nós. Sem ele, os nós ficam `NotReady`. Exemplos: Calico, Cilium, Flannel, os plugins das nuvens (VPC CNI da AWS). O kube-proxy **não** faz a rede dos Pods; quem faz é o CNI.
- **CoreDNS**: DNS interno. Resolve nomes como `meu-servico.meu-namespace.svc.cluster.local`.
- **metrics-server**: coleta uso de CPU e memória para o `kubectl top` e o autoescalonamento.
- **Ingress controller ou Gateway API**: entrada HTTP/HTTPS no cluster.

### 2.2.5 O que acontece quando você cria um Deployment

1. O `kubectl` envia o manifesto ao **API server**, que autentica, autoriza, valida e grava o Deployment no **etcd**.
2. O **controlador de Deployment** percebe o objeto novo e cria um **ReplicaSet**.

3. O **controlador de ReplicaSet** percebe que faltam Pods e cria os objetos **Pod**, ainda sem nó.
4. O **scheduler** escolhe um nó para cada Pod e grava essa decisão (o *binding*).
5. O **kubelet** do nó escolhido vê o Pod, chama o runtime pela CRI e o CNI configura a rede.
6. O kubelet reporta o status (`Running`, `Ready`). Se o contêiner cair, o kubelet o reinicia; se o Pod ou o nó sumir, o ReplicaSet cria outro Pod.

- Nenhum componente dá ordens diretas a outro: todos **observam** o API server e agem sobre o que mudou. Esse modelo de laços de reconciliação é o que torna o cluster resistente a falhas.

### 2.2.6 Alta disponibilidade

- Para estudo**, um cluster de **um nó** (control plane e workloads juntos) é suficiente.
- Em produção**, o control plane precisa de **pelo menos 3 nós** para que o etcd mantenha o quórum com a perda de um deles (a mesma lógica de quórum do Raft:  $N \text{ nós toleram } (N - 1) / 2 \text{ falhas}$ ). Os nós de trabalho são dimensionados pela carga, com no mínimo dois para que as aplicações sobrevivam à perda de um nó.
- Por padrão, os nós de control plane têm um **taint** que impede Pods comuns de rodarem neles.

## 2.3 Portas usadas pelo cluster

### 2.3.1 Control plane

| Protocolo | Porta            | Uso                                         | Quem acessa                        |
|-----------|------------------|---------------------------------------------|------------------------------------|
| TCP       | <b>6443</b>      | API server                                  | Todos (usuários, nós, componentes) |
| TCP       | <b>2379–2380</b> | etcd (clientes e comunicação entre membros) | kube-apiserver, etcd               |
| TCP       | <b>10250</b>     | API do kubelet                              | O próprio nó e o control plane     |
| TCP       | <b>10259</b>     | kube-scheduler                              | O próprio nó                       |
| TCP       | <b>10257</b>     | kube-controller-manager                     | O próprio nó                       |

### 2.3.2 Nós de trabalho

| Protocolo | Porta              | Uso                       | Quem acessa                    |
|-----------|--------------------|---------------------------|--------------------------------|
| TCP       | <b>10250</b>       | API do kubelet            | O próprio nó e o control plane |
| TCP       | <b>10256</b>       | kube-proxy (health check) | O próprio nó e load balancers  |
| TCP e UDP | <b>30000–32767</b> | Services do tipo NodePort | Todos                          |

- Todas as portas podem ser alteradas. É comum colocar o API server atrás de um load balancer na porta 443, mantendo 6443 nos nós.
- Além dessas, o **plugin CNI** usa as suas próprias portas entre nós (por exemplo, VXLAN em 4789/UDP no Flannel e no Calico, BGP em 179/TCP no Calico). Consulte a documentação do plugin escolhido.

## 2.4 Objetos e conceitos-chave

### 2.4.1 Principais objetos

| Objeto                                          | Para que serve                                                                                        |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Pod</b>                                      | Menor unidade que o Kubernetes agenda: um ou mais contêineres que compartilham rede e volumes         |
| <b>ReplicaSet</b>                               | Mantém um número fixo de Pods idênticos em execução                                                   |
| <b>Deployment</b>                               | Gerencia ReplicaSets para fazer atualizações graduais e rollback de aplicações sem estado             |
| <b>DaemonSet</b>                                | Garante um Pod em cada nó (ou em um grupo de nós)                                                     |
| <b>StatefulSet</b>                              | Pods com identidade e armazenamento estáveis (bancos, filas)                                          |
| <b>Job / CronJob</b>                            | Tarefas que terminam; o CronJob as agenda periodicamente                                              |
| <b>Service</b>                                  | Nome e IP estáveis para um grupo de Pods, com balanceamento (tipos ClusterIP, NodePort, LoadBalancer) |
| <b>Namespace</b>                                | Divide o cluster em espaços lógicos para organizar objetos, permissões e cotas                        |
| <b>ConfigMap / Secret</b>                       | Configuração e dados sensíveis entregues aos Pods                                                     |
| <b>PersistentVolume / PersistentVolumeClaim</b> | Armazenamento que sobrevive aos Pods                                                                  |

- O Kubernetes **não gerencia contêineres diretamente**: tudo é feito por meio de Pods, que por sua vez costumam ser criados por controladores (Deployment, StatefulSet, DaemonSet, Job).

### 2.4.2 Anatomia de um manifesto

Todo objeto tem a mesma estrutura:

```
apiVersion: apps/v1      # grupo/versão da API do tipo de objeto
kind: Deployment         # tipo do objeto
metadata:                # identificação
  name: web
  namespace: default
  labels:
    app: web
spec:                   # estado DESEJADO, escrito por você
  replicas: 3
  # ...
status:                # estado OBSERVADO, escrito pelo cluster (não se escreve no manifesto)
  readyReplicas: 3
```

- `apiVersion`: objetos do núcleo usam só `v1` (Pod, Service, ConfigMap); os demais usam `grupo/versão` (`apps/v1` para Deployment, `batch/v1` para Job). `kubectl api-resources` lista todos os tipos, e `kubectl explain deployment.spec.strategy` documenta cada campo.
- Labels** são pares chave-valor usados para **selecionar** objetos: um Service encontra seus Pods, e um Deployment reconhece os seus, por meio de seletores de labels. **Annotations** guardam informações que não servem para seleção (descrições, configurações de ferramentas).

## Decisão rápida — Arquitetura

## 2.5

| Situação                                                     | Abordagem recomendada                                                            | Erro comum                                          |
|--------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------|
| Control plane de produção                                    | 3 nós (ou serviço gerenciado)                                                    | Um único nó de control plane                        |
| Nós aparecem <code>NotReady</code> logo após criar o cluster | Instalar o plugin CNI                                                            | Reiniciar o kubelet repetidamente                   |
| Pod fica <code>Pending</code> com nós "ociosos"              | Conferir os <b>requests</b> e os eventos do scheduler                            | Supor que o scheduler olha o uso real               |
| Liberar acesso ao cluster em um firewall                     | 6443 para o API server; 10250 entre control plane e nós; portas do CNI entre nós | Abrir 2379 (etcd) para fora do control plane        |
| Descobrir campos de um objeto                                | <code>kubectl explain &lt;tipo&gt;.&lt;campo&gt;</code>                          | Copiar YAML de blogs sem saber o que cada campo faz |
| Onde fica o estado do cluster?                               | No etcd, acessado só pelo API server                                             | Editar arquivos nos nós                             |
| Quem cria os Pods de um Deployment?                          | O controlador de ReplicaSet, a partir do ReplicaSet criado pelo Deployment       | Achar que o scheduler cria Pods                     |

## kubectl e Clusters Locais

**Ideia central:** o `kubectl` é um cliente da API do Kubernetes. Ele lê o `kubeconfig` para saber **qual cluster**, **qual usuário** e **qual namespace** usar. Para aprender, um cluster local com **kind** ou **Minikube** sobe em minutos e pode ser apagado sem cerimônia.

### 3.1 Instalar o kubectl

#### 3.1.1 Linux, macOS e Windows

```
# Linux (x86-64; para ARM troque amd64 por arm64)
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"
echo "$(cat kubectl.sha256) kubectl" | sha256sum --check      # deve imprimir "kubectl: OK"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# macOS (Homebrew; sem sudo)
brew install kubectl

# Windows
winget install -e --id Kubernetes.kubectl      # ou: choco install kubernetes-cli

kubectl version --client
```

- **Confira o checksum** quando baixar o binário: é a garantia de que ele não foi corrompido nem adulterado.
- **Nunca rode o Homebrew com `sudo`**: ele não precisa e passa a criar arquivos com dono errado.
- **Versão**: use um `kubectl` de no máximo **uma versão menor de diferença** do cluster. `kubectl version` (sem `--client`) mostra as duas versões e avisa quando a diferença é grande demais.
- Nos repositórios de pacotes das distribuições (apt, dnf), use os repositórios oficiais `pkgs.k8s.io`, que têm um repositório por versão menor. Os antigos `apt.kubernetes.io` e `yum.kubernetes.io` foram desativados em 2024.

### 3.2 Configurar o kubectl

#### 3.2.1 Kubeconfig e contextos

- O `kubeconfig` fica em `~/.kube/config` (ou nos arquivos da variável `KUBECONFIG`, separados por `:`, que são mesclados). Ele tem três listas: **clusters** (endereço do API server e CA), **users** (credenciais) e **contexts** (combinação de cluster + usuário + namespace padrão).

```
kubectl config get-contexts           # lista; * marca o atual
kubectl config current-context
kubectl config use-context kind-estudo # troca de cluster
kubectl config set-context --current --namespace=loja # namespace padrão do contexto
kubectl cluster-info                  # endereço do API server e do CoreDNS
```

- Ferramentas como kind, Minikube e as CLIs das nuvens (`aws eks update-kubeconfig`, `gcloud container clusters get-credentials`) **adicionam contextos** ao seu kubeconfig automaticamente.
- O kubeconfig contém **credenciais do cluster**: trate como segredo e não versione.
- Os utilitários **kubectx** e **kubens** (instaláveis pelo gerenciador de plugins **krew**) trocam de contexto e de namespace com menos digitação.

### 3.2.2 Autocompletar e alias

```
# Bash (requer o pacote bash-completion)
source <(kubectl completion bash)
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc

# Zsh
echo 'source <(kubectl completion zsh)' >> ~/.zshrc
echo 'alias k=kubectl' >> ~/.zshrc
```

- No Bash, o `complete -o default -F __start_kubectl k` faz o autocompletar funcionar também com o alias `k`. No Zsh, o completar do alias já funciona.
- Também existem `kubectl completion fish` e `kubectl completion powershell`.

## 3.3 Clusters locais e distribuições

### 3.3.1 Opções para estudar e para produção

| Ferramenta                                  | Como roda                                                                      | Produção?  | Quando usar                                                            |
|---------------------------------------------|--------------------------------------------------------------------------------|------------|------------------------------------------------------------------------|
| <b>kind</b> ( <i>Kubernetes in Docker</i> ) | Cada nó é um contêiner (Docker, Podman ou nerdctl)                             | Não        | Estudo, testes automatizados e CI; clusters com vários nós em segundos |
| <b>Minikube</b>                             | VM ou contêiner local, com addons prontos (dashboard, ingress, metrics-server) | Não        | Estudo e desenvolvimento, com experiência "tudo incluído"              |
| <b>k3d</b>                                  | k3s dentro de contêineres                                                      | Não        | Como o kind, usando k3s                                                |
| <b>Docker Desktop / Rancher Desktop</b>     | Cluster embutido na aplicação de desktop                                       | Não        | Quem já usa essas ferramentas                                          |
| <b>k3s</b> (SUSE/Rancher)                   | Distribuição leve em um único binário                                          | <b>Sim</b> | Edge, IoT, Raspberry Pi, clusters pequenos                             |
| <b>MicroK8s</b> (Canonical)                 | Pacote snap com addons                                                         | <b>Sim</b> | Edge, IoT, estações Ubuntu                                             |
| <b>k0s</b> (Mirantis)                       | Distribuição em um único binário, "zero friction"                              | <b>Sim</b> | Instalação e manutenção simplificadas                                  |
| <b>kubeadm</b>                              | Ferramenta oficial para montar clusters em máquinas próprias                   | <b>Sim</b> | Aprender a fundo e montar clusters <i>on-premises</i>                  |

| Ferramenta    | Como roda                              | Produção? | Quando usar                                  |
|---------------|----------------------------------------|-----------|----------------------------------------------|
| EKS, GKE, AKS | Control plane gerenciado pelo provedor | Sim       | A maioria dos ambientes de produção em nuvem |

### 3.3.2 kind

```
# Instalação (Linux x86-64). Versão estável em set/2026: v0.33.0
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind
# macOS: brew install kind | Windows: choco install kind ou winget install Kubernetes.kind

kind create cluster --name estudo      # contexto criado: kind-estudo
kind get clusters
kubectl get nodes
kind delete cluster --name estudo
```

Cluster com vários nós:

```
# kind-3nos.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

```
kind create cluster --name multi --config kind-3nos.yaml
kind create cluster --name antigo --image kindest/node:<versão> # outra versão do Kubernetes (tags nas
notas de versão do kind)
kind load docker-image minha-app:dev --name multi              # enviar uma imagem local aos nós
kind delete clusters --all
```

- O contexto de cada cluster se chama `kind-<nome>`.
- `kind load docker-image` evita publicar em um registry imagens construídas localmente; nesse caso, use uma tag diferente de `latest` ou `imagePullPolicy: IfNotPresent`, para o kubelet não tentar baixá-la.
- O kind detecta sozinho Docker, Podman ou nerdctl; para forçar um deles, use `KIND_EXPERIMENTAL_PROVIDER=podman`.

### 3.3.3 Minikube

```
# Requisitos: 2 CPUs, 2 GB de memória livre, 20 GB de disco e um driver (Docker, Podman, QEMU, KVM, Hyper-V, VirtualBox...)
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
# macOS: brew install minikube | Windows: winget install Kubernetes.minikube

minikube start                                # escolhe o driver disponível
minikube start --driver=docker --kubernetes-version=v1.36.5
minikube start --nodes 3 -p multi              # perfil "multi" com 3 nós
minikube status
minikube ip                                  # IP do nó
minikube ssh                                  # entra no nó
minikube dashboard                            # painel web
minikube addons list && minikube addons enable metrics-server
minikube logs
minikube stop                                  # para sem apagar
minikube delete                                # apaga o cluster
minikube delete --all --purge                  # apaga tudo, inclusive ~/.minikube
```

- Com o driver **Docker**, não é preciso hypervisor nem virtualização aninhada; é a opção mais comum hoje. Drivers de VM (KVM, Hyper-V, VirtualBox, QEMU) exigem virtualização habilitada no processador e no firmware.
- Cada **perfil** (-p) é um cluster independente, com contexto próprio.

## 3.4 Primeiros comandos

### 3.4.1 Explorar o cluster

```
kubectl get nodes -o wide                    # nós, versões, IPs, runtime
kubectl get namespaces                        # default, kube-system, kube-public, kube-node-lease
kubectl get pods -n kube-system              # componentes do cluster
kubectl get pods -A -o wide                  # todos os namespaces, com IP e nó
kubectl api-resources                         # tipos de objeto e abreviações (po, deploy, svc, ns, cm...)
kubectl explain pod.spec.containers          # documentação dos campos
```

| Namespace       | Conteúdo                                                                                                     |
|-----------------|--------------------------------------------------------------------------------------------------------------|
| default         | Onde vão os objetos criados sem -n                                                                           |
| kube-system     | Componentes do cluster (CoreDNS, kube-proxy, CNI e, em clusters kubeadm, os Pods estáticos do control plane) |
| kube-public     | Dados legíveis por qualquer um, inclusive sem autenticação                                                   |
| kube-node-lease | Objetos Lease que os nós renovam como sinal de vida                                                          |

### 3.4.2 Primeiro Pod e primeiro Service

```
kubectl run web --image=nginx:1.29 --port=80      # cria um Pod
kubectl get pods                                  # STATUS Running, READY 1/1
kubectl expose pod web --port=80                  # cria um Service ClusterIP
kubectl get svc                                    # o Service "kubernetes" (10.96.0.1:443) já existia
kubectl port-forward pod/web 8080:80              # acesse http://localhost:8080
kubectl delete svc web && kubectl delete pod web
```

- `kubectl expose` precisa saber a porta. Sem `--port` no `expose` e sem `containerPort` declarado no Pod, o comando falha com `couldn't find port via --port flag or introspection`.
- Um Service **ClusterIP** só é acessível **dentro do cluster**. Para testar da sua máquina, use `kubectl port-forward` ou um Service NodePort ou LoadBalancer.
- O Service `kubernetes` no namespace `default` é o endereço interno do próprio API server.

### 3.4.3 Imperativo e declarativo

| Forma                                       | Exemplo                                                                                                                      | Quando usar                                                             |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Comandos imperativos</b>                 | <code>kubectl run</code> , <code>kubectl create deployment</code> , <code>kubectl scale</code> , <code>kubectl expose</code> | Testes rápidos e geração de esqueletos                                  |
| <b>Configuração imperativa com arquivos</b> | <code>kubectl create -f</code> , <code>kubectl replace -f</code> , <code>kubectl delete -f</code>                            | Criar uma vez; falha se o objeto já existe ( <code>create</code> )      |
| <b>Configuração declarativa</b>             | <code>kubectl apply -f arquivo.yaml</code> OU <code>-f diretório/</code>                                                     | <b>Padrão do dia a dia</b> : cria ou atualiza; os arquivos ficam no Git |

- **Gerar YAML sem criar nada**: `--dry-run=client -o yaml` produz um manifesto para editar.

```
kubectl run web --image=nginx:1.29 --port=80 --dry-run=client -o yaml > pod.yaml
kubectl create deployment web --image=nginx:1.29 --replicas=3 --dry-run=client -o yaml > deploy.yaml
kubectl apply -f deploy.yaml
kubectl diff -f deploy.yaml          # mostra o que mudaria antes de aplicar
```

- `--dry-run=server` envia o pedido ao API server, que valida tudo (inclusive webhooks de admissão) sem gravar.

### 3.4.4 Formatos de saída e seletores

| Opção                                                                | Resultado                                |
|----------------------------------------------------------------------|------------------------------------------|
| <code>-o wide</code>                                                 | Colunas extras (IP do Pod, nó)           |
| <code>-o yaml</code> / <code>-o json</code>                          | Objeto completo, com <code>status</code> |
| <code>-o name</code>                                                 | Só tipo/nome (útil em scripts)           |
| <code>-o jsonpath='{.items[*].metadata.name}'</code>                 | Campos específicos                       |
| <code>-o custom-columns=NAME:.metadata.name,NO:.spec.nodeName</code> | Tabela sob medida                        |
| <code>-l app=web</code> / <code>-l 'tier in (front,back)'</code>     | Filtra por labels                        |
| <code>--field-selector status.phase=Running</code>                   | Filtra por campos                        |
| <code>-w</code> / <code>--watch</code>                               | Acompanha mudanças em tempo real         |

- `kubectl get all` **não mostra "tudo"**: lista só alguns tipos (Pods, Services, Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, CronJobs). ConfigMaps, Secrets, Ingresses e PVCs, por exemplo, ficam de fora. Para vários tipos específicos: `kubectl get pod,svc,cm`.

## 3.5 Labels, seletores e annotations

### 3.5.1 Labels

- **Labels** são pares chave-valor usados para **identificar e selecionar** objetos. Services, Deployments, NetworkPolicies, afinidades e o próprio `kubectl` escolhem objetos por labels.
- Formato da chave: prefixo opcional com domínio (`app.kubernetes.io/`) + nome de até 63 caracteres. Os valores também têm até 63 caracteres.
- **Labels recomendadas**: `app.kubernetes.io/name`, `app.kubernetes.io/instance`, `app.kubernetes.io/version`, `app.kubernetes.io/component`, `app.kubernetes.io/part-of`, `app.kubernetes.io/managed-by`.

```
kubectl label pod web ambiente=producao          # adiciona
kubectl label pod web ambiente=homologacao --overwrite
kubectl label pod web ambiente-                  # remove
kubectl get pods -l 'ambiente=producao,tier in (web,api)'
kubectl get pods -l '!canary'                   # sem a label canary
kubectl get pods --show-labels
kubectl get pods -L app,ambiente                 # labels como colunas
```

- Em um Deployment, as labels em `metadata.labels` são **do Deployment**; as dos Pods ficam em `spec.template.metadata.labels`. Filtrar Pods pelas labels do Deployment não retorna nada.

### 3.5.2 Annotations

- **Annotations** guardam informações que **não servem para seleção**: descrições, links, configurações de ferramentas (controladores de entrada, cert-manager, Prometheus) e dados gerados por ferramentas. Não têm o limite de 63 caracteres no valor.

```
kubectl annotate deployment web descricao="API de pedidos"
kubectl annotate deployment web descricao="API de pedidos v2" --overwrite
kubectl annotate deployment web descricao-
kubectl get deployment web -o jsonpath='{.metadata.annotations}'
```

## 3.6 Decisão rápida — kubectl e clusters locais

| Situação                                     | Abordagem recomendada                                                                   | Erro comum                           |
|----------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------|
| Cluster para estudar com vários nós          | kind com arquivo de configuração                                                        | Máquinas virtuais montadas à mão     |
| Cluster local com dashboard e addons prontos | Minikube                                                                                | kind esperando addons prontos        |
| Cluster pequeno em produção (edge, IoT)      | k3s, MicroK8s ou k0s                                                                    | kind ou Minikube em produção         |
| Comando foi para o cluster errado            | Conferir <code>kubectl config current-context</code> antes; <code>kubectx/kubens</code> | Um único kubeconfig sem nomes claros |

| Situação                                                  | Abordagem recomendada                                             | Erro comum                                          |
|-----------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------|
| Criar o primeiro YAML de um objeto                        | <code>kubectl create ... --dry-run=client -o yaml</code>          | Escrever o manifesto do zero                        |
| Atualizar objetos a partir de arquivos                    | <code>kubectl apply -f</code> (e <code>kubectl diff</code> antes) | <code>kubectl create -f</code> repetidas vezes      |
| Testar um Service ClusterIP da sua máquina                | <code>kubectl port-forward</code>                                 | Tentar acessar o IP do Service de fora do cluster   |
| Usar uma imagem local no kind                             | <code>kind load docker-image</code> + tag fixa                    | Tag latest com <code>imagePullPolicy: Always</code> |
| <code>kubectl</code> muito diferente da versão do cluster | Manter no máximo uma versão menor de diferença                    | Ignorar o aviso do <code>kubectl version</code>     |

# Pods

*Ideia central: o Pod é a menor unidade que o Kubernetes agenda: um ou mais contêineres que **compartilham o mesmo IP, a mesma rede** (localhost) **e os mesmos volumes**, e que sempre rodam juntos no mesmo nó. Pods são **descartáveis**: quando um some, ninguém o "conserta"; um controlador cria outro, com outro nome e outro IP.*

## 4.1 O que é um Pod

### 4.1.1 Características

- **Um IP por Pod:** todos os contêineres do Pod compartilham o namespace de rede e se falam por localhost. Por isso, dois contêineres do mesmo Pod **não podem** escutar na mesma porta.
- **Volumes compartilhados:** um volume declarado no Pod pode ser montado em vários dos seus contêineres.
- **Agendados juntos:** todos os contêineres de um Pod vão para o **mesmo nó**, sempre.
- **Efêmeros:** um Pod não é movido para outro nó. Se o nó cair ou o Pod for apagado, ele deixa de existir; um Deployment ou outro controlador cria um **novo** Pod no lugar.
- **Um Pod "solto"** (criado diretamente, sem controlador) só tem seus contêineres reiniciados pelo kubelet enquanto existir. Se for apagado ou o nó falhar, **nada o recria**. Em produção, Pods são criados por **Deployments**, StatefulSets, DaemonSets ou Jobs.
- **Uma aplicação por Pod:** escalar significa criar mais Pods, não mais contêineres dentro do mesmo Pod. Coloque dois contêineres no mesmo Pod só quando eles precisam, de fato, viver juntos.

## 4.2 Criar e inspecionar Pods

### 4.2.1 Manifesto comentado

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web          # usado por Services e seletores
spec:
  containers:
    - name: nginx      # nome do contêiner dentro do Pod
      image: nginx:1.29 # sempre com tag; evite latest
      ports:
        - containerPort: 80 # documenta a porta (e permite nomeá-la)
```

```

kubectl apply -f pod.yaml
kubectl get pods                # NAME, READY (prontos/total), STATUS, RESTARTS, AGE
kubectl get pod web -o wide     # IP do Pod e nó
kubectl get pod web -o yaml     # objeto completo, com status, uid e nodeName
kubectl describe pod web       # configuração + eventos
kubectl delete pod web         # ou: kubectl delete -f pod.yaml

```

- `containerPort` é **informativo**, como o `EXPOSE` do Docker: o contêiner escuta na porta mesmo sem declará-la. Declarar ajuda o `kubectl expose` e permite dar nomes às portas (`name: http`).
- `imagePullPolicy`: com tag `latest` ou sem tag, o padrão é `Always`; com qualquer outra tag, `IfNotPresent`. Use tags fixas (ou digests) para saber exatamente o que está rodando.

#### 4.2.2 `apply`, `create` e `replace`

| Comando                         | Objeto não existe | Objeto já existe                             |
|---------------------------------|-------------------|----------------------------------------------|
| <code>kubectl create -f</code>  | Cria              | <b>Erro</b> <code>AlreadyExists</code>       |
| <code>kubectl apply -f</code>   | Cria              | <b>Atualiza</b> com as diferenças do arquivo |
| <code>kubectl replace -f</code> | Erro              | Substitui o objeto inteiro                   |

- Muitos campos de um Pod são **imutáveis** depois de criado (a imagem pode mudar; a lista de portas, os volumes e os comandos, não). Para mudar um Pod solto, apague e crie de novo. Em um Deployment, basta alterar o template: ele cria Pods novos.

#### 4.2.3 Logs

```

kubectl logs web                # saída do contêiner (STDOUT e STDERR)
kubectl logs -f web             # acompanha em tempo real
kubectl logs web -c sidecar     # contêiner específico em Pods com vários
kubectl logs web --all-containers
kubectl logs web --previous     # execução ANTERIOR (essencial em CrashLoopBackOff)
kubectl logs -l app=web --since=10m # vários Pods por label
kubectl logs deploy/web         # um Pod do Deployment

```

### 4.3 Pods com vários contêineres

#### 4.3.1 Padrões de uso

| Padrão                | O que o contêiner auxiliar faz                               | Exemplo                                                                                        |
|-----------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Sidecar</b>        | Complementa a aplicação principal durante toda a vida do Pod | Coletor de logs, proxy de service mesh, sincronizador de arquivos                              |
| <b>Ambassador</b>     | Proxy local para serviços externos                           | A aplicação fala com <code>localhost:6379</code> , e o ambassador encaminha para o Redis certo |
| <b>Adapter</b>        | Traduz a saída da aplicação para um formato padrão           | Converte métricas da aplicação para o formato do Prometheus                                    |
| <b>Init container</b> | Roda <b>antes</b> dos contêineres principais, até terminar   | Esperar uma dependência, aplicar migrações, baixar configuração                                |

```

apiVersion: v1
kind: Pod
metadata:
  name: web-com-logs
spec:
  volumes:
    - name: logs
      emptyDir: {}
  initContainers:
    - name: preparar                                # init container comum: roda e termina
      image: busybox:1.37
      command: ["sh", "-c", "echo configurado > /logs/inicio.txt"]
      volumeMounts: [{ name: logs, mountPath: /logs }]
    - name: coletor                                # sidecar nativo (estável desde a 1.33)
      image: busybox:1.37
      restartPolicy: Always
      command: ["sh", "-c", "tail -F /logs/acesso.log"]
      volumeMounts: [{ name: logs, mountPath: /logs }]
  containers:
    - name: app
      image: busybox:1.37
      command: ["sh", "-c", "while true; do date >> /logs/acesso.log; sleep 5; done"]
      volumeMounts: [{ name: logs, mountPath: /logs }]

```

- **Sidecars nativos:** um init container com `restartPolicy: Always` inicia **antes** dos contêineres principais, continua rodando junto com eles e só é encerrado **depois** que eles terminam. Isso resolve dois problemas antigos dos sidecars declarados em `containers`: a aplicação iniciando antes do proxy, e Jobs que nunca terminavam porque o sidecar continuava vivo.
- **Contêiner que termina logo:** imagens como `alpine` ou `busybox` executam um shell e saem imediatamente sem nada a prender. Em exemplos e depuração, use um processo que fique em primeiro plano (`sleep infinity`); em aplicações reais, o processo principal é a própria aplicação.

## 4.4 Interagir com um Pod em execução

### 4.4.1 `exec`, `attach`, `port-forward`, `cp` e `debug`

| Comando                                                              | O que faz                                                                                                             |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>kubectl exec web -- ls /usr/share/nginx/html</code>            | Executa um comando <b>novo</b> no contêiner                                                                           |
| <code>kubectl exec -it web -c app -- sh</code>                       | Shell interativo ( <code>-i</code> mantém STDIN, <code>-t</code> aloca terminal; <code>-c</code> escolhe o contêiner) |
| <code>kubectl attach -it web -c app</code>                           | Conecta ao STDIN/STDOUT do <b>processo principal</b> ; não cria processo novo                                         |
| <code>kubectl port-forward pod/web 8080:80</code>                    | Encaminha uma porta local para o Pod (também <code>svc/</code> e <code>deploy/</code> )                               |
| <code>kubectl cp web:/etc/nginx/nginx.conf ./nginx.conf</code>       | Copia arquivos (exige <code>tar</code> na imagem)                                                                     |
| <code>kubectl debug -it web --image=busybox:1.37 --target=app</code> | Adiciona um <b>contêiner efêmero</b> de depuração ao Pod, compartilhando o namespace de processos do contêiner alvo   |
| <code>kubectl debug node/&lt;nó&gt; -it --image=ubuntu</code>        | Pod de depuração com acesso ao sistema de arquivos do nó (em <code>/host</code> )                                     |

- **Prefira `exec` a `attach`:** o `attach` só faz sentido para processos interativos. Em um servidor como o `nginx`, ele apenas mostra a saída do processo principal.

- Ao sair de um `exec -it` com `exit` ou Ctrl+D, só o shell criado termina; o contêiner continua.
- **Imagens mínimas** (distroless, scratch) não têm shell: use `kubectl debug` com contêineres efêmeros, recurso estável desde a 1.25.

## 4.5 Ciclo de vida do Pod

### 4.5.1 Fases e estados

| Fase do Pod | Significado                                                                                            |
|-------------|--------------------------------------------------------------------------------------------------------|
| Pending     | Aceito pelo cluster, mas algum contêiner ainda não rodou (esperando agendamento ou download de imagem) |
| Running     | Associado a um nó, com pelo menos um contêiner rodando ou reiniciando                                  |
| Succeeded   | Todos os contêineres terminaram com sucesso e não serão reiniciados (Jobs)                             |
| Failed      | Todos terminaram e pelo menos um falhou                                                                |
| Unknown     | O estado não pôde ser obtido (em geral, o nó parou de responder)                                       |

| Motivo que aparece em STATUS    | Causa comum                                                                                                    | Onde investigar                                                                 |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| ContainerCreating               | Baixando a imagem, montando volumes                                                                            | <code>describe</code> (eventos)                                                 |
| ErrImagePull / ImagePullBackOff | Nome ou tag errados, registry privado sem <code>imagePullSecrets</code> , limite de pulls                      | <code>describe</code>                                                           |
| CrashLoopBackOff                | O contêiner inicia e sai repetidamente; o kubelet espera cada vez mais entre as tentativas (de 10 s até 5 min) | <code>logs --previous</code> , código de saída em <code>describe</code>         |
| OOMKilled                       | O contêiner passou do limite de memória (código de saída 137)                                                  | <code>describe (Last State)</code>                                              |
| CreateContainerConfigError      | ConfigMap ou Secret referenciado não existe                                                                    | <code>describe</code>                                                           |
| Pending sem nó                  | Nenhum nó com recursos ( <b>requests</b> ) suficientes, taints, afinidade impossível                           | <code>describe</code> (eventos <code>FailedScheduling</code> )                  |
| Completed                       | O processo principal terminou com código 0                                                                     | Esperado em Jobs; em Deployments, o processo não está rodando em primeiro plano |

- `restartPolicy` do Pod: `Always` (padrão, obrigatório em Deployments), `OnFailure` ou `Never` (usados em Jobs). Quem reinicia o contêiner é o **kubelet**, no mesmo nó, mantendo o Pod.
- **Encerramento gracioso**: ao apagar um Pod, o kubelet envia **SIGTERM** ao processo principal de cada contêiner, espera `terminationGracePeriodSeconds` (padrão **30 s**) e então envia **SIGKILL**. Um hook `preStop` roda antes do SIGTERM. Ao mesmo tempo, o Pod sai dos endpoints dos Services, para não receber tráfego novo.
- `kubectl delete pod web --grace-period=0 --force` apaga o objeto sem esperar o nó confirmar. Use só quando o nó está inacessível: em StatefulSets, pode levar a duas instâncias com a mesma identidade.

## 4.6 Recursos: requests e limits

### 4.6.1 Como funcionam

```
spec:
  containers:
    - name: app
      image: nginx:1.29
      resources:
        requests:          # o que o scheduler reserva no nó
          cpu: 250m
          memory: 128Mi
        limits:            # o máximo que o contêiner pode usar
          cpu: 500m
          memory: 256Mi
```

|          | requests                                                      | limits                                                                                    |
|----------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Quem usa | O <b>scheduler</b> , para escolher um nó com capacidade livre | O <b>kubelet</b> e o kernel (cgroups), durante a execução                                 |
| CPU      | Garantia de fatia de CPU sob disputa                          | <b>Limite rígido:</b> o contêiner é <b>estrangulado</b> (throttling), não morto           |
| Memória  | Reserva para o agendamento                                    | Passou do limite: o contêiner é <b>morto por OOM</b> (OOMKilled, código 137) e reiniciado |

- **Unidades de CPU:** 1 = um núcleo (vCPU). Frações em **milicores**: 500m = 0.5 = meio núcleo. O menor valor é 1m.
- **Unidades de memória:** em bytes, com sufixos **binários** Ki, Mi, Gi (potências de 1024) ou **decimais** k, M, G (potências de 1000). 128Mi = 134.217.728 bytes; 128M = 128.000.000 bytes. Cuidado: 128m (minúsculo) significa 0,128 **byte**, um erro comum.
- **Sem requests**, o scheduler supõe que o Pod não precisa de nada e pode lotar um nó. **Sem limits**, um contêiner pode consumir a memória do nó inteiro e provocar despejo (*eviction*) de outros Pods.

### 4.6.2 Classes de QoS

| Classe            | Quando o Pod recebe                                                            | Prioridade para ser despejado quando o nó fica sem memória |
|-------------------|--------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Guaranteed</b> | Todos os contêineres com requests <b>iguais</b> aos limits, para CPU e memória | Última                                                     |
| <b>Burstable</b>  | Pelo menos um request ou limit definido, sem atender ao Guaranteed             | Intermediária                                              |
| <b>BestEffort</b> | Nenhum request nem limit                                                       | Primeira                                                   |

- A classe aparece em `kubectl describe pod` (QoS Class) e em `kubectl get pod -o jsonpath='{.status.qosClass}'`.

### 4.6.3 Testar os limites e ajustar sem recriar

```
kubectl run carga --image=ubuntu:24.04 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "carga", "image": "ubuntu:24.04", "command":
[ "sleep", "infinity" ],
  "resources": { "requests": { "memory": "64Mi", "cpu": "250m" }, "limits": { "memory": "128Mi", "cpu": "500m" } } } ] } }'
kubectl exec -it carga -- bash -c 'apt-get update -qq && apt-get install -y -qq stress'
kubectl exec -it carga -- stress --vm 1 --vm-bytes 100M --timeout 20s # abaixo do limite: roda
kubectl exec -it carga -- stress --vm 1 --vm-bytes 200M --timeout 20s # acima: o worker do stress é
morto por OOM
kubectl top pod carga # exige metrics-server
```

- Quando o processo que passa do limite **não é o principal** (como o worker do `stress`), só ele morre. Quando é o processo principal, o contêiner inteiro termina com `OOMKilled` e é reiniciado.
- Redimensionamento sem recriar o Pod** (estável desde a 1.35): altere CPU e memória pelo subrecurso `resize`. A CPU muda sem reiniciar o contêiner; para a memória, o campo `resizePolicy` define se o contêiner precisa reiniciar.

```
kubectl patch pod carga --subresource=resize \
-p '{ "spec": { "containers": [ { "name": "carga", "resources": { "requests": { "cpu": "500m" }, "limits":
{ "cpu": "1" } } } ] } }'
```

- Recursos declarados **no nível do Pod** (`spec.resources`, um orçamento para todos os contêineres) estão em **beta** na 1.37 e dependem do recurso `PodLevelResources` estar ativo no cluster.

## 4.7 Volume emptyDir

### 4.7.1 Como funciona

- Um `emptyDir` é criado **vazio** quando o Pod é agendado em um nó e **apagado quando o Pod é removido**. Ele sobrevive à reinicialização de contêineres, mas não à troca de Pod.
- É muito usado para **compartilhar arquivos entre contêineres do mesmo Pod** (aplicação e sidecar), para cache e para arquivos temporários grandes (fora da camada gravável do contêiner).
- `medium: Memory` monta um **tmpfs** (RAM): é rápido, e o que for gravado **conta no limite de memória** do contêiner. `sizeLimit` limita o tamanho; se for ultrapassado, o Pod é despejado.

```
spec:
  containers:
    - name: app
      image: ubuntu:24.04
      command: ["sleep", "infinity"]
      volumeMounts:
        - name: temporario
          mountPath: /dados
  volumes:
    - name: temporario
      emptyDir:
        sizeLimit: 256Mi
        # medium: Memory
```

- Para dados que precisam sobreviver ao Pod, use volumes persistentes (PersistentVolumeClaim), assunto de um capítulo próprio.

## 4.8 Decisão rápida — Pods

| Situação                                       | Abordagem recomendada                                                                           | Erro comum                                      |
|------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Rodar uma aplicação em produção                | Deployment (ou outro controlador), nunca um Pod solto                                           | <code>kubectl run</code> e esquecer             |
| Aplicação e coletor de logs juntos             | Sidecar nativo (init container com <code>restartPolicy: Always</code> ) + <code>emptyDir</code> | Dois processos no mesmo contêiner               |
| Preparar algo antes de a aplicação subir       | Init container                                                                                  | <code>sleep</code> no comando da aplicação      |
| Pod em <code>CrashLoopBackOff</code>           | <code>kubectl logs --previous</code> e o código de saída no <code>describe</code>               | Apagar o Pod repetidas vezes                    |
| Pod em <code>Pending</code>                    | Eventos do <code>describe</code> : requests, taints, afinidade                                  | Aumentar o limit (o scheduler olha os requests) |
| Contêiner morto com 137                        | <code>OOMKilled</code> : rever o limite de memória ou o consumo da aplicação                    | Aumentar a CPU                                  |
| Aplicação lenta mas nunca reinicia             | Conferir throttling de CPU (limit baixo demais)                                                 | Procurar vazamento de memória                   |
| Depurar imagem sem shell                       | <code>kubectl debug</code> com contêiner efêmero                                                | Trocar a imagem por uma com shell               |
| Pod importante não pode ser despejado primeiro | QoS Guaranteed (requests = limits)                                                              | Deixar sem requests e limits                    |
| Ajustar CPU de um Pod em execução              | <code>kubectl patch --subresource=resize (1.35+)</code>                                         | Apagar e recriar o Pod                          |
| Testar a aplicação da sua máquina              | <code>kubectl port-forward</code>                                                               | Criar um Service LoadBalancer só para testar    |

# Deployments e ReplicaSets

**Ideia central:** um **Deployment** gerencia **ReplicaSets**, e cada **ReplicaSet** gerencia **Pods**. O **ReplicaSet** só sabe manter *N* cópias de um mesmo **template**; o **Deployment** é quem sabe **trocar de versão**: a cada mudança no **template** dos **Pods**, ele cria um **ReplicaSet** novo, aumenta esse e diminui o antigo, e guarda os anteriores para **rollback**.

## 5.1 Deployment, ReplicaSet e Pod

### 5.1.1 A hierarquia

```
Deployment web (estado desejado: 3 réplicas de nginx:1.29)
└─ ReplicaSet web-5d8f7c9b6d (um por versão do template; os antigos ficam com 0 réplicas)
    ├── Pod web-5d8f7c9b6d-4kq2x
    ├── Pod web-5d8f7c9b6d-9z17m
    └─ Pod web-5d8f7c9b6d-xw3pd
```

- O sufixo do **ReplicaSet** é o `pod-template-hash`, calculado a partir do **template** dos **Pods**; o **Kubernetes** o acrescenta como **label** nos **Pods** e no seletor do **ReplicaSet**, para que **ReplicaSets** de versões diferentes não disputem os mesmos **Pods**.
- Os **Pods** têm um `ownerReference` apontando para o **ReplicaSet**, que aponta para o **Deployment**. `kubectl describe pod` mostra `Controlled By: ReplicaSet/...`. Apagar o **Deployment** apaga, em cascata, os **ReplicaSets** e os **Pods**.

### 5.1.2 Manifesto comentado

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 10           # ReplicaSets antigos guardados para rollback (padrão 10)
  progressDeadlineSeconds: 600      # tempo sem progresso até o rollout ser marcado como falho
  (padrão 600)
  minReadySeconds: 0                # segundos que um Pod novo precisa ficar pronto para contar como
  disponível
  selector:
    matchLabels:
      app: web                       # PRECISA casar com as labels do template; imutável depois de
  criado
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%                  # Pods a mais permitidos durante a troca (padrão 25%)
      maxUnavailable: 25%           # Pods a menos permitidos durante a troca (padrão 25%)
  template:                          # o "molde" dos Pods
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.29
          ports:
            - containerPort: 80
          resources:
            requests: { cpu: 250m, memory: 128Mi }
            limits: { cpu: 500m, memory: 256Mi }
```

- **Seletor e labels do template precisam casar**, ou o API server recusa o Deployment. O seletor **não pode ser alterado** depois (a mudança exige recriar o Deployment).
- **Labels do Deployment** (em `metadata.labels`) e **labels dos Pods** (em `template.metadata.labels`) são coisas diferentes: `kubectl get pods -l app=web` filtra pelas labels dos **Pods**. Um filtro que não casa com elas não retorna nada, mesmo com os Pods rodando.

## 5.2 Criar, inspecionar e escalar

### 5.2.1 Comandos do dia a dia

```
kubectl apply -f deploy.yaml
kubectl create deployment web --image=nginx:1.29 --replicas=3          # forma imperativa
kubectl get deployments          # NAME, READY (3/3), UP-TO-DATE, AVAILABLE, AGE
kubectl get rs -l app=web        # DESIRED, CURRENT, READY
kubectl get pods -l app=web -o wide
kubectl describe deployment web  # estratégia, condições, ReplicaSet novo e antigos, eventos
kubectl scale deployment web --replicas=5
kubectl set image deployment/web nginx=nginx:1.29.1
kubectl edit deployment web      # edição direta no cluster
```

| Coluna de <code>kubectl get deployments</code> | Significado                                                 |
|------------------------------------------------|-------------------------------------------------------------|
| READY                                          | Réplicas prontas / desejadas                                |
| UP-TO-DATE                                     | Réplicas já com o template mais recente                     |
| AVAILABLE                                      | Réplicas prontas há pelo menos <code>minReadySeconds</code> |

- **Escalar não cria revisão:** só mudanças no **template dos Pods** (`spec.template`) criam um ReplicaSet novo. Mudar `replicas` altera o ReplicaSet atual.
- `kubectl scale`, `set image` e `edit` mudam o objeto **no cluster**, mas não no seu arquivo. No próximo `kubectl apply` do arquivo antigo, a mudança é desfeita. Para manter o Git como fonte da verdade, altere o YAML e aplique.
- **Quando há autoescalamento (HPA)**, retire `replicas` do manifesto, para o `apply` não brigar com o HPA.

## 5.3 Estratégias de atualização

### 5.3.1 RollingUpdate e Recreate

| Estratégia                       | Como funciona                                                                                                              | Quando usar                                                                                                                                                                                       |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RollingUpdate</b><br>(padrão) | Cria Pods da versão nova e remove os da antiga aos poucos, respeitando <code>maxSurge</code> e <code>maxUnavailable</code> | A maioria das aplicações sem estado; exige que duas versões convivam por alguns minutos                                                                                                           |
| <b>Recreate</b>                  | Remove <b>todos</b> os Pods antigos e só então cria os novos                                                               | Aplicações que não podem ter duas versões ao mesmo tempo (migração de esquema incompatível, licença por instância, volume <code>ReadWriteOnce</code> compartilhado). Tem <b>indisponibilidade</b> |

- `maxSurge`: quantos Pods **acima** de `replicas` podem existir durante a troca. Percentuais são arredondados **para cima**.
- `maxUnavailable`: quantos Pods **abaixo** de `replicas` podem ficar indisponíveis. Percentuais são arredondados **para baixo**. Os dois não podem ser 0 ao mesmo tempo.
- **Exemplo:** `replicas: 10`, `maxSurge: 1`, `maxUnavailable: 2`. Durante o rollout, o total nunca passa de **11** Pods, e pelo menos **8** ficam disponíveis. O controlador pode derrubar 2 antigos e criar 1 novo logo de início, e segue trocando conforme os novos ficam prontos.
- **Zero indisponibilidade planejada:** `maxUnavailable: 0` e `maxSurge: 1` (ou mais), com **readiness probe** configurada. Sem readiness probe, o Pod é considerado pronto assim que o contêiner inicia, e o rollout avança mesmo

com a aplicação ainda carregando (capítulo 6).

- **Blue/green e canário** não são estratégias nativas do Deployment. São feitos com dois Deployments e um Service (ou Ingress/Gateway), ou com ferramentas como Argo Rollouts e Flagger.

## 5.4 Rollout e rollback

### 5.4.1 Acompanhar, pausar, voltar

```
kubectl rollout status deployment/web          # espera o fim (útil em CI; sai com erro se
estourar o prazo)
kubectl rollout history deployment/web          # revisões
kubectl rollout history deployment/web --revision=3 # template de uma revisão
kubectl annotate deployment/web kubernetes.io/change-cause="nginx 1.29.1: correção de segurança"
kubectl rollout undo deployment/web            # volta para a revisão anterior
kubectl rollout undo deployment/web --to-revision=2
kubectl rollout pause deployment/web            # junta várias mudanças...
kubectl set image deployment/web nginx=nginx:1.29.2
kubectl set resources deployment/web -c nginx --limits=memory=512Mi
kubectl rollout resume deployment/web           # ...em um único rollout
kubectl rollout restart deployment/web          # recria os Pods sem mudar a imagem
```

- **Rollback é um rollout para trás:** o Deployment volta a escalar o ReplicaSet da revisão anterior (que estava com 0 réplicas). A revisão restaurada recebe o **próximo número** de revisão e sai da lista com o número antigo: depois de voltar da revisão 3 para a 2, o histórico mostra 1, 3 e 4.
- **CHANGE-CAUSE:** preenchido pela anotação `kubernetes.io/change-cause`. A antiga flag `--record` está **obsoleta**.
- O rollback só é possível para revisões ainda guardadas (`revisionHistoryLimit`, padrão 10).
- `rollout restart` muda uma anotação de data no template, o que dispara um rollout normal: útil para recarregar ConfigMaps e Secrets lidos só na inicialização.
- `kubectl rollout` funciona com **Deployments, DaemonSets e StatefulSets**. ReplicaSets, Jobs e CronJobs não têm rollout.
- Rollout que não avança (imagem inexistente, Pods que nunca ficam prontos) é marcado com a condição `Progressing=False` e o motivo `ProgressDeadlineExceeded` depois de `progressDeadlineSeconds`. O Kubernetes **não faz rollback sozinho**: o `rollout status` falha, e a decisão de voltar é sua (ou da ferramenta de deploy).

## 5.5 ReplicaSet diretamente

### 5.5.1 Quando e por que não

- Um **ReplicaSet** mantém um número de Pods iguais. Ele **não atualiza** Pods existentes: se você mudar a imagem no template de um ReplicaSet, os Pods em execução continuam com a imagem antiga. Só os Pods criados **depois** (porque outro morreu ou foi apagado) usam a nova, e o grupo fica com versões misturadas.
- Por isso, crie **Deployments**, não ReplicaSets. O ReplicaSet criado à mão só faz sentido em cenários muito específicos de orquestração customizada.
- **Adoção de Pods:** um ReplicaSet assume qualquer Pod sem dono cujas labels casem com o seletor. Um Pod solto com as mesmas labels pode ser "adotado" e contar como réplica, e o ReplicaSet então cria menos Pods do que você esperava.
- **ReplicationController** é o antecessor do ReplicaSet (seletor só por igualdade) e está obsoleto.

- Apagar só o ReplicaSet mantendo os Pods: `kubectl delete rs <nome> --cascade=orphan`.

```
# Ver a imagem de cada Pod (útil para achar versões misturadas)
kubectl get pods -l app=web \
  -o custom-columns='POD:.metadata.name,IMAGEM:.spec.containers[*].image'
```

## 5.6 Decisão rápida — Deployments

| Situação                                              | Abordagem recomendada                                                                                       | Erro comum                                     |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Aplicação sem estado com várias réplicas              | Deployment                                                                                                  | ReplicaSet ou Pods soltos                      |
| Atualizar sem indisponibilidade                       | RollingUpdate com <code>maxUnavailable: 0</code> , <code>maxSurge ≥ 1</code> e <code>readiness probe</code> | RollingUpdate sem <code>readiness probe</code> |
| Versões antiga e nova não podem coexistir             | Recreate (aceitando a janela de indisponibilidade)                                                          | RollingUpdate e torcer                         |
| Nova versão com problema                              | <code>kubectl rollout undo</code> e corrigir o manifesto no Git                                             | Editar Pods à mão                              |
| Saber o motivo de cada revisão                        | Anotação <code>kubernetes.io/change-cause</code>                                                            | <code>--record</code> (obsoleto)               |
| Várias mudanças em um só rollout                      | <code>rollout pause</code> , mudanças, <code>rollout resume</code>                                          | Três rollouts seguidos                         |
| Recarregar configuração lida na inicialização         | <code>kubectl rollout restart</code>                                                                        | Apagar os Pods um por um                       |
| Pipeline deve falhar se o deploy não ficar saudável   | <code>kubectl rollout status --timeout=5m</code>                                                            | <code>kubectl apply</code> e seguir            |
| <code>kubectl get pods -l ...</code> não retorna nada | Conferir as labels do <b>template</b> ( <code>kubectl get pods --show-labels</code> )                       | Filtrar pelas labels do Deployment             |
| Mudar o seletor de um Deployment                      | Criar um Deployment novo (o seletor é imutável)                                                             | Editar o seletor e aplicar                     |

## DaemonSets e Probes

***Ideia central:** um **DaemonSet** garante um **Pod por nó** (ou por grupo de nós), inclusive nos nós que entrarem depois, e é a forma padrão de rodar agentes de infraestrutura. As **probes** dizem ao Kubernetes se um contêiner está **vivo** (liveness), **pronto para tráfego** (readiness) ou **ainda iniciando** (startup). Sem elas, o cluster só sabe se o processo existe.*

### 6.1 DaemonSet

#### 6.1.1 Para que serve

- Roda uma cópia de um Pod em **cada nó elegível**. Nó novo no cluster ganha o Pod automaticamente; nó removido leva o Pod junto.
- Casos típicos: **coletores de logs** (Fluent Bit, Vector), **agentes de métricas** (Prometheus node-exporter, agente do Datadog), **plugins de rede** (Calico, Cilium) e o **kube-proxy**, **agentes de segurança** (Falco), drivers de armazenamento (CSI node plugin).
- **Não há replicas**: o número de Pods é o número de nós elegíveis.

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1          # padrão: um nó por vez
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      hostNetwork: true          # usa a rede do nó (as métricas são do nó)
      hostPID: true
      tolerations:               # também roda nos nós de control plane
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: node-exporter
          image: quay.io/prometheus/node-exporter:v1.9.1
          args:
            - --path.procfs=/host/proc
            - --path.sysfs=/host/sys
            - --path.rootfs=/host/root
          ports:
            - containerPort: 9100
          resources:
            requests: { cpu: 50m, memory: 32Mi }
            limits: { memory: 64Mi }
          volumeMounts:
            - { name: proc, mountPath: /host/proc, readOnly: true }
            - { name: sys, mountPath: /host/sys, readOnly: true }
            - { name: root, mountPath: /host/root, readOnly: true, mountPropagation: HostToContainer }
      volumes:
        - { name: proc, hostPath: { path: /proc } }
        - { name: sys, hostPath: { path: /sys } }
        - { name: root, hostPath: { path: / } }

```

- **Montar /proc e /sys não basta:** o node-exporter precisa saber onde eles estão ( `--path.procfs` , `--path.sysfs` ), senão lê o /proc do próprio contêiner.
- `hostNetwork: true` faz o Pod usar o IP e as portas do **nó**: com ela, a `containerPort` já é a porta do nó e não é preciso `hostPort`. Use com cuidado (conflito de portas, menos isolamento); é aceitável para agentes de nó.
- **Tolerations:** por padrão, os nós de control plane têm um taint que afasta Pods comuns. Agentes que precisam rodar também neles declaram a toleration correspondente. O DaemonSet já recebe automaticamente tolerations para nós com problemas ( `not-ready` , `unreachable` , pressão de disco e de memória).
- **Só alguns nós:** `nodeSelector` (ex.: `gpu: "true"`) ou `affinity` no template.

### 6.1.2 Comandos e atualização

```
kubectl apply -f node-exporter.yaml
kubectl get daemonset -n monitoring # DESIRED, CURRENT, READY, UP-TO-DATE, AVAILABLE, NODE SELECTOR
kubectl get pods -n monitoring -l app=node-exporter -o wide # um por nó
kubectl describe ds node-exporter -n monitoring
kubectl rollout status ds/node-exporter -n monitoring
kubectl rollout undo ds/node-exporter -n monitoring
kubectl delete ds node-exporter -n monitoring
```

| updateStrategy         | Comportamento                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RollingUpdate (padrão) | Troca os Pods nó a nó, respeitando <code>maxUnavailable</code> (padrão 1); <code>maxSurge</code> permite subir o novo antes de derrubar o antigo no mesmo nó |
| OnDelete               | O template muda, mas cada Pod só é recriado quando você o apaga (controle manual, nó a nó)                                                                   |

- **Não existe** `kubectl create daemonset`. Para ter um esqueleto, gere um Deployment com `kubectl create deployment ... --dry-run=client -o yaml`, troque `kind` para `DaemonSet` e remova `replicas`, `strategy` e `status`.

## 6.2 Probes

### 6.2.1 Os três tipos

| Probe                  | Pergunta                           | Se falhar                                                                                                               | Quando usar                                                                                                                |
|------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>liveness-Probe</b>  | "O contêiner está vivo ou travou?" | O kubelet <b>reinicia o contêiner</b> (o Pod continua o mesmo)                                                          | Aplicações que podem travar sem sair (deadlock, loop)                                                                      |
| <b>readiness-Probe</b> | "Pode receber tráfego agora?"      | O Pod é <b>retirado dos endpoints</b> dos Services (não reinicia) e aparece como <code>0/1</code> em <code>READY</code> | Praticamente toda aplicação que recebe tráfego                                                                             |
| <b>startup-Probe</b>   | "Já terminou de iniciar?"          | Após <code>failureThreshold × periodSeconds</code> , o contêiner é reiniciado                                           | Aplicações de inicialização lenta; enquanto ela não passa, <code>liveness</code> e <code>readiness</code> <b>não rodam</b> |

- A **readiness** roda durante **toda** a vida do contêiner, não só no início. Um Pod pode sair do balanceamento temporariamente (sobrecarga, dependência indisponível) e voltar sem reiniciar.
- Durante um **rollout**, o Deployment só considera um Pod novo disponível quando a `readiness` passa. É ela que impede que uma versão quebrada substitua a anterior.

### 6.2.2 Mecanismos de teste

| Mecanismo              | Sucesso quando                                            | Exemplo                                               |
|------------------------|-----------------------------------------------------------|-------------------------------------------------------|
| <code>httpGet</code>   | Resposta HTTP de 200 a 399                                | <code>path: /healthz</code> , <code>port: 8080</code> |
| <code>tcpSocket</code> | A conexão TCP abre                                        | <code>port: 5432</code>                               |
| <code>exec</code>      | O comando termina com código 0                            | <code>command: ["pg_isready", "-U", "app"]</code>     |
| <code>grpc</code>      | O serviço de health do gRPC responde <code>SERVING</code> | <code>port: 9090</code>                               |

### 6.2.3 Parâmetros

| Campo                                      | Padrão   | Significado                                                                    |
|--------------------------------------------|----------|--------------------------------------------------------------------------------|
| <code>initialDelaySeconds</code>           | 0        | Espera antes da primeira verificação                                           |
| <code>periodSeconds</code>                 | 10       | Intervalo entre verificações                                                   |
| <code>timeoutSeconds</code>                | <b>1</b> | Tempo máximo de <b>cada</b> verificação; passou disso, conta como falha        |
| <code>successThreshold</code>              | 1        | Sucessos seguidos para voltar a "ok" (liveness e startup <b>só aceitam 1</b> ) |
| <code>failureThreshold</code>              | 3        | Falhas seguidas para agir (reiniciar ou retirar do tráfego)                    |
| <code>terminationGracePeriodSeconds</code> | O do Pod | Tempo de encerramento quando a liveness ou a startup provoca reinício          |

- **Tempo até agir:** por volta de `failureThreshold × periodSeconds`. Com os padrões, uma aplicação travada é reiniciada em cerca de 30 s. O `timeoutSeconds` **não** é um intervalo entre tentativas: é o prazo de cada tentativa.

### 6.2.4 Exemplo completo

```
containers:
  - name: api
    image: minha-org/api:2.3.0
    ports:
      - name: http
        containerPort: 8080
    startupProbe:
      # até 30 × 5 s = 150 s para iniciar
      httpGet: { path: /healthz, port: http }
      periodSeconds: 5
      failureThreshold: 30
    livenessProbe:
      # verifica só o próprio processo
      httpGet: { path: /healthz, port: http }
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3
    readinessProbe:
      # pode incluir dependências essenciais
      httpGet: { path: /ready, port: http }
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 2
```

```
kubectl describe pod <pod>    # linhas Liveness:, Readiness:, Startup: e eventos "Unhealthy"
kubectl get pods -w           # READY 0/1 enquanto a readiness não passa; RESTARTS subindo se a
liveness falha
```

- Evento típico de falha: `Warning Unhealthy ... Liveness probe failed: HTTP probe failed with statuscode: 404`, seguido de `Container nginx failed liveness probe, will be restarted`.

### 6.2.5 Boas práticas e armadilhas

- **Liveness não deve depender de serviços externos:** se o banco cair e a liveness checar o banco, **todos** os Pods reiniciam em cadeia, sem resolver nada. Liveness verifica se **o processo** está saudável; dependências vão, no máximo, na readiness.

- **Não use** `initialDelaySeconds` **grande na liveness para "dar tempo" de iniciar**: use uma **startupProbe**. Ela protege a inicialização lenta sem atrasar a detecção de travamentos depois.
- **Mesma verificação e mesmos tempos em liveness e readiness** fazem o contêiner ser reiniciado no mesmo momento em que sai do tráfego. Deixe a liveness mais tolerante.
- `timeoutSeconds: 1` é curto para endpoints que fazem trabalho de verdade; sob carga, a probe falha e o contêiner reinicia justamente quando mais é preciso.
- **Endpoint leve**: `/healthz` não deve consultar tudo, gerar log a cada chamada nem exigir autenticação.
- `exec` **custa caro** em escala: cria um processo a cada verificação. Prefira HTTP, TCP ou gRPC.

### 6.3 Decisão rápida — DaemonSets e probes

| Situação                                            | Abordagem recomendada                                                                                  | Erro comum                                                           |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Agente de logs ou métricas em todos os nós          | DaemonSet com requests e limits pequenos                                                               | Deployment com réplicas iguais ao número de nós                      |
| Agente também nos nós de control plane              | Toleration para <code>node-role.kubernetes.io/control-plane</code>                                     | Remover o taint dos nós de control plane                             |
| Agente só nos nós com GPU                           | <code>nodeSelector</code> ou <code>affinity</code> no template do DaemonSet                            | Um DaemonSet por nó                                                  |
| Atualizar um DaemonSet crítico com controle manual  | <code>updateStrategy: OnDelete</code>                                                                  | Apagar o DaemonSet e criar de novo                                   |
| Pod não deve receber tráfego enquanto carrega cache | <code>readinessProbe</code>                                                                            | <code>livenessProbe</code> com <code>initialDelaySeconds</code> alto |
| Aplicação Java/.NET leva 2 minutos para subir       | <code>startupProbe</code> com <code>failureThreshold × periodSeconds &gt; tempo de subida</code>       | Liveness que mata o contêiner antes de ele terminar de iniciar       |
| Banco de dados fora do ar                           | Readiness pode refletir isso; liveness não                                                             | Liveness que checka o banco e reinicia tudo                          |
| Aplicação trava de vez em quando sem sair           | <code>livenessProbe</code> no endpoint do próprio processo                                             | Esperar alguém notar e apagar o Pod                                  |
| Rollout avança com a versão nova quebrada           | <code>readinessProbe</code> + <code>maxUnavailable: 0</code> + <code>rollout status</code> no pipeline | Rollout sem probes                                                   |

# Instalando um Cluster com kubeadm

***Ideia central:** o **kubeadm** é a ferramenta oficial para montar um cluster "de verdade" em máquinas próprias. Ele gera certificados, sobe o control plane como **Pods estáticos** e emite o comando de entrada dos nós. Mas não faz tudo: o runtime, os pacotes, o **plugin de rede (CNI)**, o balanceador do API server em alta disponibilidade e as atualizações são responsabilidade de quem opera.*

## 7.1 Formas de ter um cluster

### 7.1.1 Comparação

| Opção                                     | O que é                                                                           | Quando faz sentido                                               |
|-------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Serviço gerenciado</b> (EKS, GKE, AKS) | O provedor opera o control plane, faz atualizações e alta disponibilidade         | A maioria dos ambientes de produção em nuvem                     |
| <b>kubeadm</b>                            | Ferramenta oficial que monta o cluster nas suas máquinas                          | On-premises, estudo aprofundado, base de outras ferramentas      |
| <b>Kubespray</b>                          | Playbooks Ansible que usam o kubeadm por baixo                                    | Muitos nós, on-premises, automação com Ansible                   |
| <b>kops</b>                               | Cria e mantém clusters em nuvem pública (forte na AWS)                            | Quem quer operar o control plane na nuvem sem serviço gerenciado |
| <b>Cluster API</b>                        | Clusters descritos como objetos Kubernetes, gerenciados a partir de outro cluster | Frotas de clusters, plataformas internas                         |
| <b>k3s, k0s, MicroK8s, Talos</b>          | Distribuições completas, prontas para produção                                    | Edge, clusters pequenos, operação simplificada                   |
| <b>kind, Minikube</b>                     | Clusters locais                                                                   | Estudo e testes (capítulo 3)                                     |

## 7.2 Preparar os nós

### 7.2.1 Requisitos

- Linux com **2 CPUs** e **2 GB de RAM** ou mais em cada máquina (o control plane precisa de pelo menos isso; os workers, do que as aplicações pedirem).
- Rede entre todos os nós; **hostname**, **endereço MAC** e `product_uuid` **únicos** em cada um.
- Portas liberadas (capítulo 2): **6443** no control plane, **10250** em todos os nós, **2379–2380** entre os membros do etcd, **30000–32767** se houver NodePort, e as **portas do CNI** escolhido. A porta 10255 (API somente leitura do kubelet) vem **desativada** em clusters kubeadm.
- Swap**: por padrão, o kubelet **não inicia** se detectar swap. Desative (`sudo swapoff -a`) e remova a linha do `/etc/fstab`) ou configure explicitamente o kubelet para tolerá-la (`failSwapOn: false`), sabendo das implicações para limites de memória.

### 7.2.2 Módulos do kernel, sysctl e runtime

```
# Módulos e parâmetros de rede (em todos os nós)
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip_forward          = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
sudo sysctl --system

# containerd (pacote containerd.io do repositório do Docker, ou o da distribuição)
sudo apt-get install -y containerd.io
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

- `net.ipv4.ip_forward` é obrigatório. Os parâmetros `bridge-nf-call-*` (e o módulo `br_netfilter`) são exigidos por muitos CNIs para que o tráfego das bridges passe pelo iptables.
- `SystemdCgroup = true` faz o containerd usar o driver de cgroup **systemd**, o mesmo que o kubeadm configura no kubelet. Confira no `config.toml` gerado que a chave existe e mudou: o formato do arquivo muda entre versões maiores do containerd. Nas versões recentes do Kubernetes, o kubelet também consegue descobrir o driver pelo próprio runtime.

### 7.2.3 Pacotes do Kubernetes

```
sudo apt-get update && sudo apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.37/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.37/deb/ /' \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl          # evita atualização acidental
sudo systemctl enable --now kubelet                  # fica reiniciando até o kubeadm init/join: é normal
```

- Os repositórios `pkgs.k8s.io` **têm um endereço por versão menor** (`v1.37`). Para atualizar para a 1.38, é preciso trocar a URL.
- Os antigos `apt.kubernetes.io` e `packages.cloud.google.com` foram **desativados** em 2024, e `apt-key` está descontinuado. Tutoriais com esses comandos não funcionam mais.

## 7.3 Criar o cluster

### 7.3.1 kubeadm init e kubeadm join

```
# No primeiro nó de control plane
sudo kubeadm init \
  --pod-network-cidr=10.244.0.0/16 \
  --apiserver-advertise-address=10.0.0.10
# Para alta disponibilidade, acrescente: --control-plane-endpoint=k8s-api.exemplo.local:6443 --upload-certs

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Instalar o CNI (exemplos: Calico, Cilium, Flannel), seguindo a documentação dele
kubectl get nodes          # NotReady até o CNI estar rodando

# Nos workers, com o comando impresso pelo init
sudo kubeadm join 10.0.0.10:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>

# Token expirado (validade padrão: 24 h)? Gere outro comando no control plane:
kubeadm token create --print-join-command
```

| Fase do kubeadm init | O que faz                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| preflight            | Verifica requisitos (swap, portas, runtime, módulos) e baixa imagens                                                                                          |
| certs                | Cria a CA do cluster e os certificados em <code>/etc/kubernetes/pki</code>                                                                                    |
| kubeconfig           | Gera <code>admin.conf</code> , <code>kubelet.conf</code> , <code>controller-manager.conf</code> , <code>scheduler.conf</code> e <code>super-admin.conf</code> |
| control-plane / etcd | Escreve os manifestos de <b>Pods estáticos</b> em <code>/etc/kubernetes/manifests</code>                                                                      |
| mark-control-plane   | Adiciona a label <code>node-role.kubernetes.io/control-plane</code> e o taint <code>NoSchedule</code>                                                         |
| bootstrap-token      | Cria o token e as permissões para os nós entrarem                                                                                                             |
| addons               | Instala CoreDNS e kube-proxy                                                                                                                                  |

- **Pods estáticos:** o kubelet lê os arquivos de `/etc/kubernetes/manifests` e roda o API server, o scheduler, o controller-manager e o etcd como Pods, sem depender do API server para isso. Editar um desses arquivos recria o componente.
- `--pod-network-cidr` precisa combinar com a configuração do CNI e **não pode se sobrepor** à rede dos nós nem à faixa de Services (padrão `10.96.0.0/12`).
- `--discovery-token-ca-cert-hash` garante que o nó está entrando no cluster certo (evita um API server falso). O token autentica o nó; o hash autentica o cluster.
- **Alta disponibilidade:** use `--control-plane-endpoint` apontando para um **load balancer** (ou DNS) na frente dos API servers, desde o primeiro nó, e entre com os demais control planes usando `kubeadm join ... --control-plane --certificate-key <chave>`.

### 7.3.2 Kubeconfig do administrador

- O `admin.conf` tem três partes: `clusters` (endereço do API server e **CA** do cluster), `users` (o **certificado de cliente** e a **chave privada**, em base64) e `contexts`. O certificado identifica o usuário no grupo `kubeadm:cluster-admins`, com poder total.
- Quem tem esse arquivo **administra o cluster**. Distribua acessos individuais com RBAC (capítulo 17), não cópias do `admin.conf`.
- `kubectl config view` mostra o arquivo com os dados sensíveis omitidos (`--raw` mostra tudo).

### 7.3.3 Escolher o CNI

| Plugin                                                        | Destaques                                                                                                                                      |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Calico                                                        | Muito usado; roteamento BGP ou overlay (VXLAN/IP-in-IP); NetworkPolicy completa e políticas próprias                                           |
| Cilium                                                        | Baseado em <b>eBPF</b> ; NetworkPolicy em L3–L7, pode substituir o kube-proxy, observabilidade com Hubble; projeto graduado da CNCF            |
| Flannel                                                       | Simples, overlay VXLAN; <b>não aplica NetworkPolicy</b> sozinho                                                                                |
| Plugins das nuvens (AWS VPC CNI, Azure CNI, GKE Dataplane V2) | Pods com IPs da própria VPC; integração com a rede do provedor                                                                                 |
| Weave Net                                                     | Popular em tutoriais antigos; a Weaveworks encerrou as atividades em 2024 e o projeto <b>deixou de ser mantido</b> . Não use em clusters novos |

- CNI** (*Container Network Interface*) é a especificação que o runtime usa para chamar o plugin quando um sandbox de Pod é criado. O Kubernetes **não traz** implementação de rede de Pods: sem um CNI, os nós ficam `NotReady` e o CoreDNS fica `Pending`.

## 7.4 Operar o cluster

### 7.4.1 Manutenção do dia a dia

```
kubectl describe node <nó>           # labels, taints, capacidade, alocável, Pods, condições
kubectl get nodes -o wide            # versões do kubelet, do SO e do runtime
kubectl cordon <nó>                  # não recebe Pods novos
kubectl drain <nó> --ignore-daemonsets --delete-emptydir-data # esvazia para manutenção
kubectl uncordon <nó>                # volta a receber Pods
sudo kubeadm certs check-expiration # certificados do control plane valem 1 ano
sudo kubeadm upgrade plan            # atualização: uma versão menor por vez
sudo kubeadm reset                   # desfaz init/join em um nó
```

- `Capacity` × `Allocatable`: o nó reserva parte dos recursos para o sistema e o kubelet; o scheduler usa o **Allocatable**. O limite padrão é de **110 Pods por nó**.
- Condições do nó**: `Ready`, `MemoryPressure`, `DiskPressure`, `PIDPressure`, `NetworkUnavailable`. Pressão de recursos leva o kubelet a **despejar** Pods (começando pelos `BestEffort`).
- Atualizações**: suba **uma versão menor por vez** (1.36 → 1.37), primeiro o control plane, depois os nós (com `drain`). Os certificados gerados pelo `kubeadm` são renovados automaticamente em cada `kubeadm upgrade`; clusters que não são atualizados por um ano param quando eles expiram.

## 7.5 Decisão rápida — Instalação

| Situação                                                | Abordagem recomendada                                                                       | Erro comum                                      |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------|
| Produção em nuvem sem time dedicado à plataforma        | Serviço gerenciado (EKS, GKE, AKS)                                                          | kubeadm em VMs "para economizar"                |
| Nós <code>NotReady</code> logo após o <code>init</code> | Instalar o CNI com o CIDR usado no <code>--pod-network-cidr</code>                          | Reiniciar o kubelet                             |
| <code>kubeadm init</code> falha no preflight por swap   | Desativar a swap (e no <code>/etc/fstab</code> ) ou configurar o kubelet para tolerá-la     | Usar <code>--ignore-preflight-errors=all</code> |
| Token do <code>join</code> expirou                      | <code>kubeadm token create --print-join-command</code>                                      | Refazer o <code>init</code>                     |
| Control plane de produção                               | 3 nós atrás de um load balancer ( <code>--control-plane-endpoint</code> )                   | Um nó só, ou adicionar o endpoint depois        |
| Manutenção de um nó                                     | <code>kubectl drain</code> e depois <code>uncordon</code>                                   | Desligar o nó com Pods rodando                  |
| Tutorial usa <code>apt.kubernetes.io</code>             | Repositório <code>pkgs.k8s.io</code> por versão menor                                       | Insistir no repositório desativado              |
| Tutorial instala Weave Net                              | Calico, Cilium ou o CNI da sua nuvem                                                        | Instalar um projeto sem manutenção              |
| Cluster parou depois de um ano sem atualização          | Renovar certificados ( <code>kubeadm certs renew</code> ) e passar a atualizar regularmente | Recrutar o cluster                              |

# Volumes e Armazenamento Persistente

**Ideia central:** o Pod *pede* armazenamento com um *PersistentVolumeClaim* (PVC); o cluster *entrega* um *PersistentVolume* (PV), criado à mão pelo administrador ou, na prática quase sempre, *dinamicamente* por um driver CSI a partir de uma *StorageClass*. O Pod só conhece o PVC; o tipo de disco por baixo fica transparente.

## 8.1 Tipos de volume

### 8.1.1 Efêmeros e persistentes

| Volume                                    | Ciclo de vida                                 | Uso                                                                                                                           |
|-------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| emptyDir                                  | Nasce e morre com o Pod                       | Compartilhar arquivos entre contêineres, cache, temporários (capítulo 4)                                                      |
| configMap, secret, downwardAPI, projected | Do Pod                                        | Entregar configuração, credenciais e metadados como arquivos (capítulo 11)                                                    |
| Volume efêmero genérico (ephemeral)       | Do Pod, mas provisionado por uma StorageClass | Espaço temporário grande com as características de um disco de verdade                                                        |
| hostPath                                  | Do <b>nó</b>                                  | Agentes de nó (logs, métricas, sockets). <b>Evite em aplicações:</b> amarra o Pod ao nó e expõe o sistema de arquivos do host |
| persistentVolumeClaim                     | Independente do Pod                           | Dados que precisam sobreviver ao Pod (bancos, uploads)                                                                        |

## 8.2 StorageClass

### 8.2.1 O que define

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: rapido
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" # classe padrão
provisioner: ebs.csi.aws.com # driver CSI que cria os volumes
parameters:
  type: gp3 # parâmetros próprios do driver
reclaimPolicy: Delete # o que fazer com o disco quando o PVC for apagado
volumeBindingMode: WaitForFirstConsumer # criar o disco só quando um Pod for agendado
allowVolumeExpansion: true # permitir aumentar PVCs

```

| Campo             | Opções                                                                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| reclaimPolicy     | Delete (padrão: apaga o PV e o disco) ou Retain (mantém o PV e os dados para recuperação manual). Recycle está <b>obsoleto</b>                |
| volumeBindingMode | Immediate (padrão: cria e vincula na hora) ou WaitForFirstConsumer (espera o Pod ser agendado, para criar o disco <b>na mesma zona</b> do nó) |

| Campo                             | Opções                                                                                                        |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>allowVolumeExpansion</code> | Permite <b>aumentar</b> o PVC editando <code>spec.resources.requests.storage</code> . Diminuir não é possível |

- **PVC sem** `storageClassName` usa a classe marcada como **padrão**. Mantenha **uma só** classe padrão. `storageClassName: ""` pede explicitamente um PV **sem** classe (vínculo estático).
- **CSI** (*Container Storage Interface*) é o padrão atual de drivers de armazenamento. Os provisionadores antigos embutidos no Kubernetes (`kubernetes.io/aws-ebs`, `kubernetes.io/gce-pd`, `kubernetes.io/azure-disk` e outros) foram **removidos** e substituídos por drivers CSI (`ebs.csi.aws.com`, `pd.csi.storage.gke.io`, `disk.csi.azure.com`). Em clusters antigos, classes com o nome in-tree ainda funcionam porque são redirecionadas ao driver CSI, que **precisa estar instalado** (no EKS, o add-on do EBS CSI driver).
- **Clusters locais**: o kind usa o provisionador `rancher.io/local-path` (classe `standard`), e o Minikube, o `k8s.io/minikube-hostpath`. Os dois gravam em diretórios do nó.
- `kubernetes.io/no-provisioner` indica que não há criação dinâmica: os PVs são criados à mão (é o caso de volumes `local`).
- **NFS**: não existe provisionador NFS embutido. Para criação dinâmica, instale o driver CSI de NFS (`nfs.csi.k8s.io`) ou um provisionador externo; sem isso, os PVs NFS são criados manualmente.

## 8.3 PersistentVolume e PersistentVolumeClaim

### 8.3.1 Vínculo estático (PV criado à mão)

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-dados
  labels: { storage: nfs }
spec:
  capacity: { storage: 10Gi }
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: nfs-manual
  mountOptions: [nfsvers=4.1]
  nfs:
    server: 10.0.0.50
    path: /exports/dados
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dados
spec:
  accessModes: [ReadWriteMany]
  storageClassName: nfs-manual
  resources:
    requests: { storage: 10Gi }
  selector:
    matchLabels: { storage: nfs }      # opcional: escolher entre PVs pelas labels

```

### 8.3.2 Vínculo dinâmico (o caso comum)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dados-app
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: rapido
  resources:
    requests: { storage: 20Gi }
---
# No Pod (ou no template do Deployment/StatefulSet)
spec:
  containers:
    - name: app
      image: nginx:1.29
      volumeMounts:
        - name: dados
          mountPath: /usr/share/nginx/html
  volumes:
    - name: dados
      persistentVolumeClaim:
        claimName: dados-app
```

### 8.3.3 Modos de acesso

| Modo             | Sigla | Significado                                                                            |
|------------------|-------|----------------------------------------------------------------------------------------|
| ReadWriteOnce    | RWO   | Leitura e escrita por <b>um nó</b> por vez (vários Pods <b>no mesmo nó</b> podem usar) |
| ReadOnlyMany     | ROX   | Somente leitura por vários nós                                                         |
| ReadWriteMany    | RWX   | Leitura e escrita por vários nós                                                       |
| ReadWriteOncePod | RWOP  | Leitura e escrita por <b>um único Pod</b> no cluster inteiro                           |

- O modo precisa ser **suportado pelo tipo de armazenamento**. Discos de bloco (EBS, Persistent Disk, Azure Disk) são **RWO**; RWX exige sistemas de arquivos de rede (NFS, EFS, Azure Files, CephFS).
- Volume local** × **hostPath**: os dois usam disco do nó, mas o **local** declara em qual nó está (**nodeAffinity** obrigatório), e o scheduler leva os Pods para lá. O **hostPath** não tem essa informação e é perigoso em clusters com vários nós.

### 8.3.4 Estados e ciclo de vida

| Objeto | Estados                                                                                 |
|--------|-----------------------------------------------------------------------------------------|
| PV     | Available (livre) → Bound (vinculado) → Released (PVC apagado, dados ainda lá) → Failed |
| PVC    | Pending (sem PV compatível ou aguardando o primeiro Pod) → Bound                        |

- PVC em Pending com WaitForFirstConsumer** é normal até um Pod que o use ser agendado (evento **waiting for first consumer to be created before binding**).
- PV Released com Retain** não volta sozinho a **Available**: os dados continuam lá, e o administrador decide o que fazer (recuperar, limpar e liberar removendo o **claimRef**, ou apagar).

- **Proteção:** um PVC em uso por um Pod não é apagado de imediato (finalizar `kubernetes.io/pvc-protection`); fica `Terminating` até o Pod sair.
- **Snapshots** (`VolumeSnapshot`) e **clonagem** de PVCs estão disponíveis quando o driver CSI suporta.

```
kubectl get sc           # classes, provisionador, reclaim policy, binding mode, padrão
kubectl get pv           # CAPACITY, ACCESS MODES, RECLAIM POLICY, STATUS, CLAIM
kubectl get pvc -A        # eventos de provisionamento e vínculo
kubectl describe pvc dados-app
```

## 8.4 Decisão rápida — Armazenamento

| Situação                                                     | Abordagem recomendada                                                                            | Erro comum                                                            |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Banco de dados em nuvem                                      | PVC com StorageClass de disco em bloco (CSI) e <code>WaitForFirstConsumer</code>                 | <code>hostPath</code>                                                 |
| Vários Pods em nós diferentes escrevendo nos mesmos arquivos | RWX em NFS/EFS/Azure Files                                                                       | Disco em bloco RWO                                                    |
| Disco criado em outra zona e o Pod fica <code>Pending</code> | <code>volumeBindingMode: WaitForFirstConsumer</code>                                             | <code>Immediate</code> em clusters multi-zona                         |
| Não perder dados se alguém apagar o PVC                      | <code>reclaimPolicy: Retain</code> (e backups/snapshots)                                         | Confiar no <code>Delete</code> padrão                                 |
| PVC ficou pequeno                                            | <code>allowVolumeExpansion: true</code> e aumentar o request                                     | Criar outro PVC e copiar à mão                                        |
| PVC <code>Pending</code> para sempre                         | <code>describe pvc</code> : classe inexistente, driver CSI ausente, modo de acesso não suportado | Recriar o PVC repetidamente                                           |
| NFS com criação automática de volumes                        | Driver CSI de NFS                                                                                | Esperar que uma StorageClass com <code>no-provisioner</code> crie PVs |
| Disco local rápido em nós específicos                        | Volume local com <code>nodeAffinity</code>                                                       | <code>hostPath</code> em todos os nós                                 |
| Classe <code>kubernetes.io/aws-ebs</code> sem funcionar      | Instalar o driver EBS CSI (ou migrar para <code>ebs.csi.aws.com</code> )                         | Procurar o provisionador in-tree, que foi removido                    |

# Services, EndpointSlices e DNS

*Ideia central: Pods mudam de IP o tempo todo. O **Service** dá a um grupo de Pods, escolhidos por **labels**, um **nome DNS** e um **IP virtual estáveis**, e distribui o tráfego entre os que estão **prontos**. O tipo do Service define de onde ele pode ser acessado: de dentro do cluster, das portas dos nós ou de um load balancer externo.*

## 9.1 Como um Service funciona

### 9.1.1 Seletor, EndpointSlices e kube-proxy

1. O Service declara um **seletor** (`app: web`).
  2. O controlador de EndpointSlices lista os Pods que casam com o seletor e **estão prontos** (readiness probe) e grava seus IPs e portas em objetos **EndpointSlice**.
  3. O **kube-proxy** de cada nó (ou o CNI com eBPF) programa regras que levam o tráfego do IP virtual do Service para esses endereços.
  4. O **CoreDNS** responde o nome do Service com o IP virtual.
- **EndpointSlices** substituem o antigo objeto **Endpoints**, que ficou **obsoleto** na versão 1.33 (continua existindo, com avisos). Consulte com `kubectl get endpointslices -l kubernetes.io/service-name=web`.
  - **Service sem endpoints** (seletor que não casa com nenhuma label, ou Pods não prontos) aceita a conexão e não tem para onde mandar: é a primeira coisa a conferir quando um Service "não responde".

### 9.1.2 Portas

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - name: http
      protocol: TCP
      port: 80           # porta do Service (quem chama usa esta)
      targetPort: 8080   # porta do contêiner (pode ser o nome da porta, ex.: http)
```

- `port` é a porta do Service; `targetPort`, a do contêiner (se omitido, igual a `port`); `nodePort`, a porta aberta em cada nó (só nos tipos NodePort e LoadBalancer).
- Services com **várias portas** exigem `name` em cada uma.

## 9.2 Tipos de Service

### 9.2.1 Visão geral

| Tipo                              | Acessível de                                    | Como                                                                                    |
|-----------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>ClusterIP</b> (padrão)         | Dentro do cluster                               | IP virtual interno + nome DNS                                                           |
| <b>NodePort</b>                   | Fora do cluster, por IP-de-qualquer-nó:nodePort | Abre a mesma porta (faixa <b>30000–32767</b> ) em todos os nós; inclui um ClusterIP     |
| <b>LoadBalancer</b>               | Internet ou rede corporativa                    | Pede um balanceador ao provedor (cloud-controller-manager); inclui NodePort e ClusterIP |
| <b>ExternalName</b>               | Dentro do cluster                               | Responde um <b>CNAME</b> para um nome DNS externo; não tem IP nem proxy                 |
| <b>Headless</b> (clusterIP: None) | Dentro do cluster                               | Sem IP virtual: o DNS devolve <b>os IPs dos Pods</b> diretamente                        |

```
kubectl expose deployment web --port=80 --target-port=8080 # ClusterIP
kubectl expose deployment web --type=NodePort --port=80 --target-port=8080
kubectl expose deployment web --type=LoadBalancer --port=80 --target-port=8080
kubectl create service externalname banco --external-name=db.exemplo.com.br
kubectl get svc -o wide
kubectl describe svc web
```

- **LoadBalancer fora de nuvem:** sem um provedor, o `EXTERNAL-IP` fica `<pending>` para sempre. Em bare metal, use **MetalLB** ou similar; no kind, o **cloud-provider-kind**; no Minikube, `minikube tunnel`.
- **Um LoadBalancer por serviço custa caro** em nuvem. Para vários serviços HTTP, use **um** ponto de entrada (Ingress ou Gateway, capítulo 12).
- **ExternalName** aceita **nomes DNS**, não IPs. Para apontar para um IP fixo fora do cluster, crie um Service **sem seletor** e um EndpointSlice manual.
- `kubectl expose` também funciona com Pods, ReplicaSets, StatefulSets e até outro Service (para expor o mesmo conjunto de Pods com outro tipo), porque ele só copia o seletor.

### 9.2.2 Opções úteis

| Campo                                         | Efeito                                                                                                                                                   |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sessionAffinity: ClientIP</code>        | Mantém um cliente no mesmo Pod (pelo IP de origem), com timeout configurável                                                                             |
| <code>externalTrafficPolicy: Local</code>     | Em NodePort e LoadBalancer, entrega só a Pods do nó que recebeu o tráfego: <b>preserva o IP do cliente</b> e evita um salto extra, mas pode desbalancear |
| <code>internalTrafficPolicy: Local</code>     | Tráfego interno só para Pods do mesmo nó (útil com DaemonSets)                                                                                           |
| <code>trafficDistribution: PreferClose</code> | Prefere endpoints topologicamente próximos (mesma zona), reduzindo latência e custo entre zonas                                                          |
| <code>publishNotReadyAddresses: true</code>   | Inclui Pods não prontos no DNS (usado por alguns sistemas distribuídos durante a descoberta)                                                             |

## 9.3 DNS dentro do cluster

### 9.3.1 Nomes

| Registro                                     | Formato                                                                                                         | Exemplo                                            |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Service                                      | <code>&lt;serviço&gt;.&lt;namespace&gt;.svc.cluster.local</code>                                                | <code>redis.loja.svc.cluster.local</code>          |
| Pod de um StatefulSet (via Service headless) | <code>&lt;pod&gt;.&lt;serviço&gt;.&lt;namespace&gt;.svc.cluster.local</code>                                    | <code>db-0.db.loja.svc.cluster.local</code>        |
| Porta nomeada (SRV)                          | <code>_<code>&lt;porta&gt;</code>._&lt;protocolo&gt;.&lt;serviço&gt;.&lt;namespace&gt;.svc.cluster.local</code> | <code>_http._tcp.web.loja.svc.cluster.local</code> |

- Dentro do **mesmo namespace**, basta o nome curto (`redis`); de outro namespace, `redis.loja`. O `/etc/resolv.conf` dos Pods tem os domínios de busca que completam o nome.
- Teste rápido: `kubectl run -it --rm dns --image=busybox:1.37 --restart=Never -- nslookup web.loja`.
- Variáveis de ambiente `<SERVIÇO>_SERVICE_HOST` também são injetadas nos Pods, mas só para Services que já existiam quando o Pod foi criado. Prefira o DNS.

## 9.4 Decisão rápida — Services

| Situação                                                | Abordagem recomendada                                                                                           | Erro comum                             |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Comunicação entre microserviços                         | ClusterIP e nome DNS                                                                                            | IP de Pod fixo no código               |
| Service não responde                                    | <code>kubectl get endpointslices -l kubernetes.io/service-name=&lt;svc&gt;</code> e conferir labels e readiness | Recriar o Service                      |
| Vários sites HTTP expostos                              | Um Ingress ou Gateway com um único LoadBalancer                                                                 | Um LoadBalancer por aplicação          |
| Precisar do IP real do cliente                          | <code>externalTrafficPolicy: Local</code> (ou cabeçalhos do proxy)                                              | Ler o IP do nó como se fosse o cliente |
| Banco de dados fora do cluster com nome DNS             | ExternalName                                                                                                    | Endereço fixo em cada aplicação        |
| Serviço externo com IP fixo                             | Service sem seletor + EndpointSlice manual                                                                      | ExternalName com IP                    |
| LoadBalancer <code>&lt;pending&gt;</code> no bare metal | MetalLB ou similar                                                                                              | Esperar o provedor que não existe      |
| Acessar um Pod específico de um StatefulSet             | Service headless e o nome <code>&lt;pod&gt;.&lt;svc&gt;</code>                                                  | ClusterIP comum                        |
| Testar um Service da máquina local                      | <code>kubectl port-forward svc/&lt;nome&gt;</code>                                                              | NodePort aberto na internet            |

# StatefulSets

**Ideia central:** um *StatefulSet* gerencia Pods que **não são intercambiáveis**. Cada réplica tem **nome fixo com índice** (db-0, db-1), **nome DNS estável** (via Service headless) e **o próprio PVC**, que continua sendo dela mesmo quando o Pod é recriado em outro nó. É a base para bancos de dados, filas e sistemas distribuídos com consenso.

## 10.1 Garantias do StatefulSet

### 10.1.1 Deployment × StatefulSet

| Aspecto            | Deployment                        | StatefulSet                                                                                 |
|--------------------|-----------------------------------|---------------------------------------------------------------------------------------------|
| Nomes dos Pods     | Aleatórios (web-5d8f7c9b6d-4kq2x) | <b>Fixos e ordenados</b> (db-0, db-1, db-2)                                                 |
| Identidade de rede | Qualquer Pod atende pelo Service  | Cada Pod tem DNS próprio: db-0.db.<ns>.svc.cluster.local                                    |
| Armazenamento      | Um PVC compartilhado (ou nenhum)  | <b>Um PVC por réplica</b> , criado a partir de volumeClaimTemplates                         |
| Criação e remoção  | Em paralelo                       | <b>Em ordem:</b> 0, 1, 2 (cada um espera o anterior ficar pronto); remoção na ordem inversa |
| Pod recriado       | Com outro nome, qualquer volume   | Com <b>o mesmo nome</b> e <b>o mesmo PVC</b>                                                |

## 10.2 Criar um StatefulSet

### 10.2.1 Service headless e StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  clusterIP: None          # headless: o DNS devolve os IPs dos Pods
  selector:
    app: web
  ports:
    - name: http
      port: 80
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web        # Service headless responsável pelo DNS dos Pods (crie-o antes ou junto)
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.29
          ports:
            - name: http
              containerPort: 80
          volumeMounts:
            - name: dados
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: dados
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests: { storage: 1Gi }
```

```
kubectl apply -f web-sts.yaml
kubectl get sts web          # READY 3/3
kubectl get pods -l app=web  # web-0, web-1, web-2 (criados em ordem)
kubectl get pvc              # dados-web-0, dados-web-1, dados-web-2
kubectl exec web-0 -- sh -c 'echo "sou o web-0" > /usr/share/nginx/html/index.html'
kubectl run -it --rm t --image=busybox:1.37 --restart=Never -- wget -qO- http://web-0.web
kubectl delete pod web-0     # volta com o mesmo nome e o mesmo conteúdo
```

- **Nome dos PVCs:** <nome do volumeClaimTemplate>-<nome do StatefulSet>-<índice> (no exemplo, dados-web-0).
- Não existe `kubectl create statefulset`: StatefulSets são criados a partir de manifestos.

## 10.3 Escala, atualização e remoção

### 10.3.1 Comportamentos

| Campo                                             | Opções                                                                                                                                              |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>podManagementPolicy</code>                  | <code>OrderedReady</code> (padrão: um de cada vez, em ordem) ou <code>Parallel</code> (todos juntos; a ordem deixa de valer para criação e remoção) |
| <code>updateStrategy</code>                       | <code>RollingUpdate</code> (padrão: do maior índice para o menor, um por vez) ou <code>OnDelete</code> (só atualiza Pods que você apagar)           |
| <code>rollingUpdate.partition</code>              | Só atualiza Pods com índice <b>maior ou igual</b> ao valor: permite testar a versão nova em parte das réplicas (canário)                            |
| <code>persistentVolumeClaimRetentionPolicy</code> | <code>whenDeleted</code> e <code>whenScaled</code> : <code>Retain</code> (padrão) ou <code>Delete</code>                                            |
| <code>ordinals.start</code>                       | Índice inicial diferente de 0                                                                                                                       |

- **Os PVCs sobrevivem:** por padrão, nem reduzir réplicas nem apagar o StatefulSet apaga os PVCs. Voltar a escalar reaproveita os dados. Para limpar, apague os PVCs explicitamente (`kubectl delete pvc dados-web-0`) ou configure a política de retenção.
- **Apagar o StatefulSet não garante ordem de encerramento** dos Pods. Para um desligamento ordenado, escale para 0 antes de apagar.
- **StatefulSet não faz replicação de dados:** cada réplica tem o próprio disco, vazio no início. Sincronizar os dados entre elas é trabalho da aplicação (replicação do banco). Para bancos em produção, considere **operadores** (Cloud-NativePG, Percona, Strimzi) ou o serviço gerenciado da nuvem.

## 10.4 Decisão rápida — StatefulSets

| Situação                                                    | Abordagem recomendada                                                    | Erro comum                              |
|-------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------|
| Aplicação web sem estado                                    | Deployment                                                               | StatefulSet "por garantia"              |
| Cluster de banco, fila ou consenso (etcd, Kafka, Zookeeper) | StatefulSet + Service headless (ou um operador)                          | Deployment com um PVC compartilhado     |
| Acessar um membro específico                                | <code>&lt;pod&gt;.&lt;serviço-headless&gt;</code>                        | IP do Pod                               |
| Testar versão nova em uma réplica                           | <code>rollingUpdate.partition</code>                                     | Atualizar todas de uma vez              |
| Liberar disco depois de reduzir réplicas                    | Apagar os PVCs ou usar <code>persistentVolumeClaimRetentionPolicy</code> | Esperar que o Kubernetes apague sozinho |
| Desligar tudo em ordem                                      | Escalar para 0 e então apagar                                            | <code>kubectl delete sts</code> direto  |
| Banco de produção sem time especializado                    | Serviço gerenciado ou operador maduro                                    | StatefulSet escrito do zero             |

# ConfigMaps e Secrets

**Ideia central:** a mesma imagem deve rodar em todos os ambientes; o que muda é a **configuração**, entregue por **ConfigMaps** (dados comuns) e **Secrets** (dados sensíveis), como variáveis de ambiente ou arquivos. Um **Secret**, por padrão, é só **base64** guardado no **etcd**: a proteção real vem de **RBAC**, **criptografia em repouso** e, cada vez mais, de um **cofre externo** sincronizado com o cluster.

## 11.1 ConfigMaps

### 11.1.1 Criar

```
kubectl create configmap app-config --from-literal=LOG_LEVEL=info --from-literal=FEATURE_X=true
kubectl create configmap nginx-config --from-file=nginx.conf
kubectl create configmap app-env --from-env-file=app.env
kubectl get configmap nginx-config -o yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info
  app.properties: |
    timeout=30
    retries=3
immutable: true           # campo de primeiro nível (não vai em metadata)
```

- Limite de **1 MiB** por ConfigMap (e por Secret): para arquivos grandes, use volumes.
- `immutable: true` impede alterações (é preciso apagar e recriar) e alivia o API server, que deixa de observar o objeto. O campo fica no **primeiro nível** do objeto, ao lado de `data`, e não dentro de `metadata`.

### 11.1.2 Consumir no Pod

```
spec:
  containers:
    - name: app
      image: minha-org/app:1.0
      env:
        - name: LOG_LEVEL                      # uma chave específica
          valueFrom:
            configMapKeyRef: { name: app-config, key: LOG_LEVEL }
      envFrom:
        - configMapRef: { name: app-env }      # todas as chaves como variáveis
      volumeMounts:
        - name: config
          mountPath: /etc/app                  # cada chave vira um arquivo
        - name: nginx
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf                  # um arquivo só, sem esconder o diretório
  volumes:
    - name: config
      configMap: { name: app-config }
    - name: nginx
      configMap: { name: nginx-config }
```

- **Atualizações:** arquivos montados de ConfigMap ou Secret **são atualizados** automaticamente no Pod (com atraso de até um minuto, aproximadamente), **exceto** quando montados com `subPath`. **Variáveis de ambiente nunca mudam** depois que o contêiner inicia. A aplicação precisa reler o arquivo, ou o Deployment precisa de um `kubectl rollout restart`.
- `subPath` monta um único arquivo sem esconder o resto do diretório (útil para `nginx.conf`), ao custo de perder as atualizações automáticas.

## 11.2 Secrets

### 11.2.1 Base64 não é criptografia

```
echo -n 'giropops' | base64          # Z2lyb3BvcHM= (-n evita codificar a quebra de linha)
echo -n 'Z2lyb3BvcHM=' | base64 -d  # giropops
```

- Base64 só transforma bytes em texto seguro para transporte. **Qualquer pessoa que leia o Secret decodifica o valor.**
- A proteção de um Secret depende de: **RBAC** restrito (quem pode `get / list` Secrets, ou criar Pods que os montem, pode lê-los), **criptografia em repouso no etcd** (`EncryptionConfiguration`, com KMS em clusters gerenciados; nas nuvens, muitas vezes já vem habilitada), e **não versionar** manifestos de Secret em texto no Git.

### 11.2.2 Tipos

| Tipo   | Uso                                | Como criar                                                              |
|--------|------------------------------------|-------------------------------------------------------------------------|
| Opaque | Dados arbitrários (senhas, tokens) | <code>kubectl create secret generic app --from-literal=senha=...</code> |

| Tipo                                | Uso                                                                           | Como criar                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| kubernetes.io/docker-configjson     | Credenciais de registry privado                                               | <code>kubectl create secret docker-registry regcred --docker-server=... --docker-username=... --docker-password=...</code> |
| kubernetes.io/tls                   | Certificado e chave TLS (chaves <code>tls.crt</code> e <code>tls.key</code> ) | <code>kubectl create secret tls site-tls --cert=site.crt --key=site.key</code>                                             |
| kubernetes.io/basic-auth / ssh-auth | Credenciais básicas / chave SSH                                               | Manifesto com os campos do tipo                                                                                            |
| kubernetes.io/service-account-token | Token de ServiceAccount de longa duração                                      | Legado; hoje os tokens são emitidos sob demanda (capítulo 17)                                                              |
| bootstrap.kubernetes.io/token       | Tokens de entrada de nós                                                      | Criado pelo <code>kubeadm</code>                                                                                           |

```

apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:      # texto puro: o API server converte para base64 em data
  DB_USER: app
  DB_PASSWORD: troque-me

```

- `data` exige valores em base64; `stringData` aceita texto e é mais prático para escrever (mas continua sendo texto aberto no arquivo).
- **Registry privado:** o Secret `dockerconfigjson` é usado em `spec.imagePullSecrets` do Pod, ou associado à **ServiceAccount** para valer em todos os Pods que a usam (`kubectl patch serviceaccount default -p '{"imagePullSecrets":[{"name":"regcred"}]}'`).
- **Montar como arquivo é preferível a variável de ambiente:** variáveis aparecem em `/proc`, em dumps de erro e são herdadas por processos filhos. Arquivos de Secret ficam em **tmpfs** no nó.

### 11.2.3 Exemplo: nginx com HTTPS

```

openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt -subj "/CN=localhost"      # autoassinado, só para teste
kubectl create secret tls nginx-tls --cert=tls.crt --key=tls.key
kubectl create configmap nginx-config --from-file=nginx.conf

```

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-https
  labels: { app: nginx-https }
spec:
  containers:
    - name: nginx
      image: nginx:1.29
      ports:
        - containerPort: 443
      volumeMounts:
        - name: config
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf
        - name: tls
          mountPath: /etc/nginx/tls
          readOnly: true
  volumes:                                     # no nível do Pod, não dentro do contêiner
    - name: config
      configMap: { name: nginx-config }
    - name: tls
      secret:
        secretName: nginx-tls                 # arquivos /etc/nginx/tls/tls.crt e tls.key

```

- No `nginx.conf`, aponte `ssl_certificate /etc/nginx/tls/tls.crt`; e `ssl_certificate_key /etc/nginx/tls/tls.key`; . Para renomear os arquivos, use `items` com as chaves **reais** do Secret (`key: tls.crt`, `path: certificado.crt`).
- Teste com `kubectl port-forward pod/nginx-https 8443:443` e `curl -k https://localhost:8443`.

## 11.3 Segredos em cofres externos

### 11.3.1 External Secrets Operator

O **External Secrets Operator (ESO)** busca segredos em cofres externos (AWS Secrets Manager, HashiCorp Vault/OpenBao, Google Secret Manager, Azure Key Vault e outros) e cria **Secrets do Kubernetes** a partir deles, sincronizando periodicamente.

| Recurso            | Papel                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------|
| SecretStore        | Como acessar o cofre (endereço e autenticação), válido em <b>um namespace</b>                      |
| ClusterSecretStore | O mesmo, válido para <b>todos os namespaces</b>                                                    |
| ExternalSecret     | <b>Quais</b> chaves buscar, de qual store, e qual Secret criar (com <code>refreshInterval</code> ) |

```

helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets external-secrets/external-secrets -n external-secrets --create-namespace

```

```

apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: vault
spec:
  provider:
    vault:
      server: "http://vault.vault.svc:8200"
      path: "secret"
      version: "v2" # engine KV versão 2
      auth:
        kubernetes: # autenticação pela ServiceAccount, sem token fixo
          mountPath: "kubernetes"
          role: "external-secrets"
---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: postgres
  namespace: loja
spec:
  refreshInterval: 1h
  secretStoreRef: { name: vault, kind: ClusterSecretStore }
  target:
    name: postgres-secret # Secret criado e mantido pelo ESO
  data:
    - secretKey: POSTGRES_PASSWORD
      remoteRef: { key: postgres, property: password }

```

- A API atual do ESO é `external-secrets.io/v1`. Exemplos com `v1beta1` são de versões anteriores.
- **Autentique pelo mecanismo do provedor** (Kubernetes auth no Vault, IAM Roles/Pod Identity na AWS, Workload Identity no GCP e no Azure) em vez de guardar um token de longa duração em um Secret.
- **Vault em modo `-dev`** guarda tudo em memória, sem selo: só para laboratório. Em produção, o Vault precisa de armazenamento, *unseal* (preferencialmente automático via KMS) e backup. O **OpenBao** é o fork open source mantido pela Linux Foundation.
- **Alternativas ao ESO:** o **Secrets Store CSI Driver** (monta segredos do cofre como arquivos, sem criar Secrets) e o **Sealed Secrets** (criptografa Secrets para poderem ser guardados no Git).

#### 11.4 Decisão rápida — Configuração e segredos

| Situação                                      | Abordagem recomendada                                                 | Erro comum                                    |
|-----------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------|
| Mesma imagem em dev, homologação e produção   | ConfigMaps e Secrets por ambiente                                     | Uma imagem por ambiente                       |
| Arquivo de configuração completo (nginx.conf) | ConfigMap montado como volume ( <code>subPath</code> para um arquivo) | Copiar o arquivo na imagem                    |
| Configuração deve mudar sem recriar o Pod     | Volume sem <code>subPath</code> + aplicação que relê o arquivo        | Variável de ambiente                          |
| Configuração lida só na inicialização         | <code>kubectl rollout restart</code> depois de alterar                | Esperar que o Pod perceba sozinho             |
| Senha de banco                                | Secret montado como arquivo, com RBAC restrito                        | ConfigMap ou variável no manifesto versionado |

| Situação                                        | Abordagem recomendada                                                                                    | Erro comum                             |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------|
| Imagem de registry privado                      | Secret <code>docker-registry</code> + <code>imagePullSecrets</code> (ou na <code>ServiceAccount</code> ) | Imagem pública com credencial embutida |
| Certificado TLS                                 | Secret <code>kubernetes.io/tls</code> (ou <code>cert-manager</code> , capítulo 12)                       | Certificado dentro da imagem           |
| Segredos centralizados, com rotação e auditoria | Cofre externo + External Secrets Operator                                                                | Secrets copiados à mão entre clusters  |
| Manifestos de Secret no Git                     | Sealed Secrets, SOPS ou cofre externo                                                                    | Base64 no repositório                  |

# Ingress, Gateway API e Certificados

**Ideia central:** o **Ingress** descreve regras de entrada HTTP/HTTPS (por host e caminho) para Services; quem as executa é um **Ingress Controller**, que não vem no Kubernetes. A API Ingress está **congelada**, e o controlador mais usado, o **ingress-nginx**, foi **aposentado em março de 2026**. Para clusters novos, o caminho é a **Gateway API**. O **cert-manager** emite e renova os certificados TLS nos dois modelos.

## 12.1 Ingress

### 12.1.1 Componentes

| Peça                      | Papel                                                                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ingress Controller</b> | Proxy reverso que roda no cluster (NGINX, Traefik, HAProxy, Contour, Kong, controladores das nuvens) e lê os objetos Ingress. Normalmente exposto por um Service LoadBalancer |
| <b>IngressClass</b>       | Diz qual controlador atende qual Ingress (spec.ingressClassName). Uma classe pode ser marcada como padrão                                                                     |
| <b>Ingress</b>            | Regras: host, caminho, Service de destino, TLS                                                                                                                                |
| <b>Annotations</b>        | Configurações específicas <b>do controlador</b> (reescrita, autenticação, limites). Não são portáteis entre controladores                                                     |

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: loja
  namespace: loja
spec:
  ingressClassName: nginx
  tls:
    - hosts: [loja.exemplo.com.br]
      secretName: loja-tls # Secret TLS no MESMO namespace do Ingress
  rules:
    - host: loja.exemplo.com.br
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: loja-web # Service no MESMO namespace
                port: { number: 5000 }
          - path: /api
            pathType: Prefix
            backend:
              service: { name: loja-api, port: { number: 8080 } }
```

|          |             |
|----------|-------------|
| pathType | Casa quando |
|----------|-------------|

| pathType               | Casa quando                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------------|
| Exact                  | O caminho é exatamente igual                                                                     |
| Prefix                 | O caminho começa com o prefixo, por <b>segmentos</b> ( /api casa /api e /api/v1, mas não /apiv1) |
| ImplementationSpecific | Depende do controlador                                                                           |

- **Cuidado com a estrutura das regras:** `host` e `http` pertencem ao **mesmo item** da lista `rules`. Escrever `- host: x` e `- http: ...` como dois itens cria uma regra sem caminhos e outra **sem host**, que passa a responder para **qualquer** nome.
- **Reescrita de caminho** (`nginx.ingress.kubernetes.io/rewrite-target`) serve para publicar a aplicação em um subcaminho ( /app ) quando ela espera / . Aplicações que geram links absolutos (CSS, JS) costumam quebrar com isso; a solução definitiva é a aplicação aceitar um prefixo configurável, ou publicar em um host próprio.
- **Testar sem DNS:** `curl -H "Host: loja.exemplo.com.br" http://<IP-do-controlador>/` ou uma entrada no `/etc/hosts`.
- **No kind**, o controlador precisa receber as portas 80 e 443 do host: crie o cluster com `extraPortMappings` e a label `ingress-ready=true` no nó e instale a variante "kind" do manifesto do controlador.

### 12.1.2 ingress-nginx aposentado

- O projeto **kubernetes/ingress-nginx** (o "NGINX Ingress" da comunidade Kubernetes) **encerrou a manutenção em março de 2026**: não há mais versões, correções nem **patches de segurança**. Instalações existentes continuam funcionando, e as imagens e charts seguem disponíveis, mas o risco cresce com o tempo.
- **Não confunda** com o **NGINX Ingress Controller da F5/NGINX** (`nginx/kubernetes-ingress`), outro projeto, ainda mantido, com annotations diferentes (`nginx.org/...`).
- **Caminhos de migração:** Gateway API com uma implementação mantida (Envoy Gateway, NGINX Gateway Fabric, Istio, Cilium, Traefik, Kong, controladores das nuvens) ou outro controlador de Ingress mantido. A ferramenta **ingress2gateway** converte Ingress em recursos da Gateway API.
- As annotations `nginx.ingress.kubernetes.io/*` usadas em tutoriais (autenticação básica, afinidade por cookie, *upstream hash*, canário por peso, limite de requisições) só existem nesse controlador. Na Gateway API, várias dessas funções viraram campos padronizados.

| Recurso no ingress-nginx (annotation)             | Efeito                                                                                           |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>auth-type: basic + auth-secret</code>       | Pede usuário e senha (arquivo <code>htpasswd</code> em um Secret com a chave <code>auth</code> ) |
| <code>affinity: cookie</code>                     | Mantém o usuário no mesmo Pod por meio de um cookie                                              |
| <code>upstream-hash-by: "\$request_uri"</code>    | Escolhe o Pod por hash de um valor da requisição                                                 |
| <code>canary: "true" + canary-weight: "10"</code> | Envia 10% do tráfego para outro Service (um segundo Ingress com o mesmo host)                    |
| <code>limit-rps: "10"</code>                      | Limita requisições por segundo por IP de cliente; excedentes recebem erro (503 por padrão)       |

## 12.2 Gateway API

### 12.2.1 Modelo

| Recurso             | Quem mantém               | Papel                                    |
|---------------------|---------------------------|------------------------------------------|
| <b>GatewayClass</b> | Provedor da implementação | "Qual controlador" (como a IngressClass) |

| Recurso                                                      | Quem mantém                  | Papel                                                                                   |
|--------------------------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------|
| <b>Gateway</b>                                               | Time de plataforma           | Pontos de entrada: portas, protocolos, certificados, quais namespaces podem se conectar |
| <b>HTTPRoute, GRPCRoute</b> (e TLSRoute, TCPRoute, UDPRoute) | Times das aplicações         | Regras de roteamento para os Services                                                   |
| <b>ReferenceGrant</b>                                        | Dono do recurso referenciado | Autoriza referências entre namespaces                                                   |

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: publico
  namespace: infra
spec:
  gatewayClassName: envoy-gateway          # depende da implementação instalada
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      hostname: "*.exemplo.com.br"
      tls:
        certificateRefs: [{ name: wildcard-tls }]
      allowedRoutes:
        namespaces: { from: All }
  ---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: loja
  namespace: loja
spec:
  parentRefs: [{ name: publico, namespace: infra }]
  hostnames: [loja.exemplo.com.br]
  rules:
    - matches: [{ path: { type: PathPrefix, value: / } }]
      backendRefs:
        - { name: loja-web-v1, port: 5000, weight: 90 }    # canário por peso, sem annotations
        - { name: loja-web-v2, port: 5000, weight: 10 }

```

- **Vantagens sobre o Ingress:** papéis separados (plataforma × aplicação), recursos padronizados (pesos, cabeçalhos, redirecionamentos, reescrita, espelhamento de tráfego) em vez de annotations proprietárias, suporte a gRPC, TCP e UDP.
- A Gateway API é instalada como **CRDs** (não vem por padrão em todo cluster) e precisa de uma **implementação** instalada.

## 12.3 Certificados com cert-manager

### 12.3.1 Como funciona

O **cert-manager** (projeto CNCF, versão 1.21 em set/2026) pede, instala e **renova** certificados automaticamente, a partir de autoridades como **Let's Encrypt** (protocolo **ACME**), Vault, Venafi ou uma CA própria.

| Recurso                              | Papel                                                           |
|--------------------------------------|-----------------------------------------------------------------|
| Issuer / ClusterIssuer               | Autoridade e forma de validação (um namespace / o cluster todo) |
| Certificate                          | Certificado desejado; resulta em um Secret TLS                  |
| CertificateRequest, Order, Challenge | Etapas internas da emissão (úteis para depurar)                 |

| Desafio ACME | Como prova a posse do domínio                                                            | Observações                                                                                  |
|--------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| HTTP-01      | Publica um arquivo em <code>http://&lt;domínio&gt;/.well-known/acme-challenge/...</code> | Exige a porta 80 acessível da internet; <b>não</b> emite curingas ( <code>*.domínio</code> ) |
| DNS-01       | Cria um registro TXT no DNS                                                              | Emite curingas; precisa de credenciais do provedor de DNS                                    |

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory # staging: acme-staging-v02...
    privateKeySecretRef: { name: letsencrypt-prod-conta }
    solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

- No Ingress, basta a annotation `cert-manager.io/cluster-issuer: letsencrypt-prod` (ou `cert-manager.io/issuer` para um Issuer do namespace) e o bloco `tls` com `secretName`: o cert-manager cria o Certificate e mantém o Secret. Com a Gateway API, o cert-manager também emite certificados a partir dos listeners do Gateway.
- Teste com o ambiente staging** do Let's Encrypt: ele tem limites de emissão bem mais altos. Erros repetidos no ambiente de produção esbarram nos **limites de emissão** e bloqueiam novos certificados por um tempo.
- Depurar:** `kubectl get certificate,certificaterequest,order,challenge -A` e `kubectl describe` do recurso preso.

## 12.4 Decisão rápida — Entrada de tráfego

| Situação                                   | Abordagem recomendada                                                                       | Erro comum                                  |
|--------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------|
| Cluster novo precisa de entrada HTTP/HTTPS | Gateway API com uma implementação mantida                                                   | Instalar o ingress-nginx, já aposentado     |
| Cluster existente com ingress-nginx        | Planejar migração (Gateway API ou controlador mantido); ingress2gateway ajuda               | Ignorar a ausência de patches de segurança  |
| Vários sites em um único IP                | Regras por host (Ingress ou HTTPRoute)                                                      | Um LoadBalancer por site                    |
| Ingress não pega o tráfego                 | <code>ingressClassName</code> certo, Service e Secret no mesmo namespace                    | Criar o Ingress em outro namespace          |
| Regra responde para qualquer host          | Conferir se <code>host</code> e <code>http</code> estão no mesmo item de <code>rules</code> | Separar host e caminhos em itens diferentes |
| HTTPS com renovação automática             | cert-manager + ClusterIssuer ACME                                                           | Certificado manual que expira               |
| Certificado curinga                        | DNS-01                                                                                      | HTTP-01                                     |

| Situação                       | Abordagem recomendada                                                   | Erro comum                                   |
|--------------------------------|-------------------------------------------------------------------------|----------------------------------------------|
| Testar emissão de certificados | ACME staging primeiro                                                   | Repetir tentativas na produção               |
| Canário por porcentagem        | <code>weight</code> em HTTPRoute (ou annotations do controlador em uso) | Duplicar Deployments sem controle de tráfego |
| Proteger um painel com senha   | Autenticação no controlador ou em um proxy de identidade (OAuth2 Proxy) | Deixar o painel público                      |

## Metrics Server e Autoescalonamento

**Ideia central:** o **Metrics Server** coleta CPU e memória de nós e Pods para o `kubectl top` e para o **HorizontalPodAutoscaler (HPA)**, que ajusta o número de réplicas para manter uma métrica perto de um alvo. O HPA calcula a utilização **em relação aos requests**: sem requests, não há o que escalar. Para métricas de negócio e eventos, entram adaptadores de métricas e o **KEDA**; para nós, o **Cluster Autoscaler** ou o **Karpenter**.

### 13.1 Metrics Server

#### 13.1.1 Instalação e uso

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
minikube addons enable metrics-server # no Minikube

# kind e clusters de laboratório com certificados autoassinados no kubelet:
kubectl patch deployment metrics-server -n kube-system --type=json \
  -p '[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'

kubectl get apiservice v1beta1.metrics.k8s.io # AVAILABLE deve ser True
kubectl top nodes
kubectl top pods -A --sort-by=memory
kubectl top pod <pod> --containers
```

- O Metrics Server lê os dados do **kubelet** de cada nó (por padrão a cada 15 s) e os expõe pela API agregada `metrics.k8s.io`. Ele guarda só o valor mais recente, em memória.
- **Não é ferramenta de monitoramento:** não tem histórico, gráficos nem alertas. Para isso, use Prometheus (ou o serviço de observabilidade da nuvem).
- Erro `Metrics API not available` ou `x509: cannot validate certificate`: o Metrics Server não consegue falar com os kubelets. Em clusters de estudo, `--kubelet-insecure-tls` resolve; em produção, o certo é o kubelet ter certificados válidos.

## 13.2 HorizontalPodAutoscaler

### 13.2.1 Manifesto

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60          # % do REQUEST de CPU, na média dos Pods
    - type: Resource
      resource:
        name: memory
        target:
          type: AverageValue
          averageValue: 400Mi
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - { type: Percent, value: 100, periodSeconds: 15 }    # pode dobrar a cada 15 s
    scaleDown:
      stabilizationWindowSeconds: 300                          # olha 5 min antes de reduzir
      policies:
        - { type: Pods, value: 2, periodSeconds: 60 }        # remove no máximo 2 Pods por minuto
```

```
kubectl autoscale deployment web --cpu-percent=60 --min=3 --max=10    # forma imperativa
kubectl get hpa -w              # TARGETS (atual/alvo), MINPODS, MAXPODS, REPLICAS
kubectl describe hpa web        # condições e eventos de cada decisão
```

- A API atual é `autoscaling/v2`. As versões `v2beta1` e `v2beta2` foram **removidas** (1.25 e 1.26); exemplos com elas não são aceitos por clusters atuais.
- Métrica** `ContainerResource` (estável desde a 1.30) usa o consumo de **um contêiner** do Pod, ignorando sidecars: `type: ContainerResource` com `container: <nome>`.

### 13.2.2 Algoritmo

**réplicas desejadas** =  $\lceil \text{réplicas atuais} \times (\text{valor atual da métrica} \div \text{valor alvo}) \rceil$

| Situação | Cálculo | Resultado |
|----------|---------|-----------|
|----------|---------|-----------|

| Situação                               | Cálculo                                          | Resultado                                      |
|----------------------------------------|--------------------------------------------------|------------------------------------------------|
| 2 réplicas, CPU média em 90%, alvo 60% | $\lceil 2 \times 90/60 \rceil = \lceil 3 \rceil$ | 3 réplicas                                     |
| 4 réplicas, CPU em 80%, alvo 50%       | $\lceil 4 \times 1,6 \rceil = \lceil 6,4 \rceil$ | 7 réplicas                                     |
| 5 réplicas, CPU em 30%, alvo 50%       | $\lceil 5 \times 0,6 \rceil$                     | 3 réplicas (depois da janela de estabilização) |

- O controlador roda a cada **15 s** e ignora variações dentro de uma **tolerância de 10%** (sem mudança se a razão estiver entre 0,9 e 1,1).
- **Várias métricas:** calcula o número de réplicas para cada uma e usa o **maior**.
- **Pods que ainda não estão prontos** ou sem métricas são tratados de forma conservadora para não provocar escalonamento errado durante a inicialização. Readiness e startup probes bem configuradas ajudam.
- A redução espera a **janela de estabilização** (padrão **300 s**) para evitar oscilações; o aumento é imediato por padrão.
- **Requests obrigatórios:** `averageUtilization` é uma porcentagem do **request**. Contêiner sem request de CPU → o HPA mostra `<unknown>` e não age.

### 13.2.3 Teste de carga

```
kubectl create deployment web --image=nginx:1.29
kubectl set resources deployment web --requests=cpu=100m --limits=cpu=200m
kubectl expose deployment web --port=80 # o gerador de carga precisa do Service
kubectl autoscale deployment web --cpu-percent=50 --min=1 --max=10
kubectl run carga -it --rm --image=busybox:1.37 --restart=Never -- \
  sh -c 'while true; do wget -q -O- http://web > /dev/null; done'
kubectl get hpa web -w # em outro terminal
```

### 13.2.4 Outros escaladores

| Ferramenta                                                  | O que escala                                                                                                       | Quando usar                                                     |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>HPA</b> com métricas personalizadas (Prometheus Adapter) | Réplicas, por métricas da aplicação (requisições por segundo, latência)                                            | CPU e memória não refletem a carga real                         |
| <b>KEDA</b> (CNCF)                                          | Réplicas, a partir de <b>eventos</b> (tamanho de fila, Kafka, cron, métricas externas), inclusive <b>para zero</b> | Workers de fila, cargas intermitentes                           |
| <b>VerticalPodAutoscaler</b>                                | <b>Requests e limits</b> dos contêineres                                                                           | Descobrir o tamanho certo; não combine com HPA na mesma métrica |
| <b>Cluster Autoscaler / Karpenter</b>                       | <b>Nós</b> do cluster                                                                                              | Pods Pending por falta de capacidade; nós ociosos               |

- HPA e o número de nós se complementam: o HPA cria Pods; se não houver espaço, eles ficam `Pending`, e o auto-escalador de nós adiciona máquinas.
- Com HPA, **retire** replicas **do manifesto** do Deployment (ou do `values.yaml` do Helm), senão cada `apply` volta o número ao valor fixo.

## 13.3 Decisão rápida — Autoescalamento

| Situação | Abordagem recomendada | Erro comum |
|----------|-----------------------|------------|
|----------|-----------------------|------------|

| Situação                                            | Abordagem recomendada                                                            | Erro comum                             |
|-----------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------|
| <code>kubectl top</code> retorna erro               | Instalar o Metrics Server (e <code>--kubelet-insecure-tls</code> em laboratório) | Procurar métricas no Prometheus        |
| HPA com <code>TARGETS &lt;unknown&gt;</code>        | Definir requests de CPU/memória e conferir o Metrics Server                      | Aumentar <code>maxReplicas</code>      |
| Escalar por fila de mensagens                       | KEDA                                                                             | HPA por CPU                            |
| Réplicas oscilando                                  | <code>behavior.scaleDown.stabilizationWindowSeconds</code> e políticas de ritmo  | Diminuir o alvo repetidamente          |
| Pods novos ficam <code>Pending</code> durante picos | Cluster Autoscaler ou Karpenter                                                  | Só aumentar o <code>maxReplicas</code> |
| Não sabe quanto pedir de CPU e memória              | VPA em modo recomendação                                                         | Chutar valores e nunca revisar         |
| <code>kubectl apply</code> desfaz o escalonamento   | Tirar <code>replicas</code> do manifesto                                         | Brigar com o HPA a cada deploy         |
| Monitorar histórico de uso                          | Prometheus/Grafana                                                               | Metrics Server                         |

# Agendamento: Taints, Tolerations e Afinidade

**Ideia central:** o scheduler decide **onde** cada Pod roda. **Labels nos nós + nodeSelector/nodeAffinity** atraem Pods para certos nós. **Taints** repelem Pods de nós, e **tolerations** permitem (mas não obrigam) que um Pod ignore um taint. **podAffinity/podAntiAffinity** e **topologySpreadConstraints** aproximam ou espalham Pods em relação a outros Pods, por nó, zona ou região.

## 14.1 Labels nos nós

### 14.1.1 Organizar o cluster

```
kubectl label nodes worker-1 topology.kubernetes.io/region=br-sp topology.kubernetes.io/zone=br-sp-1
kubectl label nodes worker-1 gpu=true
kubectl label nodes worker-1 gpu- # remove a label
kubectl get nodes -L topology.kubernetes.io/zone,gpu
kubectl get nodes --show-labels
```

- **Labels bem conhecidas:** `kubernetes.io/hostname`, `kubernetes.io/os`, `kubernetes.io/arch`, `node.kubernetes.io/instance-type`, `topology.kubernetes.io/region`, `topology.kubernetes.io/zone`. Em nuvem, as de topologia e tipo de instância já vêm preenchidas. Use essas chaves em vez de inventar `region` e `datacenter`: ferramentas do ecossistema as entendem.
- `nodeSelector`, a forma mais simples de escolher nós: `spec.nodeSelector: { gpu: "true" }` (valores são strings: use aspas em `"true"`).

## 14.2 Taints e tolerations

### 14.2.1 Efeitos

| Efeito           | Pods novos sem toleration                                   | Pods que já estão no nó                                                                |
|------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------|
| NoSchedule       | Não são agendados                                           | Continuam                                                                              |
| PreferNoSchedule | O scheduler <b>evita</b> , mas usa o nó se não houver outro | Continuam                                                                              |
| NoExecute        | Não são agendados                                           | <b>São despejados</b> (os que toleram podem ficar por <code>tolerationSeconds</code> ) |

```
kubectl taint nodes worker-1 gpu=true:NoSchedule # aplica
kubectl taint nodes worker-1 gpu=true:NoSchedule- # remove (sufixo -)
kubectl describe node worker-1 | grep Taints
```

```
spec:
  tolerations:
    - key: gpu
      operator: Equal          # ou Exists (qualquer valor da chave)
      value: "true"
      effect: NoSchedule
  nodeSelector:
    gpu: "true"               # toleration + seletor = roda SÓ nos nós com GPU
```

- **Toleration não atrai:** ela só permite que o Pod vá para o nó com taint. Para **garantir** que vá, combine com `nodeSelector` ou `nodeAffinity`. Para **reservar** nós a certos Pods, use os dois: taint (afasta os outros) e afinidade (atrai os escolhidos).
- **Taints automáticos:** o control plane recebe `node-role.kubernetes.io/control-plane:NoSchedule`; nós com problemas recebem `node.kubernetes.io/not-ready` e `node.kubernetes.io/unreachable` (`NoExecute`), e os Pods comuns ganham tolerations automáticas de **300 s** para eles: é por isso que um Pod leva cerca de 5 minutos para ser recriado em outro nó quando o nó dele cai. Há também taints de pressão (`memory-pressure`, `disk-pressure`).
- **Reorganizar Pods:** remover um taint `NoSchedule` não move os Pods que já estão em outros nós. O scheduler só age em Pods **novos**; um `kubectl rollout restart` redistribui. Para rebalanceamento contínuo, existe o **Descheduler**.
- **Manutenção de nó:** prefira `kubectl cordon + kubectl drain`, que respeitam **PodDisruptionBudgets** e encerram os Pods de forma graciosa, a aplicar um taint `NoExecute` à mão.

## 14.3 Afinidade e antiafinidade

### 14.3.1 nodeAffinity

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:      # obrigatório
        nodeSelectorTerms:
          - matchExpressions:
              - key: gpu
                operator: In
                values: ["true"]
        preferredDuringSchedulingIgnoredDuringExecution:    # preferência (peso 1-100)
          - weight: 80
            preference:
              matchExpressions:
                - key: topology.kubernetes.io/zone
                  operator: In
                  values: [br-sp-1]
```

- `required...`: se nenhum nó atender, o Pod fica `Pending`. `preferred...`: soma pontos para os nós que atendem.
- `IgnoredDuringExecution`: as regras valem **no agendamento**. Se a label do nó mudar depois, o Pod continua lá.
- Operadores: `In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`. Vários `nodeSelectorTerms` são combinados com **OU**; várias `matchExpressions` do mesmo termo, com **E**.

### 14.3.2 podAffinity, podAntiAffinity e distribuição

```
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels: { app: web }
            topologyKey: topology.kubernetes.io/zone      # evite duas réplicas na mesma zona
      topologySpreadConstraints:
        - maxSkew: 1                                     # diferença máxima entre domínios
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway               # ou DoNotSchedule
          labelSelector:
            matchLabels: { app: web }
```

- `topologyKey` define o "domínio": `kubernetes.io/hostname` (nó), `topology.kubernetes.io/zone`, `topology.kubernetes.io/region` ou qualquer label dos nós.
- **podAffinity** aproxima (ex.: cache no mesmo nó da aplicação); **podAntiAffinity** afasta (réplicas em nós ou zonas diferentes).
- **Antiafinidade obrigatória limita as réplicas ao número de domínios:** com `required` e 4 zonas, a quinta réplica fica `Pending` para sempre. Para alta disponibilidade, prefira a forma `preferred` ou `topologySpreadConstraints` com `maxSkew`, que espalham sem travar o escalonamento.
- Afinidade entre Pods é cara de calcular em clusters muito grandes; `topologySpreadConstraints` é a forma recomendada de distribuir réplicas.

### 14.3.3 Prioridade e disrupções

- **PriorityClass** dá prioridade a Pods; quando falta espaço, o scheduler pode **preemptar** (despejar) Pods de menor prioridade para agendar os mais importantes.
- **PodDisruptionBudget** (`minAvailable` ou `maxUnavailable`) limita quantos Pods de uma aplicação podem ser tirados **voluntariamente** ao mesmo tempo (drain, atualizações de nós).

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web
spec:
  minAvailable: 2
  selector:
    matchLabels: { app: web }
```

## 14.4 Decisão rápida — Agendamento

| Situação                          | Abordagem recomendada                                            | Erro comum                                     |
|-----------------------------------|------------------------------------------------------------------|------------------------------------------------|
| Nós com GPU só para cargas de GPU | Taint nos nós de GPU + toleration e nodeAffinity nos Pods de GPU | Só toleration (o Pod pode ir para qualquer nó) |

| Situação                                                               | Abordagem recomendada                                                                      | Erro comum                                                      |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Réplicas em zonas diferentes                                           | <code>topologySpreadConstraints</code> por zona (ou antiafinidade <code>preferred</code> ) | Antiafinidade <code>required</code> com mais réplicas que zonas |
| Manutenção de nó                                                       | <code>kubectl drain</code> (respeita PDB) e <code>uncordon</code>                          | Taint <code>NoExecute</code> improvisado                        |
| Pod Pending com <code>didn't match Pod's node affinity/selector</code> | Conferir labels dos nós e valores com aspas                                                | Remover a afinidade                                             |
| Garantir mínimo de réplicas durante manutenções                        | <code>PodDisruptionBudget</code>                                                           | Confiar no número de réplicas                                   |
| Cargas críticas precisam de espaço em cluster cheio                    | <code>PriorityClass</code>                                                                 | Aumentar requests de tudo                                       |
| Cache perto da aplicação                                               | <code>podAffinity</code> por <code>kubernetes.io/hostname</code>                           | Fixar os dois no mesmo nó pelo nome                             |
| Nós do control plane rodando aplicações                                | Manter o taint padrão                                                                      | Remover o taint "para aproveitar o nó"                          |
| Pods levam 5 minutos para sair de um nó morto                          | É a toleration automática de 300 s; ajuste <code>tolerationSeconds</code> se precisar      | Achar que o cluster travou                                      |

## Políticas e Controle de Admissão

**Ideia central:** depois de autenticar e autorizar um pedido, o API server passa o objeto por **controladores de admissão**, que podem **modificar** (mutating) e **validar** (validating) antes de gravar no etcd. É nesse ponto que se impõem regras como "toda imagem vem do registry da empresa" ou "nenhum contêiner roda como root". As ferramentas são o **Pod Security Admission** (nativo), a **ValidatingAdmissionPolicy** (nativa, em CEL) e motores de políticas como o **Kyverno** e o **OPA Gatekeeper**.

### 15.1 O caminho de um pedido

#### 15.1.1 Etapas

1. **Autenticação:** quem é você (certificado, token, OIDC).
  2. **Autorização:** você pode fazer isso (RBAC, capítulo 17).
  3. **Admissão com mutação:** webhooks e políticas podem **alterar** o objeto (adicionar labels, valores padrão, sidecars).
  4. **Validação de esquema** do objeto.
  5. **Admissão com validação:** webhooks e políticas **aceitam ou recusam**.
  6. **Gravação** no etcd.
- Admissão age sobre **pedidos** de criação e alteração. Objetos que já existiam antes da política só são afetados quando mudam; relatórios de auditoria mostram quais estão fora da regra.

### 15.2 Pod Security Admission

#### 15.2.1 Padrões por namespace

O **Pod Security Admission** (estável desde a 1.25) aplica os **Pod Security Standards** por meio de labels no namespace:

| Nível      | O que exige                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------|
| privileged | Nada: sem restrições                                                                                                |
| baseline   | Bloqueia escaladas conhecidas: contêineres privilegiados, hostNetwork, hostPID, hostPath, capabilities perigosas    |
| restricted | Além do baseline: runAsNonRoot, sem escalada de privilégio, drop: [ALL] capabilities, perfil seccomp RuntimeDefault |

```
kubectl label namespace loja \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

- Modos: `enforce` (recusa), `warn` (avisa o usuário) e `audit` (registra no log de auditoria). Comece com `warn` e `audit` para ver o impacto antes do `enforce`.
- Substitui a antiga **PodSecurityPolicy**, removida na 1.25.

## 15.3 ValidatingAdmissionPolicy

### 15.3.1 Regras em CEL, sem webhook

Estável desde a 1.30, a **ValidatingAdmissionPolicy** escreve regras em **CEL** que o próprio API server executa, sem instalar nada. A **MutatingAdmissionPolicy** segue o mesmo modelo para mutação nas versões recentes.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: exigir-limits
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  validations:
    - expression: >
        object.spec.template.spec.containers.all(c,
          has(c.resources.limits) && has(c.resources.limits.memory))
      message: "Todo contêiner precisa de limite de memória."
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: exigir-limits
spec:
  policyName: exigir-limits
  validationActions: [Deny]          # ou Warn, Audit
  matchResources:
    namespaceSelector:
      matchLabels: { ambiente: producao }
```

## 15.4 Kyverno

### 15.4.1 O que faz

O **Kyverno** (projeto CNCF) é um motor de políticas feito para Kubernetes: **valida, modifica, gera e apaga** recursos e **verifica assinaturas de imagens**, com políticas escritas como objetos do cluster.

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm install kyverno kyverno/kyverno -n kyverno --create-namespace
kubectl get pods -n kyverno
kubectl get crd | grep kyverno
```

### 15.4.2 Tipos de política

| Tipo (Kyverno 1.19)                    | Função                                                                                                                                         |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| ValidatingPolicy                       | Aceitar ou recusar recursos (CEL)                                                                                                              |
| MutatingPolicy                         | Alterar recursos na admissão                                                                                                                   |
| GeneratingPolicy                       | Criar recursos a partir de outros (ex.: NetworkPolicy padrão em cada namespace novo)                                                           |
| DeletingPolicy                         | Apagar recursos periodicamente por critério                                                                                                    |
| ImageValidatingPolicy                  | Verificar assinaturas e atestados de imagens                                                                                                   |
| ClusterPolicy / Policy (kyverno.io/v1) | Formato antigo, baseado em padrões YAML. <b>Obsoleto</b> a partir do Kyverno 1.19, mas ainda muito presente em tutoriais e clusters existentes |

```
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: registry-confiavel
spec:
  validationActions: [Deny]           # ou Audit para só relatar
  matchConstraints:
    resourceRules:
      - apiGroups: [""]
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: "Imagens devem vir de registry.empresa.com.br"
      expression: >
        object.spec.containers.all(c, c.image.startsWith('registry.empresa.com.br/')) &&
        object.spec?.initContainers.orValue([]).all(c, c.image.startsWith('registry.empresa.com.br/'))
```

No formato antigo, ainda comum:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: exigir-limits
spec:
  rules:
    - name: validar-limits
      match:
        any:
          - resources: { kinds: [Pod] }
      exclude:
        any:
          - resources: { namespaces: [kube-system] }
      validate:
        failureAction: Enforce # por regra; o campo spec.validationFailureAction é antigo
        message: "CPU e memória precisam de limits."
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    memory: "?*"
                    cpu: "?*"

```

- **Enforce x Audit:** `Enforce` (ou `Deny`) recusa; `Audit` deixa passar e registra em **PolicyReports** (`kubectl get policyreport -A`). Comece em `Audit`.
- **Regras de Pod valem para os controladores:** o Kyverno gera automaticamente as regras equivalentes para Deployments, StatefulSets, Jobs e outros, recusando o Deployment na hora, em vez de deixar os Pods falharem depois.
- **Cuidados:** regras que olham só `containers` deixam `initContainers` e `ephemeralContainers` de fora; um padrão que exige um valor exato (por exemplo, `readiness` em `/` na porta 8080) recusa aplicações corretas que usam outro caminho. Exclua namespaces de sistema com cuidado e rode o Kyverno com réplicas: se o webhook cair e a política for `Fail`, o cluster deixa de aceitar alterações.
- **Teste antes de aplicar:** a CLI `kyverno apply` e `kyverno test` avaliam políticas contra manifestos locais, útil no CI.
- **Alternativa:** o **OPA Gatekeeper** usa a linguagem Rego e o modelo de `ConstraintTemplates`.

## 15.5 Decisão rápida — Políticas

| Situação                                             | Abordagem recomendada                                                       | Erro comum                                     |
|------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------|
| Bloquear Pods privilegiados e root por namespace     | Pod Security Admission ( <code>restricted</code> ou <code>baseline</code> ) | Escrever tudo do zero em um motor de políticas |
| Regra simples de validação, sem instalar nada        | ValidatingAdmissionPolicy (CEL)                                             | Webhook próprio                                |
| Mutação, geração de recursos, verificação de imagens | Kyverno                                                                     | Scripts que corrigem recursos depois           |
| Introduzir políticas em cluster em uso               | Modo Audit/Warn, analisar relatórios, depois Enforce                        | Enforce direto e quebrar deploys               |
| Restringir registries de imagens                     | Política que cobre <code>containers</code> e <code>initContainers</code>    | Checar só <code>containers</code>              |

| Situação                                          | Abordagem recomendada                                    | Erro comum                                       |
|---------------------------------------------------|----------------------------------------------------------|--------------------------------------------------|
| Webhook de política fora do ar travando o cluster | Réplicas e exclusão de namespaces de sistema             | Uma réplica com <code>failurePolicy: Fail</code> |
| Escrever políticas novas no Kyverno               | Tipos novos ( <code>ValidatingPolicy</code> e similares) | Novos <code>ClusterPolicy</code> sem necessidade |

# Network Policies

**Ideia central:** por padrão, **todo Pod fala com todo Pod** do cluster. Uma **NetworkPolicy** seleciona Pods e define quem pode falar com eles (**ingress**) e com quem eles podem falar (**egress**). As políticas são **listas de permissão que se somam**: a partir do momento em que um Pod é selecionado por uma política de uma direção, tudo o que não for permitido naquela direção é bloqueado. E nada disso funciona sem um **CNI que implemente NetworkPolicy**.

## 16.1 Como funcionam

### 16.1.1 Regras fundamentais

- **Suporte do CNI:** o API server **aceita** NetworkPolicies em qualquer cluster (a API `networking.k8s.io/v1` sempre existe), mas quem as aplica é o plugin de rede. **Calico** e **Cilium** aplicam; o **Flannel** sozinho, não; na AWS, o **VPC CNI** aplica quando a opção de NetworkPolicy está habilitada no add-on. Verificar se a API existe (`kubectl api-versions`) **não** prova que as políticas são aplicadas: teste o bloqueio de verdade.
- **Pod não selecionado** por nenhuma política: tudo liberado.
- **Pod selecionado** em uma direção: só entra ou sai o que alguma política permitir. Políticas **somam** permissões; não existe regra de "negar" na API padrão.
- `policyTypes`: se omitido, a política vale para `Ingress` e, se tiver seção `egress`, também para `Egress`. Declare explicitamente.
- A **resposta** de uma conexão permitida sempre volta (as regras são *stateful*).

### 16.1.2 Seletores: E e OU

```
ingress:
- from:
  - namespaceSelector:           # item 1
    matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
  - podSelector:                 # item 2 → item 1 OU item 2
    matchLabels: { app: web }
- from:
  - namespaceSelector:           # MESMO item: namespace E Pod
    matchLabels: { kubernetes.io/metadata.name: monitoring }
    podSelector:
      matchLabels: { app: prometheus }
```

- **Itens separados** na lista `from/ to` (cada um começa com `-`) são alternativas (**OU**).
- `namespaceSelector` e `podSelector` **no mesmo item** significam "**Pods com estas labels, nos namespaces com estas labels**" (**E**).
- Não é possível repetir a mesma chave (`namespaceSelector` duas vezes) no mesmo item: em YAML isso é uma chave duplicada, e só uma das duas vale.
- `podSelector` **sem** `namespaceSelector` se refere a Pods **do mesmo namespace** da política. `podSelector: {}` significa todos os Pods daquele namespace.

- Todo namespace tem automaticamente a label `kubernetes.io/metadata.name: <nome>`, útil para selecionar namespaces pelo nome.
- `ipBlock` (com `except`) serve para endereços **fora do cluster**. IPs de Pods mudam; para Pods, use seletores.

## 16.2 Receitas

### 16.2.1 Negar tudo e liberar o necessário

#### 1. Negar tudo (entrada e saída) no namespace:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: negar-tudo, namespace: loja }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

#### 2. Liberar o DNS para todos os Pods do namespace:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: permitir-dns, namespace: loja }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: kube-system }
        podSelector:
            matchLabels: { k8s-app: kube-dns }
  ports:
    - { protocol: UDP, port: 53 }
    - { protocol: TCP, port: 53 }
```

#### 3. Entrada na aplicação só a partir do controlador de entrada:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: web-do-ingress, namespace: loja }
spec:
  podSelector:
    matchLabels: { app: web }
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
  ports:
    - { protocol: TCP, port: 5000 }
```

#### 4. Aplicação → Redis: saída na aplicação e entrada no Redis:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: web-para-redis, namespace: loja }
spec:
  podSelector:
    matchLabels: { app: web }
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector: { matchLabels: { app: redis } }
      ports:
        - { protocol: TCP, port: 6379 }
  ---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: redis-da-web, namespace: loja }
spec:
  podSelector:
    matchLabels: { app: redis }
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector: { matchLabels: { app: web } }
      ports:
        - { protocol: TCP, port: 6379 }

```

- Com "negar tudo" nas duas direções, uma comunicação só funciona com **duas permissões: egress** na origem e **ingress** no destino.
- **Não esqueça o DNS** (UDP e TCP 53): sem ele, nomes de Services não resolvem, e o sintoma parece "a aplicação não acha o Redis".
- **Portas** em NetworkPolicy são as do **Pod** (a `targetPort`), não a porta do Service.

### 16.2.2 Testar

```

kubectl get networkpolicy -n loja
kubectl describe networkpolicy web-do-ingress -n loja
# De fora do namespace (deve falhar com timeout)
kubectl run t -it --rm --image=redis:8 --restart=Never -- redis-cli -h redis.loja ping
# De dentro, com a label permitida (deve responder PONG)
kubectl run t -n loja -it --rm --image=redis:8 --restart=Never --labels=app=web -- redis-cli -h redis ping

```

- Bloqueio por NetworkPolicy aparece como **timeout** (pacotes descartados), não como "conexão recusada".
- CNIs como **Cilium** e **Calico** oferecem políticas estendidas: regras por nome DNS (FQDN), por método e caminho HTTP (L7) e políticas de cluster inteiro que o administrador impõe acima dos namespaces.

## 16.3 Decisão rápida — Network Policies

| Situação                                | Abordagem recomendada                                                       | Erro comum                                   |
|-----------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------|
| Políticas criadas, mas nada é bloqueado | Conferir se o CNI aplica NetworkPolicy (Calico, Cilium, VPC CNI habilitado) | Confiar em <code>kubectl api-versions</code> |

| Situação                              | Abordagem recomendada                                | Erro comum                       |
|---------------------------------------|------------------------------------------------------|----------------------------------|
| Isolar um namespace                   | Negar tudo + liberações explícitas                   | Uma política por Pod, sem padrão |
| Tudo parou depois do "negar tudo"     | Liberar DNS (UDP e TCP 53) e as saídas necessárias   | Desfazer o isolamento            |
| Pods da origem X em namespaces Y      | namespaceSelector e podSelector no <b>mesmo</b> item | Itens separados (vira OU)        |
| Comunicação A → B com egress restrito | Egress em A <b>e</b> ingress em B                    | Só um dos lados                  |
| Liberar um serviço externo            | ipBlock (ou política FQDN do CNI)                    | ipBlock com IPs de Pods          |
| Porta errada na política              | Usar a porta do contêiner ( targetPort )             | Usar a porta do Service          |

## RBAC e Controle de Acesso

**Ideia central:** o Kubernetes **não tem objeto "usuário"**. Pessoas se autenticam de fora (certificados, OIDC) e **ServiceAccounts** identificam processos dentro do cluster. O **RBAC** define o que cada identidade pode fazer: **Roles/ClusterRoles** listam permissões (verbos sobre recursos) e **RoleBindings/ClusterRoleBindings** as concedem. Tudo é **permissão positiva**: o que não foi concedido, é negado.

### 17.1 Identidades

#### 17.1.1 Usuários, grupos e ServiceAccounts

| Identidade                 | Como é autenticada                                                                                                                                                      | Uso                                                                     |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Usuário</b><br>(pessoa) | Certificado X.509 ( <code>CN</code> = usuário, <code>O</code> = grupos), token <b>OIDC</b> do provedor de identidade, integração da nuvem (IAM no EKS, Entra ID no AKS) | Acesso humano                                                           |
| <b>Grupo</b>               | Vem junto da autenticação                                                                                                                                               | Conceder permissões a times                                             |
| <b>ServiceAccount</b>      | Token emitido pelo cluster (JWT), montado nos Pods                                                                                                                      | Aplicações e ferramentas dentro do cluster (CI, operadores, Prometheus) |

- **Em produção, prefira OIDC** (ou a integração da nuvem) para pessoas: certificados de cliente **não podem ser revogados** antes de expirar. Se usar certificados, dê validade curta.
- **Grupo** `system:masters` ignora o RBAC por completo. Por isso o `admin.conf` do kubeadm usa o grupo `kubeadm:cluster-admins`, e o arquivo `super-admin.conf` (com `system:masters`) deve ficar guardado para emergências.

### 17.1.2 Criar um usuário com certificado

```
openssl genrsa -out dev.key 2048
openssl req -new -key dev.key -out dev.csr -subj "/CN=maria/O=desenvolvimento"

cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: maria
spec:
  request: $(base64 < dev.csr | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 604800          # 7 dias
  usages: [client auth]
EOF

kubectl certificate approve maria
kubectl get csr maria -o jsonpath='{.status.certificate}' | base64 -d > dev.crt

kubectl config set-credentials maria --client-certificate=dev.crt --client-key=dev.key --embed-
certs=true
kubectl config set-context maria --cluster=<nome-do-cluster> --user=maria --namespace=dev
kubectl --context maria get pods      # Forbidden até existir um RoleBinding
```

- O certificado é assinado pela **CA do cluster**; a identidade é o que está no **CN** e no **O** do CSR. O objeto CSR aprova- do expira do cluster depois de algum tempo, mas o certificado continua válido até a data de expiração.

### 17.1.3 ServiceAccounts e tokens

```
kubectl create serviceaccount ci -n dev
kubectl create token ci -n dev --duration=1h          # token temporário (TokenRequest API)
```

```
spec:
  serviceAccountName: ci          # identidade do Pod
  automountServiceAccountToken: false # desligue quando o Pod não fala com a API
```

- Desde a **1.24**, criar uma ServiceAccount **não gera mais** um Secret com token de longa duração. Os Pods recebem **tokens projetados**, com validade limitada e renovados automaticamente pelo kubelet (em `/var/run/secrets/kubernetes.io/serviceaccount/`). Tokens de longa duração só existem se criados explicitamente (Secret do tipo `service-account-token`), e devem ser evitados.
- Todo namespace tem a ServiceAccount `default`, usada por Pods que não declaram outra. Não conceda permissões a ela.

## 17.2 Roles e bindings

### 17.2.1 Anatomia de uma regra

| Campo | Significado |
|-------|-------------|
|-------|-------------|

| Campo          | Significado                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| apiGroups      | Grupo da API: "" para o grupo principal (Pods, Services, ConfigMaps, Secrets), apps (Deployments, StatefulSets), batch, networking.k8s.io ... Veja a coluna APIVERSION de kubectl api-resources |
| resources      | Tipos no plural (pods, deployments) e <b>subrecursos</b> (pods/log, pods/exec, deployments/scale)                                                                                               |
| verbs          | get, list, watch, create, update, patch, delete, deletecollection (e especiais: bind, escalate, impersonate)                                                                                    |
| resource-Names | Opcional: limita a objetos com nomes específicos                                                                                                                                                |

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: desenvolvedor
  namespace: dev
rules:
- apiGroups: [""]
  resources: [pods, pods/log, services, configmaps]
  verbs: [get, list, watch, create, update, patch, delete]
- apiGroups: [apps]
  resources: [deployments]
  verbs: [get, list, watch, create, update, patch, delete]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: desenvolvedores
  namespace: dev
subjects:
- kind: Group                                # prefira grupos a usuários individuais
  name: desenvolvimento
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci
  namespace: dev
roleRef:
  kind: Role
  name: desenvolvedor
  apiGroup: rbac.authorization.k8s.io

```

| Objeto             | Escopo                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Role               | Permissões <b>dentro de um namespace</b>                                                                    |
| ClusterRole        | Permissões em recursos de cluster (nós, PVs, namespaces), em todos os namespaces, ou um modelo reutilizável |
| RoleBinding        | Concede uma Role <b>ou uma ClusterRole</b> dentro de <b>um</b> namespace                                    |
| ClusterRoleBinding | Concede uma ClusterRole no <b>cluster inteiro</b>                                                           |

- **RoleBinding + ClusterRole** é o jeito de reaproveitar um conjunto de permissões em vários namespaces sem copiá-lo.
- **ClusterRoles padrão:** view (leitura, sem Secrets), edit (altera a maioria dos recursos, sem mexer em RBAC), admin (tudo no namespace, inclusive RBAC local) e cluster-admin (tudo). Use-as antes de escrever as suas.
- O roleRef **é imutável:** para trocar a Role de um binding, apague e crie outro.

```
kubectl create role leitor-pods --verb=get,list,watch --resource=pods -n dev
kubectl create rolebinding maria-leitora --role=leitor-pods --user=maria -n dev
kubectl create clusterrolebinding plataforma-edit --clusterrole=edit --group=plataforma
kubectl auth can-i create deployments -n dev --as=maria
kubectl auth can-i --list -n dev --as=system:serviceaccount:dev:ci
kubectl auth whoami # quem sou eu para o cluster
```

### 17.2.2 Cuidados de segurança

- **Menor privilégio:** comece por `view` e acrescente. Evite `*` em verbos e recursos.
- **Criar Pods é poderoso:** quem pode criar Pods em um namespace pode montar qualquer Secret dele e usar qualquer ServiceAccount dele. `pods/exec` permite entrar em contêineres.
- `list` **em Secrets revela o conteúdo** (não só os nomes).
- **Verbos** `bind`, `escalate` e `impersonate` permitem ganhar mais permissões; reserve-os aos administradores.
- **Revise periodicamente** quem tem `cluster-admin`: `kubectl get clusterrolebindings -o wide | grep cluster-admin`.

### 17.3 Decisão rápida — RBAC

| Situação                                          | Abordagem recomendada                                           | Erro comum                                    |
|---------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------|
| Acesso de pessoas ao cluster                      | OIDC ou integração IAM da nuvem, por grupos                     | Cópias do <code>admin.conf</code>             |
| Laboratório com usuário por certificado           | CSR com <code>expirationSeconds</code> curto                    | Certificado de 1 ano sem forma de revogar     |
| Mesmo conjunto de permissões em vários namespaces | Uma ClusterRole + RoleBindings por namespace                    | Copiar a Role em cada namespace               |
| Pipeline de CI implantando em um namespace        | ServiceAccount + RoleBinding com <code>edit</code> no namespace | ClusterRoleBinding <code>cluster-admin</code> |
| Aplicação que não fala com a API                  | <code>automountServiceAccountToken: false</code>                | Deixar o token montado                        |
| Conferir o que alguém pode fazer                  | <code>kubectl auth can-i --list --as=...</code>                 | Testar na tentativa e erro                    |
| Leitura para um time de suporte                   | ClusterRole <code>view</code>                                   | Role com <code>*</code>                       |
| Token para um sistema externo                     | <code>kubectl create token</code> com validade curta            | Secret de token sem expiração                 |

# Helm

**Ideia central:** o **Helm** é o gerenciador de pacotes do Kubernetes. Um **chart** é um pacote de **templates** de manifestos mais um **values.yaml** com os valores padrão; cada instalação é uma **release** com histórico de **revisões**, que pode ser atualizada e revertida. A versão atual é o **Helm 4** (novembro de 2025); o Helm 3 recebe só correções de segurança até **10 de fevereiro de 2027**.

## 18.1 Instalar e usar charts

### 18.1.1 Instalação

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4 | bash
brew install helm # macOS
winget install Helm.Helm # Windows
helm version
```

### 18.1.2 Comandos do dia a dia

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm search repo prometheus # nos repositórios adicionados
helm search hub prometheus # no Artifact Hub
helm show values prometheus-community/kube-prometheus-stack > monitoramento-values.yaml

helm install monitoramento prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace -f monitoramento-values.yaml
helm install app oci://ghcr.io/minha-org/charts/app --version 1.4.0 # chart em registry OCI
helm upgrade --install app ./app -n loja -f valores-prod.yaml --set image.tag=1.4.2
helm list -A
helm status app -n loja
helm history app -n loja
helm rollback app 3 -n loja # volta para a revisão 3 (e cria uma nova revisão)
helm get values app -n loja # valores usados
helm get manifest app -n loja # manifestos aplicados
helm uninstall app -n loja
```

| Conceito    | Significado                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------------------|
| Chart       | O pacote (templates + valores padrão + metadados)                                                                         |
| Release     | Uma instalação de um chart, com nome, em um namespace                                                                     |
| Revisão     | Cada <code>install</code> , <code>upgrade</code> ou <code>rollback</code> cria uma nova, guardada em Secrets do namespace |
| Repositório | Onde os charts são publicados: HTTP (arquivo <code>index.yaml</code> ) ou <b>registry OCI</b> (o mesmo das imagens)       |

- **Precedência de valores:** `values.yaml` do chart < arquivos `-f` (na ordem) < `--set`. Guarde os valores de cada ambiente em arquivos versionados; `--set` é para pequenos ajustes.
- `helm upgrade --install` instala se não existir e atualiza se existir: é o comando típico de pipelines.
- **Mudanças do Helm 4:** aplica os manifestos com **server-side apply** nas releases novas; `--atomic` virou `--roll-back-on-failure` e `--force` virou `--force-replace`; post-renderers passaram a ser plugins; `helm registry login` recebe só o domínio. Scripts escritos para o Helm 3 podem precisar de ajustes.

## 18.2 Criar um chart

### 18.2.1 Estrutura

```
app/
├─ Chart.yaml           # nome, versão do chart, appVersion, dependências
├─ values.yaml          # valores padrão
├─ values.schema.json   # opcional: valida os valores (tipos, obrigatórios)
├─ charts/              # dependências empacotadas
├─ crds/                # CRDs instaladas antes dos templates
├─ templates/
│   ├─ _helpers.tpl     # templates nomeados (não gera manifesto)
│   ├─ deployment.yaml
│   ├─ service.yaml
│   ├─ NOTES.txt        # mensagem exibida depois do install
│   └─ tests/
```

```
# Chart.yaml
apiVersion: v2
name: app
description: Aplicação de exemplo
type: application
version: 0.3.0          # versão do CHART (SemVer); mude a cada alteração no chart
appVersion: "1.4.2"     # versão da APLICAÇÃO empacotada (informativa)
dependencies:
- name: redis
  version: "~2.1"
  repository: oci://registry.exemplo.com.br/charts # repositório ilustrativo
  condition: redis.enabled                        # dependência opcional
```

- `helm create app` gera um esqueleto completo (Deployment, Service, Ingress, HPA, ServiceAccount, helpers e testes), bom ponto de partida.
- `version` × `appVersion`: a primeira é a do pacote; a segunda, a do software dentro dele.

### 18.2.2 Templates

```
# values.yaml
replicaCount: 2
image:
  repository: ghcr.io/minha-org/app
  tag: "" # vazio = usar appVersion
service:
  type: ClusterIP
  port: 5000
resources:
  requests: { cpu: 250m, memory: 128Mi }
  limits: { memory: 256Mi }
env:
  REDIS_HOST: app-redis
```

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "app.fullname" . }}
  labels:
    {{- include "app.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount | default 1 }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "app.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          ports:
            - containerPort: {{ .Values.service.port }}
          env:
            {{- range $nome, $valor := .Values.env }}
            - name: {{ $nome }}
              value: {{ $valor | quote }}
            {{- end }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

| Recurso de template                                                                                                                                                   | Uso                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>.Values</code> , <code>.Release</code> ( <code>Name</code> , <code>Namespace</code> , <code>Revision</code> ), <code>.Chart</code> , <code>.Capabilities</code> | Objetos disponíveis nos templates                         |
| <code>{{- ... -}}</code>                                                                                                                                              | Remove espaços e quebras de linha antes/depois            |
| <code>if / else / with</code>                                                                                                                                         | Condicionais (e mudança de escopo com <code>with</code> ) |

| Recurso de template                                                | Uso                                                                                        |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>range</code>                                                 | Repetir para cada item de uma lista ou mapa ( <code>range \$chave, \$valor := ...</code> ) |
| <code>default</code> , <code>quote</code> , <code>required</code>  | Valor padrão, aspas, falha com mensagem se o valor faltar                                  |
| <code>toYaml</code> / <code>toJson</code> + <code>nindent N</code> | Inserir blocos inteiros (resources, labels) com a indentação certa                         |
| <code>include "nome" .</code>                                      | Chamar um template definido em <code>_helpers.tpl</code> (com <code>define</code> )        |
| <code>index .Values "chave-com-hifen"</code>                       | Acessar chaves que não são identificadores válidos                                         |
| <code>tpl</code>                                                   | Renderizar um valor que contém template                                                    |

- **Chaves com hífen** (`giropops-senhas`) não funcionam com a notação de ponto (`.Values.giropops-senhas`) e geram `bad character U+002D '-'`. Use **camelCase** nos valores (`giropopsSenhas`) ou `index`.
- **Helpers** (`_helpers.tpl`) centralizam nomes, labels e seletores. Arquivos que começam com `_` não geram manifestos.
- **Várias peças no mesmo arquivo**: separe com `---`. Um `range` que gera vários objetos precisa do `---` dentro do laço.
- **Segredos** não devem ficar em `values.yaml` versionado: use referências a Secrets existentes, External Secrets ou plugins como `helm-secrets`.

### 18.2.3 Validar e publicar

```
helm lint ./app # erros de estrutura e boas práticas
helm template app ./app -f valores-prod.yaml # renderiza localmente, sem cluster
helm install app ./app --dry-run=server # renderiza e valida no API server
helm dependency update ./app # baixa as dependências para charts/
helm package ./app # gera app-0.3.0.tgz
helm push app-0.3.0.tgz oci://ghcr.io/minha-org/charts
```

- **Confira a origem e a manutenção dos charts de terceiros**: em 2025 a Bitnami reduziu o catálogo gratuito de imagens e charts, e muitos tutoriais que usavam `bitnami/...` deixaram de funcionar como antes. Prefira charts mantidos pelos próprios projetos.
- **Repositórios OCI** (GHCR, ECR, Harbor, Docker Hub) são o modo recomendado de publicar charts. O modelo antigo, com `index.yaml` servido por HTTP (por exemplo, no GitHub Pages com `helm repo index`), continua funcionando.

## 18.3 Decisão rápida — Helm

| Situação                                                         | Abordagem recomendada                                                    | Erro comum                                              |
|------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------|
| Instalar software de terceiros (Redis, Prometheus, cert-manager) | Chart oficial ou mantido pelo projeto, com arquivo de valores versionado | Copiar manifestos soltos da internet                    |
| Mesma aplicação em vários ambientes                              | Um chart + um arquivo de valores por ambiente                            | Um chart por ambiente                                   |
| Deploy em pipeline                                               | <code>helm upgrade --install ... --rollback-on-failure</code> (Helm 4)   | <code>helm install</code> que falha na segunda execução |
| Upgrade quebrou a aplicação                                      | <code>helm rollback &lt;release&gt; &lt;revisão&gt;</code>               | Desinstalar e instalar de novo                          |
| Conferir o YAML antes de aplicar                                 | <code>helm template</code> ou <code>--dry-run=server</code>              | Aplicar direto em produção                              |

| Situação                     | Abordagem recomendada                              | Erro comum                               |
|------------------------------|----------------------------------------------------|------------------------------------------|
| bad character U+002D '-'     | Chaves em camelCase ou index                       | Tirar o hífen só de alguns lugares       |
| Publicar charts internos     | Registry OCI                                       | Enviar .tgz por e-mail                   |
| Senhas no values.yaml        | Secrets existentes, External Secrets, helm-secrets | Versionar as senhas                      |
| Scripts antigos com --atomic | --rollback-on-failure no Helm 4                    | Ficar no Helm 3 depois do fim do suporte |

## Pegadinhas e Informações Desatualizadas

**Como usar:** revise antes de uma prova (CKA, CKAD, KCNA), de uma entrevista ou de uma revisão de arquitetura. As afirmações falsas abaixo aparecem com frequência em tutoriais, cursos e até em manifestos que "funcionam".

### 19.1 Afirmações falsas clássicas

| Afirmação falsa                                                                     | O que é verdade                                                                                            |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| "O Kubernetes executa contêineres"                                                  | Ele agenda Pods; o kubelet pede a um runtime CRI (containerd, CRI-O) que os execute                        |
| "Kubernetes não roda imagens Docker desde a 1.24"                                   | Saiu o dockershim (Docker Engine como runtime); as imagens, que são OCI, continuam rodando                 |
| "O scheduler olha o uso real de CPU e memória"                                      | Ele decide pelos <b>requests</b> declarados                                                                |
| "O kube-proxy cria a rede dos Pods"                                                 | A rede dos Pods é do <b>CNI</b> ; o kube-proxy implementa os Services                                      |
| "livenessProbe que falha reinicia o Pod"                                            | Reinicia o <b>contêiner</b> ; o Pod (nome, IP, volumes) continua o mesmo                                   |
| "A readiness só roda na inicialização"                                              | Roda durante toda a vida do contêiner; falhou, sai dos endpoints do Service                                |
| "timeoutSeconds é o intervalo entre tentativas"                                     | É o prazo de <b>cada</b> verificação; o intervalo é <code>periodSeconds</code>                             |
| "Passar do limit de CPU mata o contêiner"                                           | CPU é estrangulada (throttling); só a <b>memória</b> acima do limite causa <code>OOMKilled</code>          |
| "128m de memória são 128 megabytes"                                                 | São 0,128 <b>byte</b> ; o certo é <code>128Mi</code> ou <code>128M</code>                                  |
| "Toleration faz o Pod ir para o nó com taint"                                       | Ela só <b>permite</b> ; para atrair, use <code>nodeSelector</code> ou <code>nodeAffinity</code>            |
| "Mudar o template de um ReplicaSet atualiza os Pods"                                | Só Pods novos usam o template; quem troca versões é o Deployment                                           |
| "Escalar um Deployment cria uma revisão"                                            | Só mudanças em <code>spec.template</code> criam revisões                                                   |
| "O Deployment faz rollback sozinho se a versão falhar"                              | Ele marca <code>ProgressDeadlineExceeded</code> ; o rollback é uma decisão sua                             |
| " <code>kubectl get all</code> mostra tudo"                                         | Mostra só alguns tipos; ConfigMaps, Secrets, Ingresses e PVCs ficam de fora                                |
| "Secret é criptografado"                                                            | É base64; a proteção vem de RBAC, criptografia em repouso e cofres externos                                |
| "ConfigMap montado nunca é atualizado no Pod"                                       | Arquivos montados são atualizados (exceto com <code>subPath</code> ); variáveis de ambiente não            |
| " <code>immutable: true</code> vai em <code>metadata</code> "                       | É um campo de primeiro nível do ConfigMap/Secret                                                           |
| "Existe <code>kubectl create daemonset</code> "                                     | Não existe; DaemonSets e StatefulSets vêm de manifestos                                                    |
| "Apagar o StatefulSet apaga os dados"                                               | Os PVCs ficam, a menos que a política de retenção diga o contrário                                         |
| "O volume de um StatefulSet se chama <code>&lt;template&gt;-&lt;índice&gt;</code> " | Se chama <code>&lt;template&gt;-&lt;statefulset&gt;-&lt;índice&gt;</code> (ex.: <code>dados-web-0</code> ) |
| "ExternalName aceita IP"                                                            | Aceita nome DNS (gera um CNAME); para IP, Service sem seletor + <code>EndpointSlice</code>                 |

| Afirmção falsa                                                                                                                 | O que é verdade                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| " <code>kubectl api-versions</code> mostra <code>networking.k8s.io/v1</code> , logo as <code>NetworkPolicies</code> funcionam" | A API sempre existe; quem aplica é o CNI                                                                              |
| " <code>NetworkPolicy</code> tem regras de negar"                                                                              | São listas de permissão que se somam; o bloqueio vem de o Pod estar selecionado                                       |
| "Dois <code>namespaceSelector</code> no mesmo item fazem um E"                                                                 | É uma chave YAML duplicada; o E é <code>namespaceSelector</code> + <code>podSelector</code> no mesmo item             |
| "Criar uma <code>ServiceAccount</code> gera um <code>Secret</code> com token"                                                  | Não desde a 1.24; tokens são temporários e emitidos sob demanda                                                       |
| "O Kubernetes tem objetos de usuário"                                                                                          | Usuários são identidades externas (certificado, OIDC); só <code>ServiceAccounts</code> são objetos                    |
| "O Metrics Server serve para monitoramento"                                                                                    | Ele só guarda o valor atual para <code>kubectl top</code> e autoescalamento                                           |
| "O HPA calcula a utilização sobre o limit"                                                                                     | Sobre o <b>request</b> ; sem request, o HPA não age                                                                   |
| "Antiafinidade obrigatória garante alta disponibilidade"                                                                       | Com mais réplicas que domínios, as sobras ficam <code>Pending</code> ; prefira <code>topologySpreadConstraints</code> |
| "4 nós de control plane são mais seguros que 3"                                                                                | O etcd com 4 membros tolera a perda de 1, como com 3; use número ímpar                                                |
| "Helm acessa <code>.Values.minha-chave</code> "                                                                                | Chaves com hífen exigem <code>index</code> ou <code>camelCase</code>                                                  |

## 19.2 Informações desatualizadas em materiais antigos

| Se o material diz...                                                                                            | Hoje (set/2026)                                                                                                |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Instalar kubelet/kubeadm por <code>apt.kubernetes.io/packages.-cloud.google.com</code> com <code>apt-key</code> | Repositórios <code>pkgs.k8s.io</code> por versão menor, chave em <code>/etc/apt/key-rings</code>               |
| Docker Engine como runtime do cluster, IDs <code>docker://</code> no <code>describe</code>                      | <code>containerd</code> ou CRI-O ( <code>containerd://</code> ); Docker só com <code>cri-dockerd</code>        |
| Clusters 1.23 a 1.29 nos exemplos                                                                               | Versões com suporte: 1.35, 1.36 e 1.37                                                                         |
| <code>kind v0.20</code> e imagens <code>kindest/node:v1.24</code>                                               | <code>kind v0.33</code> com imagens de versões atuais                                                          |
| Weave Net como CNI                                                                                              | Projeto sem manutenção; Calico, Cilium ou o CNI da nuvem                                                       |
| <b>ingress-nginx</b> como controlador padrão                                                                    | <b>Aposentado em março de 2026</b> ; Gateway API ou controlador mantido                                        |
| <code>autoscaling/v2beta2</code> no HPA                                                                         | <code>autoscaling/v2</code> (as versões beta foram removidas)                                                  |
| Provisionadores <code>kubernetes.io/aws-ebs</code> , <code>gce-pd</code> , <code>azure-disk</code>              | Drivers CSI ( <code>ebs.csi.aws.com</code> etc.)                                                               |
| Política de retenção <code>Recycle</code>                                                                       | Obsoleta; <code>Delete</code> ou <code>Retain</code>                                                           |
| Objeto <code>Endpoints</code>                                                                                   | Obsoleto desde a 1.33; <b>EndpointSlices</b>                                                                   |
| <code>PodSecurityPolicy</code>                                                                                  | Removida na 1.25; <b>Pod Security Admission</b>                                                                |
| <code>kubectl rollout ... --record</code>                                                                       | Obsoleto; anotação <code>kubernetes.io/change-cause</code>                                                     |
| Sidecars em <code>containers</code> sem controle de ordem                                                       | Sidecars nativos ( <code>init container</code> com <code>restartPolicy: Always</code> ), estáveis desde a 1.33 |
| Recriar o Pod para mudar CPU e memória                                                                          | Redimensionamento no lugar, estável desde a 1.35                                                               |
| External Secrets com <code>external-secrets.io/v1beta1</code>                                                   | <code>external-secrets.io/v1</code>                                                                            |
| Kyverno <code>ClusterPolicy</code> com <code>spec.validationFailureAction</code>                                | Tipos novos ( <code>ValidatingPolicy</code> etc.); no formato antigo, <code>failureAction</code> por regra     |

| Se o material diz...                                                              | Hoje (set/2026)                                                                                                                                   |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Helm 3 ( <code>get-helm-3</code> , <code>--atomic</code> , <code>--force</code> ) | <b>Helm 4</b> ( <code>get-helm-4</code> , <code>--rollback-on-failure</code> , <code>--force-replace</code> ); Helm 3 com patches só até fev/2027 |
| Charts e imagens <code>bitnami/...</code> gratuitos                               | Catálogo gratuito reduzido em 2025; confira a disponibilidade                                                                                     |
| Docker Hub limitado a 100/200 pulls "desde 2022"                                  | Limites mudam; confira a política atual e autentique os pulls                                                                                     |

### 19.3 Pares que mais se confundem

| Par                                                         | Diferença                                                                                                                                 |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Container engine × container runtime                        | O engine é a ferramenta completa do usuário (Docker, Podman); o runtime executa contêineres (containerd, CRI-O; runc no nível mais baixo) |
| OCI × CRI                                                   | OCI padroniza imagens e execução; CRI é a API entre o kubelet e o runtime                                                                 |
| <code>kubectl apply</code> × <code>create</code>            | <code>apply</code> cria ou atualiza; <code>create</code> falha se o objeto existir                                                        |
| <code>requests</code> × <code>limits</code>                 | Requests reservam e orientam o agendamento; limits são o teto de execução                                                                 |
| Liveness × readiness × startup                              | Reinicia contêiner × tira do tráfego × protege a inicialização                                                                            |
| Deployment × StatefulSet                                    | Réplicas intercambiáveis × identidade, ordem e PVC por réplica                                                                            |
| Deployment × DaemonSet                                      | N réplicas em qualquer nó × uma por nó                                                                                                    |
| RollingUpdate × Recreate                                    | Troca gradual, versões convivem × derruba tudo e sobe de novo                                                                             |
| <code>maxSurge</code> × <code>maxUnavailable</code>         | Pods a mais × Pods a menos durante o rollout                                                                                              |
| ClusterIP × NodePort × LoadBalancer                         | Interno × porta em todos os nós × balanceador externo                                                                                     |
| Service comum × headless                                    | IP virtual com balanceamento × DNS devolve os IPs dos Pods                                                                                |
| PV × PVC × StorageClass                                     | O volume × o pedido de volume × o "tipo" que cria volumes dinamicamente                                                                   |
| <code>Immediate</code> × <code>WaitForFirstConsumer</code>  | Cria o disco na hora × espera o Pod ser agendado (zona certa)                                                                             |
| RWO × RWX × RWOP                                            | Um nó × vários nós × um único Pod                                                                                                         |
| ConfigMap × Secret                                          | Configuração comum × dados sensíveis (tratamento e permissões diferentes)                                                                 |
| <code>data</code> × <code>stringData</code> (Secret)        | Base64 × texto que o API server converte                                                                                                  |
| Ingress × Gateway API                                       | API congelada com annotations por controlador × API padronizada com papéis separados                                                      |
| Issuer × ClusterIssuer                                      | Um namespace × todo o cluster                                                                                                             |
| HTTP-01 × DNS-01                                            | Arquivo no site × registro TXT (permite curinga)                                                                                          |
| Taint × toleration                                          | No nó, repele × no Pod, permite                                                                                                           |
| <code>nodeAffinity required</code> × <code>preferred</code> | Obrigatória × pontuação                                                                                                                   |
| <code>podAffinity</code> × <code>podAntiAffinity</code>     | Aproxima × afasta, por <code>topologyKey</code>                                                                                           |
| Role × ClusterRole                                          | Um namespace × cluster (ou modelo reutilizável)                                                                                           |
| RoleBinding × ClusterRoleBinding                            | Concede em um namespace × concede no cluster todo                                                                                         |
| HPA × VPA × Cluster Autoscaler                              | Número de Pods × tamanho dos Pods × número de nós                                                                                         |
| Chart × release                                             | O pacote × uma instalação dele                                                                                                            |

| Par                                                            | Diferença                             |
|----------------------------------------------------------------|---------------------------------------|
| <code>version</code> × <code>appVersion</code><br>(Chart.yaml) | Versão do chart × versão da aplicação |

## Referência Rápida de Comandos

**Como usar:** consulta de bancada. Com o alias `k=kubectl` e o autocompletar configurados, a maioria cabe em poucas teclas.

### 20.1 Contexto, descoberta e diagnóstico

| Comando                                                                                     | Para que serve                                      |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>kubectl config get-contexts / use-context / set-context --current --namespace=</code> | Clusters e namespace padrão                         |
| <code>kubectl api-resources / kubectl explain &lt;tipo&gt;.&lt;campo&gt;</code>             | Tipos, abreviações, grupos e documentação de campos |
| <code>kubectl get &lt;tipo&gt; -A -o wide</code>                                            | Listar em todos os namespaces, com detalhes         |
| <code>kubectl describe &lt;tipo&gt; &lt;nome&gt;</code>                                     | Configuração e eventos                              |
| <code>kubectl get events -A --sort-by=.lastTimestamp</code>                                 | Linha do tempo de eventos do cluster                |
| <code>kubectl logs &lt;pod&gt; [-c contêiner] [-f] [--previous]</code>                      | Logs                                                |
| <code>kubectl exec -it &lt;pod&gt; -- sh</code>                                             | Shell no contêiner                                  |
| <code>kubectl debug -it &lt;pod&gt; --image=busybox:1.37 --target=&lt;contêiner&gt;</code>  | Contêiner efêmero de depuração                      |
| <code>kubectl port-forward svc/&lt;nome&gt; 8080:80</code>                                  | Acessar um Service ou Pod da máquina local          |
| <code>kubectl top nodes / kubectl top pods -A</code>                                        | Consumo atual (Metrics Server)                      |
| <code>kubectl auth can-i --list --as=&lt;usuário&gt;</code>                                 | Permissões de uma identidade                        |

### 20.2 Criar e alterar

| Comando                                                                                             | Para que serve                |
|-----------------------------------------------------------------------------------------------------|-------------------------------|
| <code>kubectl apply -f &lt;arquivo ou diretório&gt; / kubectl diff -f</code>                        | Aplicar e comparar manifestos |
| <code>kubectl create deployment web --image=nginx:1.29 --replicas=3 --dry-run=client -o yaml</code> | Gerar o YAML de um Deployment |
| <code>kubectl run t -it --rm --image=busybox:1.37 --restart=Never -- sh</code>                      | Pod temporário para testes    |
| <code>kubectl expose deployment web --port=80 --target-port=8080 [--type=NodePort]</code>           | Criar um Service              |
| <code>kubectl create configmap / create secret generic tls docker-registry</code>                   | Configuração e segredos       |
| <code>kubectl scale deployment web --replicas=5</code>                                              | Escalar                       |
| <code>kubectl set image deployment/web nginx=nginx:1.29.1</code>                                    | Trocar a imagem               |
| <code>kubectl label / kubectl annotate (sufixo - remove; --overwrite troca)</code>                  | Labels e annotations          |
| <code>kubectl taint nodes &lt;nó&gt; chave=valor:NoSchedule[-]</code>                               | Aplicar/remover taints        |
| <code>kubectl delete -f &lt;arquivo&gt; / kubectl delete &lt;tipo&gt; &lt;nome&gt;</code>           | Remover                       |

## 20.3 Rollouts, nós e autoescalonamento

| Comando                                                                                                               | Para que serve                                               |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>kubectl rollout status history undo pause resume restart deployment/&lt;nome&gt;</code>                         | Ciclo de atualização (Deployments, DaemonSets, StatefulSets) |
| <code>kubectl autoscale deployment web --cpu-percent=60 --min=2 --max=10</code>                                       | Criar um HPA                                                 |
| <code>kubectl cordon uncordon &lt;nó&gt; / kubectl drain &lt;nó&gt; --ignore-daemonsets --delete-emptydir-data</code> | Manutenção de nós                                            |
| <code>kubeadm token create --print-join-command</code>                                                                | Novo comando de entrada de nós                               |
| <code>kubeadm certs check-expiration / kubeadm upgrade plan</code>                                                    | Certificados e atualização do cluster                        |

## 20.4 Ferramentas do ecossistema

| Comando                                                                                         | Para que serve                                       |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <code>kind create cluster --name x --config c.yaml / kind delete cluster --name x</code>        | Clusters locais com kind                             |
| <code>minikube start --nodes 3 -p x / minikube addons enable metrics-server</code>              | Clusters locais com Minikube                         |
| <code>helm upgrade --install &lt;release&gt; &lt;chart&gt; -n &lt;ns&gt; -f valores.yaml</code> | Instalar ou atualizar um chart                       |
| <code>helm list -A / helm history / helm rollback / helm uninstall</code>                       | Gerenciar releases                                   |
| <code>helm template / helm lint / helm package / helm push</code>                               | Desenvolver e publicar charts                        |
| <code>kubectl get certificate,order,challenge -A</code>                                         | Depurar certificados do cert-manager                 |
| <code>kubectl get policyreport -A</code>                                                        | Resultados de políticas do Kyverno em modo auditoria |

## Autoteste — Flashcards de Revisão

Cada card esconde a resposta: tente responder antes de abrir. Uma rodada de 20 cards por dia cobre o banco em pouco mais de uma semana.

### 21.1 Runtimes e arquitetura

Qual é a diferença entre container engine e container runtime?

#### RESPOSTA

O **engine** (Docker, Podman) é a ferramenta completa que o usuário opera: imagens, redes, volumes, CLI. O **runtime** executa contêineres: os de alto nível (containerd, CRI-O) gerenciam imagens e ciclo de vida; os de baixo nível (runc, crun) criam namespaces e cgroups.

O que o dockershim removido na 1.24 mudou para quem usa imagens Docker?

#### RESPOSTA

Nada nas imagens: elas seguem o padrão OCI e rodam em containerd ou CRI-O. Mudou apenas que o Docker Engine deixou de ser runtime dos nós sem o adaptador cri-dockerd.

O que é a CRI?

#### RESPOSTA

A **Container Runtime Interface**, API gRPC pela qual o kubelet pede ao runtime (containerd, CRI-O) que crie sandboxes, baixe imagens e inicie contêineres.

Qual componente é o único que fala diretamente com o etcd?

#### RESPOSTA

O **kube-apiserver**. Todos os outros componentes, e o kubectl, passam por ele.

O que o kube-scheduler faz, e com base em quê?

#### RESPOSTA

Escolhe o nó de cada Pod sem nó definido: filtra os nós viáveis (requests, taints, afinidade, volumes) e pontua os restantes. Usa os **requests** declarados, não o consumo real.

Qual é o papel do kube-controller-manager?

RESPOSTA

Executar os controladores (Deployment, ReplicaSet, Node, Job, EndpointSlice...), que observam o estado desejado e agem para que o estado real chegue a ele.

Por que um cluster de produção usa 3 nós de control plane?

RESPOSTA

Para o etcd manter quórum com a perda de um membro:  $N$  membros toleram  $(N - 1) / 2$  falhas. Com 2 ou 4, a tolerância não melhora.

Quais portas o API server, o kubelet e os NodePorts usam por padrão?

RESPOSTA

API server **6443**, kubelet **10250**, NodePort **30000–32767** (TCP e UDP). O etcd usa 2379–2380.

Por que os nós ficam `NotReady` logo depois do `kubeadm init`?

RESPOSTA

Porque ainda não há **plugin CNI** instalado; sem rede de Pods, o kubelet não reporta o nó como pronto.

Qual é a regra de diferença de versão entre `kubectl` e o cluster?

RESPOSTA

No máximo **uma versão menor** acima ou abaixo (um `kubectl` 1.37 fala com 1.36, 1.37 e 1.38).

## 21.2 kubectl e primeiros passos

Como gerar o YAML de um Deployment sem criá-lo?

RESPOSTA

```
kubectl create deployment web --image=nginx:1.29 --dry-run=client -o yaml > deploy.yaml.
```

Qual é a diferença entre `kubectl apply -f` e `kubectl create -f`?

RESPOSTA

`apply` cria ou atualiza o objeto; `create` só cria e falha com `AlreadyExists` se ele já existir.

Por que `kubectl expose pod web` pode falhar com "couldn't find port"?

**RESPOSTA**

Porque o Pod não declara `containerPort` e o comando não recebeu `--port`: o Kubernetes não sabe qual porta expor.

Como acessar um Service ClusterIP a partir da sua máquina?

**RESPOSTA**

Com `kubectl port-forward svc/<nome> <porta-local>:<porta-do-service>`. O ClusterIP só é alcançável de dentro do cluster.

O que `kubectl get all` deixa de fora?

**RESPOSTA**

Vários tipos, como ConfigMaps, Secrets, Ingresses, PVCs e ServiceAccounts. Ele lista só alguns tipos de workload e Services.

21.3

## Pods

O que os contêineres de um mesmo Pod compartilham?

**RESPOSTA**

O IP e o namespace de rede (falam por `localhost`), os volumes declarados no Pod e o nó onde rodam.

O que acontece com um Pod criado diretamente, sem controlador, se o nó falhar?

**RESPOSTA**

Nada o recria: ele simplesmente deixa de existir. Por isso aplicações rodam em Deployments, StatefulSets, DaemonSets ou Jobs.

Como declarar um sidecar nativo e o que ele garante?

**RESPOSTA**

Como init container com `restartPolicy: Always`. Ele inicia antes dos contêineres principais, roda junto com eles e só é encerrado depois deles (estável desde a 1.33).

Qual é a diferença entre `kubectl exec` e `kubectl attach`?

**RESPOSTA**

`exec` cria um processo novo no contêiner (ex.: um shell); `attach` se conecta ao STDIN/STDOUT do processo principal, sem criar processo.

### Como ler os logs de um contêiner que está em CrashLoopBackOff?

#### RESPOSTA

`kubectl logs <pod> --previous`, que mostra a saída da execução anterior, a que falhou.

### O que significa `OOMKilled` com código 137?

#### RESPOSTA

O contêiner passou do **limit de memória** e foi morto pelo kernel.

### O que acontece quando um contêiner atinge o limit de CPU?

#### RESPOSTA

Ele é **estrangulado** (throttling): fica mais lento, mas não é morto.

### Quanto é `500m` de CPU e qual a diferença entre `128Mi` e `128M`?

#### RESPOSTA

`500m` é meio núcleo (0,5 CPU). `128Mi` =  $128 \times 1024^2$  bytes; `128M` =  $128 \times 1000^2$  bytes.

### Quais são as classes de QoS e qual é despejada primeiro?

#### RESPOSTA

**Guaranteed** (requests = limits em todos os contêineres), **Burstable** e **BestEffort** (sem requests nem limits). **BestEffort** é despejada primeiro.

### Como aumentar a CPU de um Pod em execução sem recriá-lo?

#### RESPOSTA

Com o subrecurso `resize` (estável desde a 1.35): `kubectl patch pod <pod> --subresource=resize -p '{...}'`.

### Quanto tempo vive um `emptyDir` e para que ele serve?

#### RESPOSTA

O mesmo que o Pod (sobrevive a reinícios de contêiner, não à remoção do Pod). Serve para compartilhar arquivos entre contêineres do Pod, cache e temporários.

### Quanto tempo o kubelet espera entre o SIGTERM e o SIGKILL?

#### RESPOSTA

`terminationGracePeriodSeconds`, **30 s** por padrão.

## 21.4 Deployments, ReplicaSets e DaemonSets

Qual é a relação entre Deployment, ReplicaSet e Pod?

### RESPOSTA

O Deployment gerencia ReplicaSets (um por versão do template); cada ReplicaSet mantém um número de Pods idênticos.

O que cria uma nova revisão em um Deployment?

### RESPOSTA

Mudanças em `spec.template` (imagem, variáveis, recursos). Escalar não cria revisão.

Quais são os padrões de `maxSurge` e `maxUnavailable`?

### RESPOSTA

**25%** cada. `maxSurge` arredonda para cima e `maxUnavailable` para baixo; os dois não podem ser 0.

Com 10 réplicas, `maxSurge: 1` e `maxUnavailable: 2`, quais são os limites durante o rollout?

### RESPOSTA

No máximo **11** Pods ao todo e no mínimo **8** disponíveis.

Quando usar a estratégia Recreate?

### RESPOSTA

Quando duas versões não podem rodar ao mesmo tempo (esquema incompatível, volume RWO compartilhado), aceitando indisponibilidade.

Como registrar o motivo de uma revisão hoje?

### RESPOSTA

Com a anotação `kubernetes.io/change-cause`; a flag `--record` está obsoleta.

O que o `kubectl rollout restart` faz?

### RESPOSTA

Altera uma anotação de data no template, disparando um rollout que recria os Pods sem mudar a imagem.

Por que não se deve criar ReplicaSets diretamente?

RESPOSTA

Porque o ReplicaSet não atualiza os Pods existentes quando o template muda; não há rollout nem rollback.

Como fazer um DaemonSet rodar também nos nós de control plane?

RESPOSTA

Adicionando uma toleration para o taint `node-role.kubernetes.io/control-plane:NoSchedule`.

Quais são as estratégias de atualização de um DaemonSet?

RESPOSTA

RollingUpdate (padrão, `maxUnavailable: 1`) e OnDelete (só recria os Pods que você apagar).

21.5

## Probes

O que acontece quando cada tipo de probe falha?

RESPOSTA

**Liveness:** o contêiner é reiniciado. **Readiness:** o Pod sai dos endpoints do Service. **Startup:** após o limite, o contêiner é reiniciado; enquanto ela não passa, as outras não rodam.

Quais são os padrões de `periodSeconds`, `timeoutSeconds` e `failureThreshold`?

RESPOSTA

10 s, 1 s e 3.

Por que a liveness não deve checar o banco de dados?

RESPOSTA

Se o banco cair, todos os contêineres reiniciam em cadeia sem resolver nada. Dependências vão, no máximo, na readiness.

Como proteger uma aplicação que demora 2 minutos para iniciar?

RESPOSTA

Com uma **startupProbe** cujo `failureThreshold × periodSeconds` seja maior que o tempo de inicialização.

## 21.6 Cluster, armazenamento e Services

O que são os Pods estáticos do kubeadm?

### RESPOSTA

Manifestos em `/etc/kubernetes/manifests` que o kubelet executa diretamente: API server, scheduler, controller-manager e etcd.

Como gerar um novo comando de `kubeadm join` depois que o token expirou?

### RESPOSTA

`kubeadm token create --print-join-command` no control plane. O token padrão vale 24 h.

O que significa `volumeBindingMode: WaitForFirstConsumer`?

### RESPOSTA

O disco só é criado e vinculado quando um Pod que usa o PVC é agendado, garantindo que fique na mesma zona do nó.

Qual é a diferença entre as reclaim policies `Delete` e `Retain`?

### RESPOSTA

`Delete` apaga o PV e o disco quando o PVC é removido; `Retain` mantém o PV (`Released`) e os dados para recuperação manual.

Quais são os modos de acesso de um volume?

### RESPOSTA

RWO (um nó), ROX (leitura em vários nós), RWX (leitura e escrita em vários nós) e RWOP (um único Pod).

O que substituiu os provisionadores `kubernetes.io/aws-ebs` e similares?

### RESPOSTA

Os **drivers CSI** (`ebs.csi.aws.com`, `pd.csi.storage.gke.io`, `disk.csi.azure.com`).

Como o Service descobre para quais Pods mandar tráfego?

### RESPOSTA

Pelo seletor de labels; o controlador grava os Pods **prontos** em `EndpointSlices`, e o kube-proxy programa as regras.

Qual é o nome DNS completo do Service `redis` no namespace `loja`?

**RESPOSTA**

`redis.loja.svc.cluster.local`.

O que é um Service headless?

**RESPOSTA**

Um Service com `clusterIP: None`: sem IP virtual, o DNS devolve os IPs dos Pods. É usado por StatefulSets para dar nome a cada réplica.

Como o LoadBalancer funciona fora de nuvem?

**RESPOSTA**

Precisa de uma implementação como MetalLB (bare metal), cloud-provider-kind (kind) ou `minikube tunnel`; sem ela, o IP externo fica `<pending>`.

Quais garantias um StatefulSet dá?

**RESPOSTA**

Nomes fixos e ordenados, DNS por réplica via Service headless, um PVC por réplica que acompanha o Pod, e criação, remoção e atualização em ordem.

Como testar uma versão nova em só algumas réplicas de um StatefulSet?

**RESPOSTA**

Com `updateStrategy.rollingUpdate.partition`: só Pods com índice maior ou igual ao valor são atualizados.

## 21.7 Configuração, entrada de tráfego e certificados

Base64 protege um Secret?

**RESPOSTA**

Não. É só codificação. A proteção vem de RBAC, criptografia em repouso no etcd e de não versionar os valores.

Quando um ConfigMap alterado chega ao Pod?

**RESPOSTA**

Arquivos montados como volume são atualizados em até cerca de um minuto (exceto com `subPath`); variáveis de ambiente só mudam com a recriação do Pod.

### Como usar uma imagem de registry privado?

#### RESPOSTA

Criando um Secret `docker-registry` e referenciando-o em `imagePullSecrets` do Pod ou da ServiceAccount.

### Quais são as chaves de um Secret do tipo `kubernetes.io/tls` ?

#### RESPOSTA

`tls.crt` e `tls.key`.

### O que o External Secrets Operator faz?

#### RESPOSTA

Busca segredos em cofres externos (Vault, AWS Secrets Manager, Key Vault...) e cria e atualiza Secrets do Kubernetes a partir deles, por meio de `SecretStore` / `ClusterSecretStore` e `ExternalSecret`.

### O que aconteceu com o ingress-nginx?

#### RESPOSTA

O projeto da comunidade Kubernetes foi aposentado em março de 2026: sem novas versões nem correções de segurança. A recomendação é migrar para a Gateway API ou outro controlador mantido.

### Quais são os recursos principais da Gateway API?

#### RESPOSTA

**GatewayClass** (implementação), **Gateway** (pontos de entrada, mantidos pela plataforma) e **HTTPRoute/GRPCRoute** (rotas, mantidas pelas aplicações).

### Por que uma regra de Ingress com `- host:` e `- http:` em itens separados é um problema?

#### RESPOSTA

Porque vira uma regra sem caminhos e outra sem host, que responde para qualquer nome.

### Qual desafio ACME permite certificados curinga?

#### RESPOSTA

O **DNS-01**. O HTTP-01 não emite curingas.

### Por que testar o cert-manager com o ambiente staging do Let's Encrypt?

#### RESPOSTA

Porque a produção tem limites de emissão; erros repetidos bloqueiam novos certificados por um tempo.

**Qual é a fórmula do HPA?****RESPOSTA**

$\text{réplicas desejadas} = \lceil \text{réplicas atuais} \times (\text{métrica atual} \div \text{alvo}) \rceil$ , com tolerância de 10% e janela de estabilização de 300 s para reduzir.

**Por que um HPA mostra `<unknown>` em TARGETS?****RESPOSTA**

Falta o Metrics Server ou os contêineres não têm **requests** do recurso usado na métrica.

**O que o KEDA faz que o HPA sozinho não faz?****RESPOSTA**

Escala a partir de eventos externos (filas, Kafka, cron) e pode reduzir a zero réplicas.

**Quais são os efeitos de um taint?****RESPOSTA**

`NoSchedule` (não agenda novos), `PreferNoSchedule` (evita) e `NoExecute` (não agenda e despeja os que não toleram).

**Por que um Pod leva cerca de 5 minutos para ser recriado quando o nó dele morre?****RESPOSTA**

Porque os Pods recebem tolerations automáticas de 300 s para os taints `not-ready` e `unreachable`.

**Como reservar nós de GPU só para cargas de GPU?****RESPOSTA**

Taint nos nós de GPU (afasta os outros) + toleration e `nodeAffinity/nodeSelector` nos Pods de GPU (atrai estes).

**Por que preferir `topologySpreadConstraints` à antiafinidade obrigatória?****RESPOSTA**

Porque espalha réplicas sem travar o escalonamento: a antiafinidade `required` deixa `Pending` as réplicas que excedem o número de domínios.

### Para que serve um PodDisruptionBudget?

#### RESPOSTA

Limitar quantos Pods de uma aplicação podem ser removidos voluntariamente ao mesmo tempo (drain, atualizações de nós).

### Quais são os níveis do Pod Security Admission?

#### RESPOSTA

privileged, baseline e restricted, aplicados por labels no namespace nos modos enforce, warn e audit.

### O que é a ValidatingAdmissionPolicy?

#### RESPOSTA

Um mecanismo nativo (estável desde a 1.30) de validação na admissão com regras em CEL, sem webhook.

### Quais funções o Kyverno oferece?

#### RESPOSTA

Validar, modificar, gerar e apagar recursos e verificar imagens. Na versão 1.19, os tipos novos (ValidatingPolicy, MutatingPolicy ...) substituem o ClusterPolicy, que ficou obsoleto.

21.9

## Network Policies, RBAC e Helm

### Qual é o comportamento padrão da rede de Pods sem NetworkPolicies?

#### RESPOSTA

Todo Pod pode falar com todo Pod. Um Pod passa a ser isolado em uma direção quando alguma política o seleciona nessa direção.

### Qual é a diferença entre namespaceSelector e podSelector em itens separados ou no mesmo item?

#### RESPOSTA

Itens separados são **OU**; no mesmo item, significam "Pods com estas labels **nos** namespaces com estas labels" (**E**).

### O que costuma quebrar depois de uma política "negar tudo" com Egress?

#### RESPOSTA

O **DNS**. É preciso liberar a saída para o CoreDNS em UDP e TCP 53.

### Como o Kubernetes identifica um usuário autenticado por certificado?

#### RESPOSTA

Pelo CN (nome do usuário) e pelos O (grupos) do certificado assinado pela CA do cluster.

### Qual é a diferença entre RoleBinding e ClusterRoleBinding?

#### RESPOSTA

O RoleBinding concede uma Role ou ClusterRole dentro de um namespace; o ClusterRoleBinding concede uma ClusterRole no cluster inteiro.

### Como verificar as permissões de uma ServiceAccount?

#### RESPOSTA

```
kubectl auth can-i --list -n <ns> --as=system:serviceaccount:<ns>:<nome>.
```

### Por que dar permissão de criar Pods é sensível?

#### RESPOSTA

Porque quem cria Pods pode montar qualquer Secret e usar qualquer ServiceAccount do namespace.

### Como obter um token para uma ServiceAccount hoje?

#### RESPOSTA

`kubectl create token <sa> --duration=1h`, que emite um token temporário. Desde a 1.24, não são mais criados Secrets de token automaticamente.

### Qual é a diferença entre chart e release no Helm?

#### RESPOSTA

O chart é o pacote; a release é uma instalação dele, com nome, namespace e histórico de revisões.

### Qual é a ordem de precedência dos valores no Helm?

#### RESPOSTA

values.yaml do chart < arquivos -f (na ordem) < --set .

### Como acessar no template um valor cuja chave tem hífen?

#### RESPOSTA

Com `index .Values "minha-chave"`, ou renomeando a chave para camelCase.

### Como conferir o YAML que um chart vai gerar sem instalá-lo?

#### RESPOSTA

`helm template <release> <chart> -f valores.yaml` (ou `helm install --dry-run=server` para validar no cluster).

### Quais flags mudaram de nome no Helm 4?

#### RESPOSTA

`--atomic` virou `--rollback-on-failure` e `--force` virou `--force-replace`.

## Síntese de revisão

Este guia percorre o Kubernetes da arquitetura do cluster à entrega de aplicações com Helm, reunindo formatos de consulta rápida e prática ativa.

- O Kubernetes é declarativo: o API server guarda o estado desejado no etcd, e controladores, scheduler e kubelet trabalham em laços para alcançá-lo. O runtime (containerd ou CRI-O) executa os contêineres pela CRI (capítulos 1 a 3).
- Aplicações rodam em Pods criados por controladores: Deployments para réplicas sem estado, StatefulSets para identidade e dados, DaemonSets para agentes de nó. Requests, limits e probes definem onde e como elas rodam (capítulos 4 a 7 e 10).
- Volumes com CSI e StorageClasses, Services com DNS, ConfigMaps e Secrets e a entrada de tráfego pela Gateway API (ou por um Ingress mantido) com cert-manager completam a aplicação (capítulos 8 a 12).
- HPA, regras de agendamento, políticas de admissão, Network Policies e RBAC sustentam a operação segura; o Helm empacota tudo em charts versionados (capítulos 13 a 18).
- As pegadinhas, a referência de comandos e o autoteste (capítulos 19 a 21) oferecem revisão rápida, com 87 perguntas. Versões e comportamentos foram conferidos em setembro de 2026.